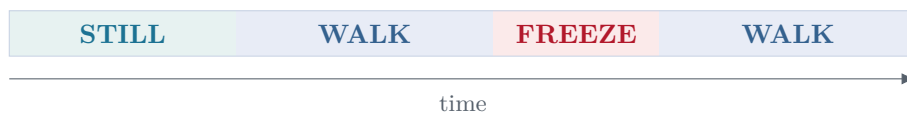

Cadence

A Wearable Freeze-of-Gait Monitor
for Parkinson's Disease

Technical Report • How It Works, End to End



ENGF0031 Scenario 2 • “Smart Clothing”

Every quantitative result in this report is **real** — computed from on-body captures, the public Daphnet benchmark, and the project’s own analysis code. The device is a **monitor**: it detects and displays freezes; the fitted vibration motor is disabled in firmware.

Contents

1	Aim & Clinical Motivation	4
1.1	Freeze of gait: feet glued to the floor	4
1.2	Why a cheap, continuous wearable matters	4
1.3	Cadence: the aim, stated precisely	5
1.4	A monitor, not a therapy	5
1.5	What a freeze looks like in the data	5
1.6	What this report covers	6
2	System Overview: The End-to-End Pipeline	7
2.1	The journey of one window	7
2.2	Two detectors, one question	8
2.3	Three boards, three jobs	9
3	Sensing: The Accelerometer & Data Acquisition	10
3.1	What a MEMS accelerometer actually measures	10
3.2	Range: why $\pm 8g$ and not $\pm 2g$	11
3.3	Sampling rate, Nyquist, and why 64Hz is plenty	12
3.4	Units and the gravity offset	13
3.5	Drift-free acquisition on the SAMD21	13
4	Acceleration Magnitude	14
4.1	Why one scalar, not three	14
4.2	Removing the mean: motion, not posture	15
4.3	A worked example	16
5	Fourier Analysis: From Time to Frequency	17
5.1	Any signal is a sum of sinusoids	18
5.2	The Discrete Fourier Transform and its bins	18
5.3	Power, leakage, and the Hann window	19
5.4	Welch's method: a steadier spectrum	20
6	Band Power & the Freeze Index	21
6.1	From a spectrum to a band power	22
6.2	The Freeze Index: a ratio of two bands	22
6.3	What it does on real data	23
6.4	A scalar the state machine can act on	24
7	Real-Time Windowing	25
7.1	The streaming geometry	25
7.2	Why 4s? Spectral resolution versus latency	26
7.3	The ring buffer	27
7.4	The cost of windowing: dilution at the edges	28
8	The Shipped Detector: Threshold, Gate, Debounce	29
8.1	Why a gate comes first: the still-leg problem	30
8.2	The threshold: 1.815 versus the textbook optimum 2.10	30
8.3	Seeing the rule on real data	32
8.4	Debounce: don't trust a one-window verdict	33
8.5	Putting it together: the state machine	33

9	The CNN: FoGNet Architecture	34
9.1	Why a CNN, and why 1-D	34
9.2	What one convolution actually does	35
9.3	Stacking blocks: wider receptive fields	35
9.4	FoGNet, end to end	35
9.5	Global average pooling: a freeze anywhere counts	36
9.6	How it is trained and how it scores	37
10	Training the CNN: Loss, Gradient Descent & Generalisation	38
10.1	From a raw score to a probability: the sigmoid	38
10.2	The loss: punishing confident mistakes	38
10.3	Gradient descent: rolling the weights downhill	39
10.4	Over-fitting and the regularisers	40
10.5	Evaluating without fooling ourselves: Leave-One-Subject-Out	41
10.6	Calibration: do the probabilities mean what they say?	42
11	Measuring Accuracy: Confusion, Sensitivity & Specificity	43
11.1	The confusion matrix: four ways a window can be scored	43
11.2	Four rates, and why accuracy is the wrong one	44
11.3	The real numbers: FoGNet under leave-one-subject-out	45
11.4	Sweeping the trip line: ROC, PR and Youden's J	46
12	Interpretability: Opening the Box	47
12.1	The interpretable alternative: a 9-feature RandomForest	48
12.2	Permutation importance: break a feature, watch the score fall	48
12.3	SHAP: why <i>this</i> window was called a freeze	49
12.4	Partial dependence: which way does the feature push?	50
12.5	The threshold, not the model, sets the trade-off	51
12.6	Why Cadence ships the Freeze Index, not a black box	51
13	Hardware	52
13.1	The one board that does the work: Circuit Playground Express	52
13.2	Pins and connections	53
13.3	Telling the wearer: NeoPixels and the OLED	55
13.4	Going wireless: the optional 3-board topology	55
13.5	Wearing it: the ankle sleeve	56
13.6	Two firmware roles on one board	56
13.7	Does it fit on the chip?	57
14	Software	59
14.1	Architecture at a glance	59
14.2	The signal-processing core (dsp)	60
14.3	Streaming: from ankle to ring buffer	61
14.4	The dashboard server's control loop	62
14.5	The offline data pipeline	62
14.6	Design lesson: the live detector reads raw acceleration	63
15	The Live Dashboard: What Every Metric Means	63
15.1	The screen at a glance	64
15.2	The two quantities everything is built from	64
15.3	Panel by panel	65
15.4	Reading a real run off the screen	66

16 Results, Limitations, Future Work & Conclusion	67
16.1 What works — the headline numbers	68
16.2 Limitations — where Cadence falls short	70
16.3 Future work	71
16.4 Conclusion	72
Appendix A: Signal-processing parameters	72
Appendix B: Glossary	73

1 Aim & Clinical Motivation

Parkinson’s disease (PD) is a progressive disorder of the brain’s movement-control circuitry. As the dopamine-producing neurons of the *substantia nigra* die off, the messages that normally let movement flow smoothly become noisy and unreliable. The familiar outward signs — a resting tremor, stiffness, and a general slowing of movement — are only part of the picture. For many people in the middle and later stages of the disease, the most frightening symptom is one that the textbook descriptions barely capture: their feet simply stop.

1.1 Freeze of gait: feet glued to the floor

Freeze of gait (FoG) is a sudden, brief inability to take the next step. People who live with it describe the sensation vividly — as though their feet have been *glued to the floor* — even while the upper body still wants to move forward. The intention to walk is intact; the legs will not answer. Episodes are typically short, lasting only seconds, and they are often triggered by exactly the moments where a person most needs their feet: turning around, stepping through a narrow doorway, starting to walk from standing, or approaching a chair.

A crucial and counter-intuitive detail is that a freeze is *not* the same as standing still. During a freeze the feet are usually trembling and shuffling rapidly *in place* — a fast, small-amplitude tremor of roughly 3–8 Hz — while the body fails to make forward progress. This rapid trembling-in-place is the tell-tale mechanical signature that an instrument can latch onto, and it is what distinguishes a genuine freeze from someone calmly waiting at a kerb. It is also the single most important fact behind every design decision in this report.

Intuition

A freeze is not a pause. When you stand at rest, your feet are quiet. During a freeze the feet are *busy* — buzzing in place at 3–8 Hz — yet you go nowhere. “Quiet feet, no progress” is rest; “busy feet, no progress” is a freeze. That contrast is exactly what an ankle accelerometer can hear.

Why it is dangerous. Freezing is one of the leading causes of falls in Parkinson’s disease. When the feet stop but the body’s momentum does not, the person pitches forward with nothing to catch them. The consequences cascade far beyond the fall itself. A single fracture can mean surgery, a long recovery, and a lasting loss of confidence; the fear of the *next* fall then shrinks a person’s world. Many people respond by walking less, going out less, and leaning more heavily on carers — so freezing quietly erodes the very independence that mobility represents. Because episodes are brief and unpredictable, they are also hard to capture in a clinic: a patient may walk perfectly down a corridor for a neurologist and freeze repeatedly at home an hour later. What the clinician needs, and rarely gets, is an honest record of *when* and *how often* freezing actually happens during ordinary life.

Key idea

Freezing is brief, unpredictable, and hard to reproduce on demand — which is exactly why a short clinic visit underestimates it. The clinical value of a wearable is not that it does anything clever in the moment; it is that it watches *continuously* and turns scattered, invisible episodes into a record a clinician can act on.

1.2 Why a cheap, continuous wearable matters

If freezing only mattered in the consulting room, an expensive lab gait mat would be enough. But freezing happens at home, in the supermarket, in the doorway at night — precisely where no instrument is watching. A wearable changes that arithmetic. It is worn all day, it costs little, and

it converts a symptom that is otherwise reported from fading memory into objective, time-stamped data. A low-cost device can be given to many patients at once; a continuous one captures the rare episodes that a snapshot misses. Together, “cheap” and “continuous” are what make population-scale, real-life freeze monitoring realistic rather than aspirational.

1.3 Cadence: the aim, stated precisely

Cadence is a low-cost, ankle-worn wearable whose aim is narrow and deliberate: to *faithfully detect* freeze-of-gait episodes from a *single ankle accelerometer*, and to *intuitively display* them — on the device itself and on a live dashboard — so that the person and their clinician can see when freezing happened. It is built on an Adafruit Circuit Playground Express, worn in a soft ankle sleeve, and it does its sensing and decision-making from the three axes of one tiny accelerometer. Two verbs carry the whole project: *detect* and *display*.

Takeaway

Cadence has exactly one job: to *watch* the ankle, recognise the 3–8 Hz signature of a freeze, and *show* it clearly. Detect, and display — nothing more.

1.4 A monitor, not a therapy

This positioning must be crisp, because it is easy to get wrong. There is a well-known class of Parkinson’s aids called *cueing therapies*: devices that, when they sense a freeze, fire a stimulus — a rhythmic buzz, a beep, a flash — to help the person break out of it and step again. Cueing is an *intervention*: it acts *on* the patient in the moment.

Cadence is not one of these. It is a *monitor*. Its purpose is to observe and record freezing faithfully, not to interrupt or treat it. A coin vibration motor is physically fitted to the board, but it is *deliberately disabled in firmware*: the hardware exists, but no firmware path ever drives it. When Cadence recognises a freeze it *reports* that fact — on an OLED screen, on the onboard NeoPixel LED ring, and on a live web dashboard — and it does *not* buzz, cue, or otherwise act on the wearer. The distinction between observing and intervening is the line Cadence is careful never to cross. Table 1 makes the contrast explicit.

Table 1. Cadence is a monitor, not a cueing therapy. The two aim at different goals and are evaluated by different yardsticks.

	Cueing therapy	Cadence (monitor)
Goal	Help end a freeze <i>now</i>	Record <i>when</i> freezes happen
Acts on patient	Yes — fires a cue	No — only reports
On a freeze	Buzz / beep / flash to cue	Show on OLED + LEDs + dashboard
Coin motor	Driven	Fitted but <i>disabled</i>
Success means	Patient steps again	Faithful, honest detection

1.5 What a freeze looks like in the data

Before diving into signal processing in later sections, it helps to see, once, what Cadence is actually looking at. Figure 1 sketches the canonical sequence Cadence is built to follow: a calm *still* period, ordinary *walking*, a sudden *freeze*, and a return to walking. The accelerometer trace tells the story in amplitude alone — the freeze is the burst of fast, jagged activity sandwiched between two stretches of smoother motion.

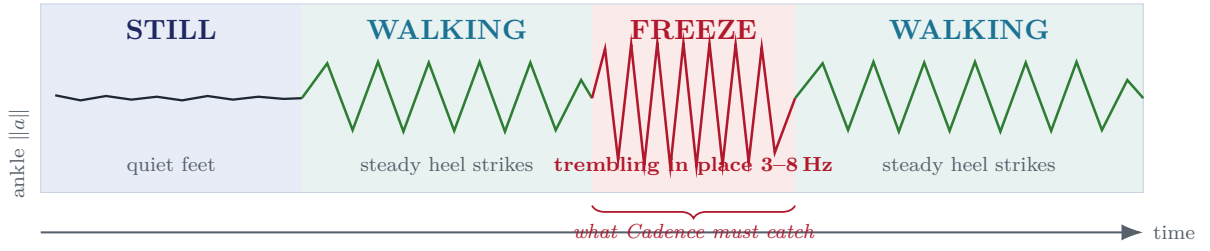


Figure 1. The canonical gait sequence Cadence is built to follow. Calm *still* periods and ordinary *walking* both produce orderly traces; a *freeze* appears as a burst of fast, large-amplitude trembling “in place” before walking resumes. The red episode — not the quiet stretches — is what the device must detect.

That schematic is not just an illustration; it matches what the hardware actually records. Figure 2 shows the central decision quantity — the *Freeze Index*, the ratio of fast (3–8 Hz) to slow (0.5–3 Hz) energy in the ankle signal — measured on a real worksheet capture that runs through exactly this still → walk → freeze → walk sequence. Standing still and ordinary walking both sit around a Freeze Index of roughly 1.1 and 1.2; the freeze phase leaps to a mean of 6.89, far above the deployed trip line of 1.815. The full mechanics of how that index is computed, thresholded, and debounced are the subject of later sections; here it is enough to see that the freeze genuinely stands apart from everything around it.

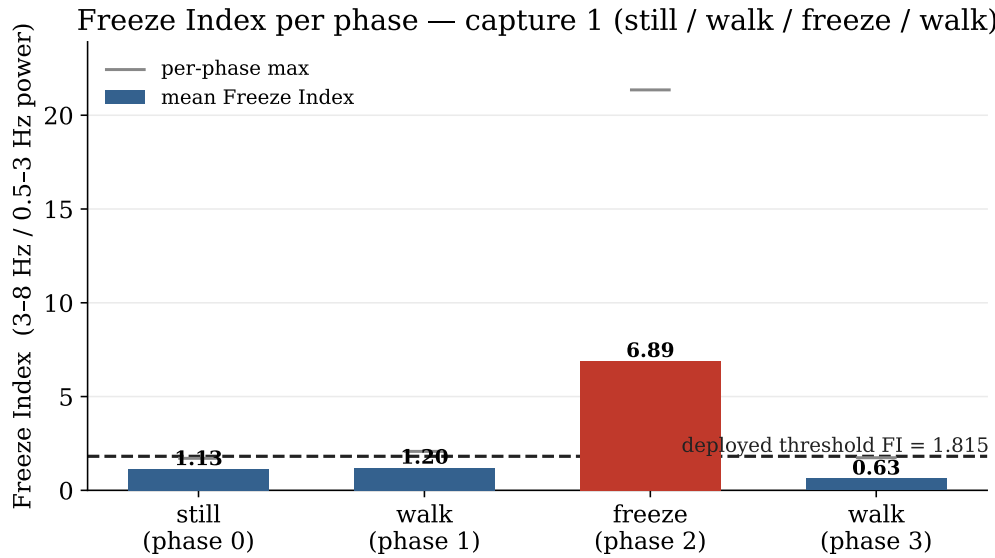


Figure 2. Real Freeze Index per phase on the worksheet capture (*capture1*, 64 Hz, 41.4 s). Still and walking phases stay near 1.1–1.2; the freeze phase jumps to a mean of 6.89, well clear of the deployed threshold of 1.815 (dashed). The ordered gait sequence of Figure 1 is visible directly in the data.

Key idea

The freeze does not hide. On real data the Freeze Index of a freeze (≈ 6.9) is several times that of still or walking (≈ 1.1 – 1.2). Cadence’s job is to draw a sensible line between them and to report — never to interfere.

1.6 What this report covers

The remainder of this report follows Cadence from physical signal to clinical readout.

- **Sensing.** The accelerometer, why it is sampled at 64 Hz over ± 8 g, and how three axes are fused into one orientation-invariant magnitude.

- **Signal processing.** Welch power spectra, the locomotor / freeze / rest-tremor frequency bands, and the Freeze Index that turns a wiggling trace into a single number.
- **The shipped detector.** The transparent “glass-box” rule — an energy gate, a Freeze-Index threshold, and a debounce filter — and how its operating point was chosen to favour catching freezes.
- **A learned alternative.** *FoGNet*, a small 1-D convolutional network explored as a “given more time” upgrade, with its honest cross-validated results.
- **Evaluation.** Why sensitivity and specificity, not raw accuracy, are the metrics that matter when freezes are rare.
- **Hardware & software.** The Circuit Playground build, the live dashboard, and the firmware that records and displays — but, by design, never cues.

Takeaway

Read the rest of this report through one lens: Cadence is a *faithful witness* to freezing, not a treatment for it. Every choice that follows — sensor, maths, detector, display — serves the single goal of detecting freezes honestly and showing them clearly.

2 System Overview: The End-to-End Pipeline

Before we dissect any single stage, it helps to see the whole machine at once. Cadence takes one stream of numbers — the wobble of a single accelerometer strapped to the ankle — and turns it, every two seconds, into one of three plain-English words: **STILL**, **WALKING**, or **FREEZE**. That verdict is shown on the body (a small OLED and a ring of coloured LEDs) and, optionally, streamed over Wi-Fi to a live laptop dashboard. Everything else in this report is an expansion of one of the boxes in Figure 44.

Key idea

Cadence is a **monitor**, not a therapy. It *detects* and *displays* freeze-of-gait episodes so they can be logged and reviewed; it never tries to cue, buzz, or treat the wearer. A coin vibration motor is physically present on the build but is **disabled in firmware** — the only outputs are visual and on-screen.

2.1 The journey of one window

The pipeline is a straight line with a single branch. Signal flows left to right; the two analysis paths in the middle answer the same question — *is this a freeze?* — in two very different ways.

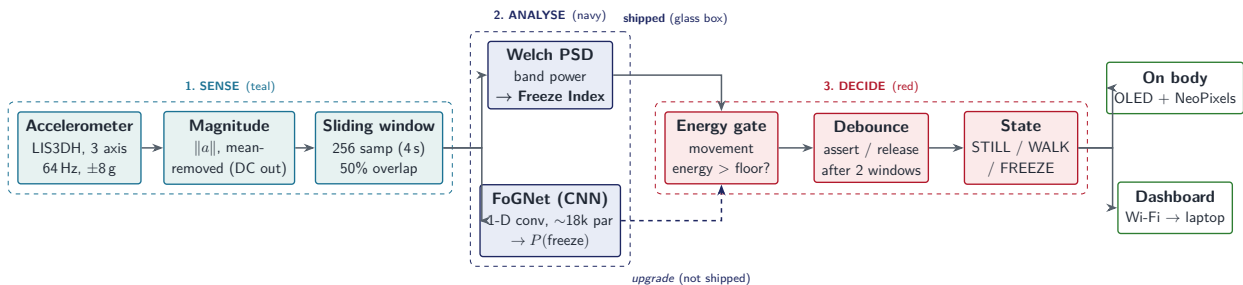


Figure 3. The Cadence pipeline, end to end. One accelerometer stream is reduced to a STILL/WALKING/FREEZE verdict every 2s. Stages are colour coded: **sensing** (teal), **analysis** (navy), **decision** (red). The *shipped* detector is the glass-box Freeze Index (solid path); the 1-D CNN FoGNET is a drop-in *upgrade* (dashed) explored later, not on the deployed board.

Reading Figure 44 from left to right:

1. **Sense.** The onboard LIS3DH MEMS accelerometer samples three axes at 64 Hz over ± 8 g. The wide range matters: a heel-strike momentarily exceeds ± 2 g, so a ± 2 g part would clip. The Nyquist frequency of 32 Hz comfortably covers every band we care about (the highest, freeze, tops out at 8 Hz).
2. **Magnitude.** The three axes collapse into one orientation-invariant scalar $\|a\| = \sqrt{a_x^2 + a_y^2 + a_z^2}$; subtracting its mean removes gravity and any DC offset, leaving pure motion to analyse.
3. **Window.** The stream is chopped into overlapping 256-sample (4 s) windows with a 128-sample (2 s) hop. Fifty-percent overlap yields one fresh decision every 2 s.
4. **Analyse (the branch).** Each window is scored by either the shipped *Freeze Index* — a ratio of spectral band powers — or, as an upgrade, the FOGNET CNN that reads the raw window directly. Both answer the same yes/no question.
5. **Decide.** A movement-energy *gate* forces STILL when the leg is barely moving (so noise cannot masquerade as a freeze); a two-window *debounce* stops a single bump from flipping the verdict; the result is the committed state.
6. **Display.** The state lights the body-worn OLED and NeoPixel ring and, on the optional networked build, drives the live laptop dashboard.

Intuition

The whole device is one long funnel. A 3-axis, 64-per-second firehose of numbers is squeezed, stage by stage, down to a single word twice a second. Each stage throws away information that does not help answer “are the feet trembling in place?” — first *direction* (magnitude), then *time detail* (the spectrum), then *ambiguity* (the gate and debounce).

2.2 Two detectors, one question

The single most important thing to understand about Cadence is that the box labelled **Analyse** contains two interchangeable answers to the same question, and the report treats them very differently.

- The **Freeze Index (FI)** is the *shipped* detector — a transparent “glass box”. It is the ratio of energy in the freeze band (3 Hz to 8 Hz) to energy in the locomotor band (0.5 Hz to 3 Hz). When the feet shuffle in place, the freeze band lights up and the ratio jumps. Every step of this calculation is inspectable, which is exactly why it runs on the board.
- FOGNET, a small 1-D **convolutional neural network** (~ 18 k parameters), is the “given more time” *upgrade*. It learns its own features from the raw 3×256 window. It is **not** on the deployed device; it appears later as a benchmark of what a learned model buys (and costs) over the glass box.

Takeaway

The FI path is what actually ships and runs on the ankle; the CNN is an offline comparison. Wherever this report says “the detector” without qualification, it means the glass-box Freeze Index.

Figure 4 shows the pipeline’s raw input and its headline statistic on a real 41.4 s capture. The signal is calm during standing and walking, erupts into rapid trembling during the freeze phase, and the Freeze Index — computed exactly as the device computes it — tracks that eruption faithfully.

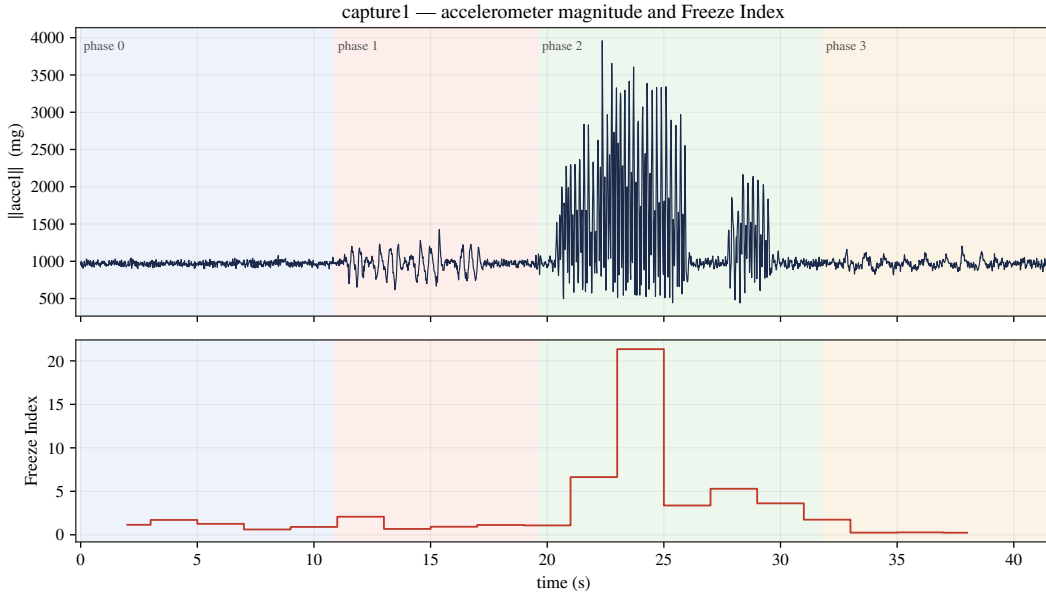


Figure 4. Real input and output of the analysis stage on `capture1.csv` (64Hz, 41.4s). *Top:* the mean-removed accelerometer magnitude through four labelled phases. The freeze phase (centre) shows the tell-tale rapid 3 Hz to 8 Hz trembling in place. *Bottom:* the Freeze Index over the same window; it stays near 1 during stillness and walking and spikes above 20 during the freeze, far past the 1.815 trip line.

2.3 Three boards, three jobs

Cadence runs untethered, but the optional networked build spreads the work across three physical boards (Table 2). The simplest deployment is just the first board: the Circuit Playground Express senses, runs the on-board FI detector, and drives its own OLED and LEDs — no laptop required. Adding the ESP32 and a laptop only changes *where* the verdict is displayed, never how it is computed.

Board	Role	Job in the pipeline
CPX	Sensor + detector (always present)	Samples the LIS3DH at 64 Hz, computes magnitude, the Freeze Index, the gate and debounce; drives the OLED + NeoPixels.
ESP32	Wi-Fi bridge (optional)	Relays the stream over UART → Wi-Fi/TCP to the laptop. Carries data only; makes no decision.
Laptop	Dashboard host (optional)	Runs the asyncio + WebSocket server that renders the live display panels and, offline, trains/evaluates FOGNET.

Table 2. The three boards. Only the **CPX** is required; the freeze decision always happens there. The ESP32 and laptop add a live remote dashboard but do not move the detector off the ankle.

A matching split exists in firmware. One role, the *recorder* (`cpx_fog_logger.ino`), logs auto-labelled accelerometer data to the CPX’s 2MB flash for later analysis; the other, the *monitor* (`cpx_fog_standalone.ino`), runs the live OLED-and-LED freeze display. The figures of real data throughout this report — like Figure 4 — come from the recorder; the device the reader would wear runs the monitor.

Key idea

The decision lives on the ankle. Whether or not a laptop is in the loop, the CPX alone turns raw acceleration into a FREEZE verdict and shows it on the body. The rest of the report simply opens up each box in Figure 44: the spectral maths (§2 → Fourier), the Freeze Index and its

threshold, the decision logic, and the CNN upgrade.

3 Sensing: The Accelerometer & Data Acquisition

Everything Cadence knows about a wearer’s gait arrives through a single channel: the motion of one ankle, measured many times a second. Before any spectral analysis, freeze index, or neural network can run, that motion must be turned into clean, evenly spaced numbers. This section explains the sensor that does it — the LIS3DH MEMS accelerometer on the Circuit Playground Express (CPX) — and the disciplined, drift-free loop on the SAMD21 microcontroller that samples it. The choices here (which axes, what range, how fast, what units) set the ceiling on everything downstream, so they deserve to be understood from first principles.

3.1 What a MEMS accelerometer actually measures

A MEMS (Micro-Electro-Mechanical System) accelerometer is, quite literally, a microscopic spring–mass system etched into silicon. A tiny proof mass is suspended on flexible silicon beams that behave as springs. When the chip accelerates, the mass lags behind by inertia and the beams flex; that deflection is read out *capacitively*. The moving mass carries interleaved fingers that sit between fixed fingers, forming a pair of capacitors. As the mass shifts by a few nanometres, one gap narrows and the other widens, the capacitances change in opposite directions, and the chip’s analogue front end converts that differential capacitance into a digital number proportional to acceleration along that axis.

Intuition

Picture a ball on a spring inside a sealed box. Shake the box and the ball swings; how far it swings tells you how hard you shook. A MEMS accelerometer is that toy shrunk onto a grain of silicon, with the “how far” read not by eye but by measuring a capacitance. Crucially, it cannot tell “being pushed” from “resting in gravity”: sitting still on a table, the proof mass still sags under its own weight, so the sensor always reports a steady $\approx 1\text{ g}$ pull. That ever-present offset is a feature we exploit later, not a fault.

The LIS3DH integrates three such structures at right angles, giving acceleration along three orthogonal axes a_x, a_y, a_z simultaneously. We mount it in an ankle sleeve, so the axes are fixed to the leg rather than to the world. Figure 5 shows the convention: one axis runs roughly vertically (and so carries most of gravity when standing), one along the direction of travel, and one across the body. Because a sleeve can be donned at slightly different angles each day, we never trust any single axis on its own — the orientation is not guaranteed. The remedy is the magnitude $a = \sqrt{a_x^2 + a_y^2 + a_z^2}$, introduced in the next section, which is identical no matter how the device is rotated.

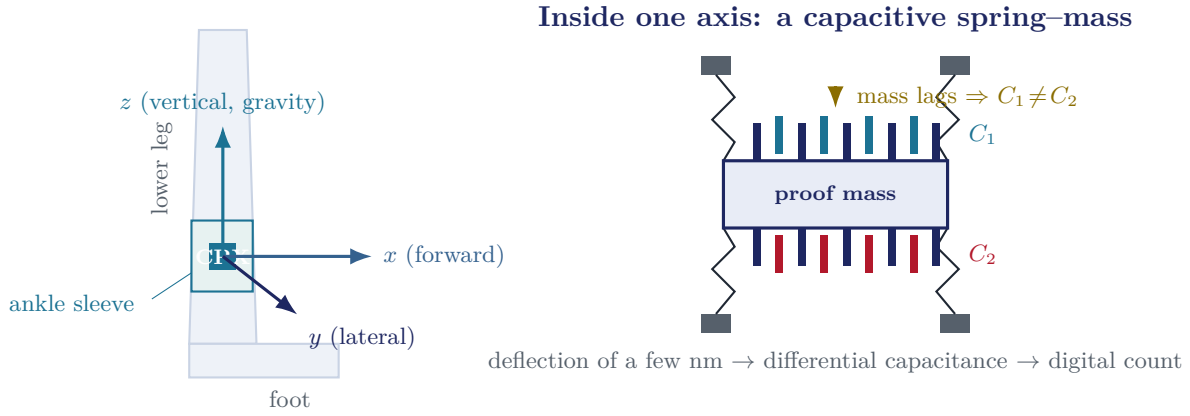


Figure 5. Left: the LIS3DH is worn in an ankle sleeve, fixing its three orthogonal axes to the leg — z vertical (and so loaded by gravity at rest), x forward, y lateral. Right: along each axis a silicon proof mass on spring beams deflects under acceleration, unbalancing a pair of capacitors C_1, C_2 ; that differential capacitance is digitised. Because the sleeve’s orientation is not guaranteed day to day, Cadence never trusts a single axis and instead works from the rotation-invariant magnitude.

3.2 Range: why $\pm 8g$ and not $\pm 2g$

The LIS3DH can be configured for several full-scale ranges. A smaller range gives finer resolution per count, which is tempting — but a range that is too small *clips*: any acceleration beyond the limit is recorded as a flat-topped maximum, destroying exactly the peaks we care about. Walking is not gentle. Each *heel strike* sends a sharp impulse up the leg that comfortably exceeds $\pm 2g$. If we ran at $\pm 2g$, those impacts would saturate the sensor, flattening the very spikes that distinguish vigorous gait from a freeze. We therefore run the LIS3DH at $\pm 8g$, which leaves clean headroom above every heel strike.

Takeaway

A heel strike exceeds $\pm 2g$, so the small range would clip. Cadence runs at $\pm 8g$ — never trading dynamic range for resolution when the peaks are the signal.

Figure 6 is real data from the worksheet capture (`capture1.csv`, 64 Hz, 41.4 s). The top trace is the acceleration magnitude in milli- g . Two facts jump out and tie directly to the choices in this section. First, the baseline sits near 1000 mg even when the wearer is still — that is the $\approx 1g$ of gravity the proof mass always feels, the DC offset we remove before spectral analysis. Second, during the freeze phase the magnitude swings to roughly 3500 mg, far above 2000 mg: had the range been $\pm 2g$ those excursions would have been sheared flat at the rail.

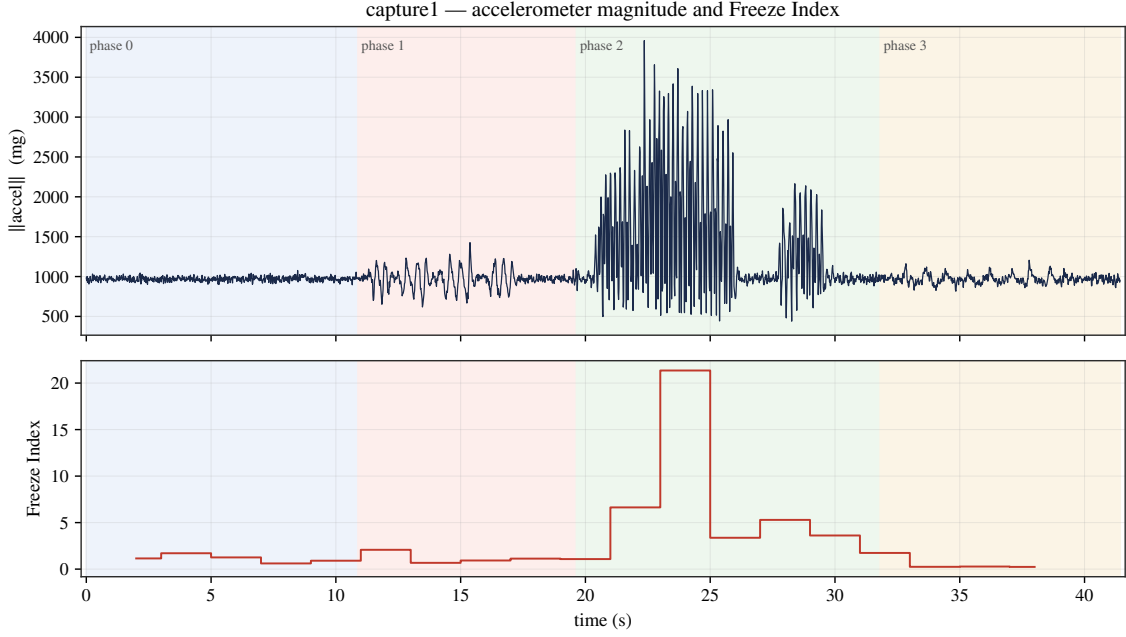


Figure 6. Real 64 Hz capture (`capture1.csv`, 41.4 s). Top: the acceleration magnitude $\|a\|$ in milli- g , sitting on a ≈ 1000 mg gravity baseline; the freeze phase drives it well past 2000 mg, which a ± 2 g range would clip. Bottom: the corresponding Freeze Index, computed downstream from this same magnitude (Section 3 feeds it). The sensor’s job is to deliver these excursions *without saturating*; the analysis that turns them into a verdict comes later.

3.3 Sampling rate, Nyquist, and why 64 Hz is plenty

Cadence samples all three axes at $f_s = 64$ Hz — 64 fresh readings per second. The right way to judge whether that is fast enough is the Nyquist–Shannon sampling theorem: a sampled signal can faithfully represent frequencies only up to half the sampling rate. Here the Nyquist frequency is $f_s/2 = 32$ Hz. Every frequency band Cadence cares about sits far below that ceiling: the locomotor (walking-rhythm) band at 0.5 Hz to 3 Hz, the freeze band at 3 Hz to 8 Hz, and the rest-tremor band at 4 Hz to 6 Hz. The highest of these tops out at 8 Hz, four times below Nyquist (Figure 7). There is comfortable margin, and no aliasing of the freeze signature into a misleading frequency.

The maths, unpacked

Nyquist, in one line. To capture a frequency f without aliasing you must sample faster than twice it:

$$f_s > 2f_{\max} \quad \Longleftrightarrow \quad f_{\max} < \frac{f_s}{2} = \frac{64 \text{ Hz}}{2} = 32 \text{ Hz}. \quad (1)$$

Our highest band of interest is 8 Hz. Since $8 < 32$ with a factor of four to spare, 64 Hz represents the 3 Hz to 8 Hz freeze trembling exactly. Sample *too slowly* — say below 16 Hz — and an 8 Hz tremor would fold down (*alias*) to a lower, fictitious frequency and corrupt the Freeze Index. 64 Hz avoids that entirely while keeping data rates and on-board compute modest.

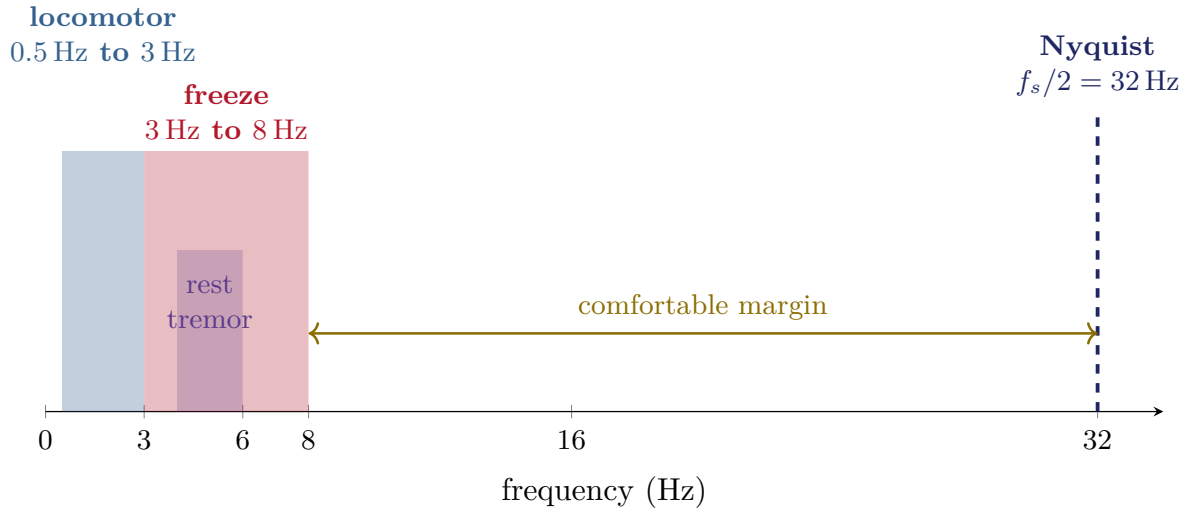


Figure 7. Where the action is, against the sampling ceiling. All three bands Cadence analyses lie below 8 Hz; the Nyquist frequency for 64 Hz sampling is 32 Hz, leaving a four-fold margin so nothing aliases. (The rest-tremor band 4 Hz to 6 Hz sits inside the freeze band.)

3.4 Units and the gravity offset

On the device the SAMD21 works in SI units, m/s^2 , because that is what the LIS3DH driver returns. On the host side we convert to milli- g ($1 \text{ g} \approx 9.81 \text{ m/s}^2$, so $1 \text{ mg} \approx 9.81 \times 10^{-3} \text{ m/s}^2$) because milli- g is the natural language of human movement: a still limb reads $\approx 1000 \text{ mg}$, a brisk heel strike a few thousand. Whichever unit is used, one preprocessing step is non-negotiable. The proof mass always feels gravity, so every axis carries a constant $\approx 1 \text{ g}$ bias — a strong DC component. Before any spectral analysis we subtract the mean of the magnitude window, removing that DC term so the power spectrum reflects *movement*, not the constant pull of the Earth.

Key idea

Gravity is a DC offset, not motion. Subtracting the window mean from the magnitude zeroes the $\approx 1000 \text{ mg}$ baseline, so the spectrum that feeds the Freeze Index measures how the leg is *shaking*, not the fact that it is on a planet. Because the Freeze Index is later formed as a ratio of band powers, the choice of unit (mg versus g) cancels — but the DC removal does not, and must happen first.

3.5 Drift-free acquisition on the SAMD21

A spectral analysis is only as honest as its time axis. If samples arrive at uneven intervals — because the firmware did some I2C work, drew the OLED, or updated the NeoPixels in between — the assumed 64 Hz grid is a lie, and a 6 Hz tremor can smear into the wrong bin. The classic beginner’s mistake is to sample, then `delay(15)` (roughly $1/64 \text{ s}$): the delay does not account for the time the sampling and processing themselves took, so the true period creeps and *drifts*.

Cadence instead uses fixed-rate, non-blocking scheduling. It records the timestamp of the next sample that is *due* and advances that target by a fixed period each time, rather than sleeping for a fixed period. The loop never blocks: between samples it is free to run the on-board detector, refresh the OLED, and drive the LED ring, yet a sample is taken the instant its scheduled deadline passes. Errors do not accumulate, because each deadline is computed from the previous *target*, not from when the previous sample actually fired.

```
1 // Fixed-rate, non-blocking 64 Hz sampler on the SAMD21 (sketch)
2 const uint32_t PERIOD_US = 1000000UL / 64; // 15625 us between samples
3 uint32_t next_due = micros(); // first deadline = now
4
```

```

5 void loop() {
6   if ((int32_t)(micros() - next_due) >= 0) { // deadline passed?
7     next_due += PERIOD_US; // advance by a FIXED step
8     float ax, ay, az;
9     lis3dh_read(&ax, &ay, &az); // m/s2, +/-8 g range
10    push_sample(ax, ay, az); // into the ring buffer
11  }
12  service_detector(); // FI / gate / debounce -- never blocks
13  service_display(); // OLED + NeoPixel ring -- never blocks
14 }

```

Intuition

Think of it as catching a train by the clock, not by a stopwatch. “Sleep 15 ms” restarts the stopwatch *after* you have already spent time reading the sensor and drawing the screen, so each cycle runs a little long and the schedule slips. “Take a sample whenever the wall clock crosses the next 15.625 ms tick” keeps every sample pinned to an absolute deadline — the timing never drifts, no matter how much work happens in between.

Because the advance is by a fixed `PERIOD_US` computed from the target rather than the actual fire time, the long-run average rate stays locked to 64 Hz and the time base under the Welch analysis is trustworthy. The cast to `int32_t` in the comparison makes the deadline test robust to the microsecond counter wrapping around. Two firmware roles share this same acquisition core: the *recorder* (`cpx_fog_logger.ino`) streams samples to the CPX’s 2 MB flash for later analysis, and the *monitor* (`cpx_fog_standalone.ino`) feeds them straight into the live detector and display. The sensing front end is identical in both; only what happens to the samples afterwards differs.

Takeaway

One ankle accelerometer, sampled at a rock-steady 64 Hz over ± 8 g, delivers gravity-baselined, alias-free motion. Get the range, rate, and timing right here and every downstream stage — magnitude, spectrum, Freeze Index, dashboard — inherits clean data; get them wrong and no clever analysis can recover the lost peaks.

4 Acceleration Magnitude

The accelerometer hands us three numbers every 15.6 ms: a_x , a_y , a_z , the projection of acceleration onto the board’s own three axes. Those three numbers are a poor place to start a freeze detector, because they describe how the *board* is oriented as much as how the *leg* is moving. Before we compute a single spectrum we collapse the triplet into one orientation-invariant scalar, the *magnitude*, and then strip its mean. This short chapter justifies both moves and walks a worked example through them.

Intuition

Tape the Circuit Playground onto an ankle and the x -axis might point forward today and slightly skyward tomorrow, depending on how the sleeve was pulled on. A freeze — a 3 Hz to 8 Hz tremble of the foot — is the *same physical event* either way, but its energy would land in different axes. We want a number that asks “how hard is the ankle shaking?” without ever asking “which way is the board facing?” The length of the acceleration vector answers exactly that.

4.1 Why one scalar, not three

An accelerometer measures a vector, $\mathbf{a} = (a_x, a_y, a_z)$. Rotating the sensor on the ankle applies a rotation R to that vector: the components a_x, a_y, a_z change, but the underlying physical acceleration

is unchanged. Any per-axis feature inherits the sensor’s pose and therefore drifts as the sleeve shifts. The Euclidean length, by contrast, is a rotational invariant – $\|R\mathbf{a}\| = \|\mathbf{a}\|$ for every rotation R – so it reports the strength of the acceleration regardless of how the board sits. That is the whole motivation. We define the magnitude

$$a = \|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2}, \quad (2)$$

a single non-negative time series sampled, like its three parents, at 64 Hz. Everything downstream – the Welch spectrum, the band powers, the FI ratio – runs on this one channel.

Key idea

The magnitude $a = \sqrt{a_x^2 + a_y^2 + a_z^2}$ is **orientation-invariant**: rotate the board however you like and a is unchanged. A freeze that shows up as 3 Hz to 8 Hz energy is detected the same way whether the sleeve was pulled on neatly or twisted, which is precisely the robustness an ankle-worn wearable needs.

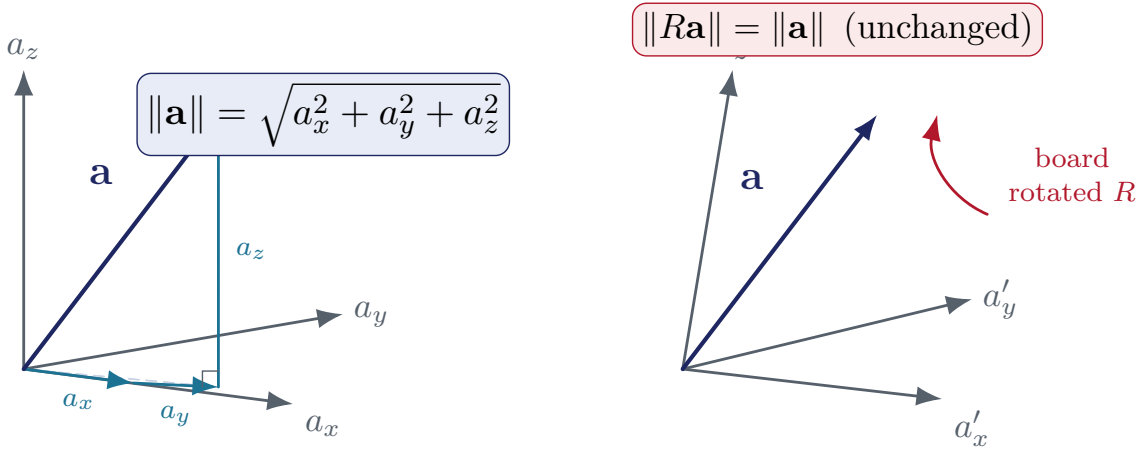


Figure 8. The acceleration vector $\mathbf{a}$ and its Euclidean norm. Left: the three axis components combine, by Pythagoras in 3-D, into one length $\|\mathbf{a}\|$. Right: rotating the board on the ankle changes the components a_x, a_y, a_z but leaves the length untouched – the property that makes the magnitude a safe foundation for the detector.

The maths, unpacked

Equation (2) is just Pythagoras in three dimensions. The length of a vector is the square root of the sum of its squared components, so it cannot depend on the *labels* we attach to the axes – only on the vector itself. Squaring also makes the result blind to sign: a downward jolt ($a_z < 0$) and an upward one of equal size contribute identically. The magnitude is therefore a non-negative “how much acceleration, in total” signal, and it discards the directional information we deliberately do not want.

4.2 Removing the mean: motion, not posture

There is a second problem the raw magnitude does not solve. A still ankle is not at zero – it sits in Earth’s gravity, so $\|\mathbf{a}\| \approx 1g$ ($\approx 9.81 \text{ m/s}^2$) even when nothing moves. That constant is a large DC offset. If we fed it straight into a spectrum it would dominate the lowest bins and swamp the small 3 Hz to 8 Hz wiggle we are hunting for. Worse, the exact value of that offset depends on *posture* – how the foot is tilted – which is again something we want the detector to ignore.

The fix is to subtract the window mean before any spectral analysis:

$$\tilde{a}[n] = a[n] - \bar{a}, \quad \bar{a} = \frac{1}{N} \sum_{n=0}^{N-1} a[n]. \quad (3)$$

This kills the DC term (the 0 Hz bin) and, with it, the gravity plateau and any slow posture drift. What survives is the *fluctuation* of the magnitude about its own average – the part caused by the leg actually moving. Because the Welch spectrum and the band powers all operate on $\tilde{a}$, every later quantity describes motion rather than orientation.

Takeaway

Magnitude removes *direction*; mean-subtraction removes *the static pull of gravity and posture*. What is left – $\tilde{a}[n]$ – is pure ankle motion, the only thing the freeze detector should ever react to.

4.3 A worked example

Take one sample with components (in g) $a_x = 0.20$, $a_y = -0.30$, $a_z = 0.96$ – a foot mostly upright with a small forward-and-sideways shove. Its magnitude is

$$a = \sqrt{0.20^2 + (-0.30)^2 + 0.96^2} = \sqrt{0.04 + 0.09 + 0.9216} = \sqrt{1.0516} \approx 1.025 \text{ g}.$$

Now rotate the board so the same physical acceleration reads $a'_x = 0.96$, $a'_y = 0.20$, $a'_z = -0.30$ instead. The components are scrambled, yet

$$a' = \sqrt{0.96^2 + 0.20^2 + (-0.30)^2} = \sqrt{0.9216 + 0.04 + 0.09} = \sqrt{1.0516} \approx 1.025 \text{ g},$$

exactly the same length – a numerical demonstration of the invariance in Figure 8. Finally, over a short window suppose the magnitude takes the three values 1.02, 1.04, 1.00 (g). Their mean is $\bar{a} = 1.02$, so the mean-removed signal is $\tilde{a} = (0, +0.02, -0.02)$: a tiny zero-centred wiggle. That wiggle – not the $\approx 1 \text{ g}$ baseline – is what the spectrum sees.

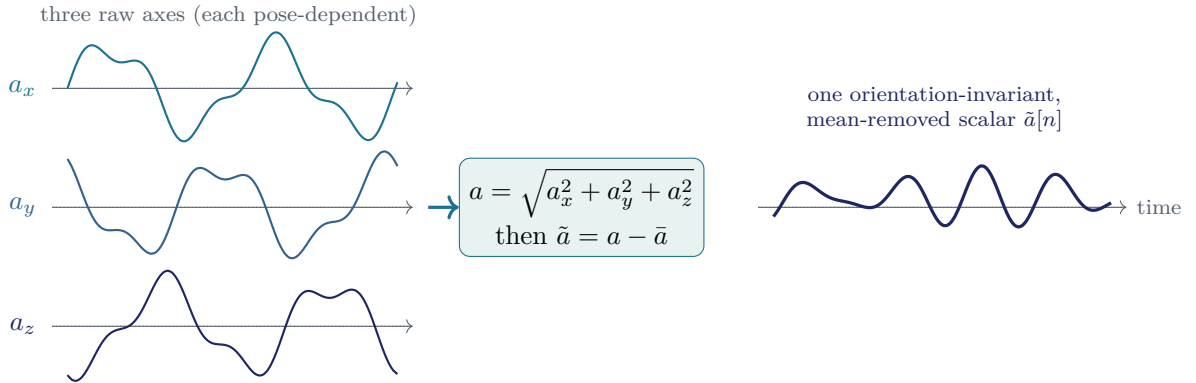


Figure 9. The collapse, schematically: three pose-dependent axis traces (left) are combined sample-by-sample into a single magnitude by Eq. (2), then mean-centred by Eq. (3), yielding one orientation-invariant scalar $\tilde{a}[n]$ (right). The detector’s spectrum, band powers and FI all run on this single channel.

Figure 10 shows the same pipeline on real data: the worksheet capture (capture1.csv, 64 Hz, 41.4 s). The freeze segment is visibly a small, fast tremor riding on the magnitude trace – exactly the fluctuation that mean-subtraction isolates and the spectrum then quantifies.

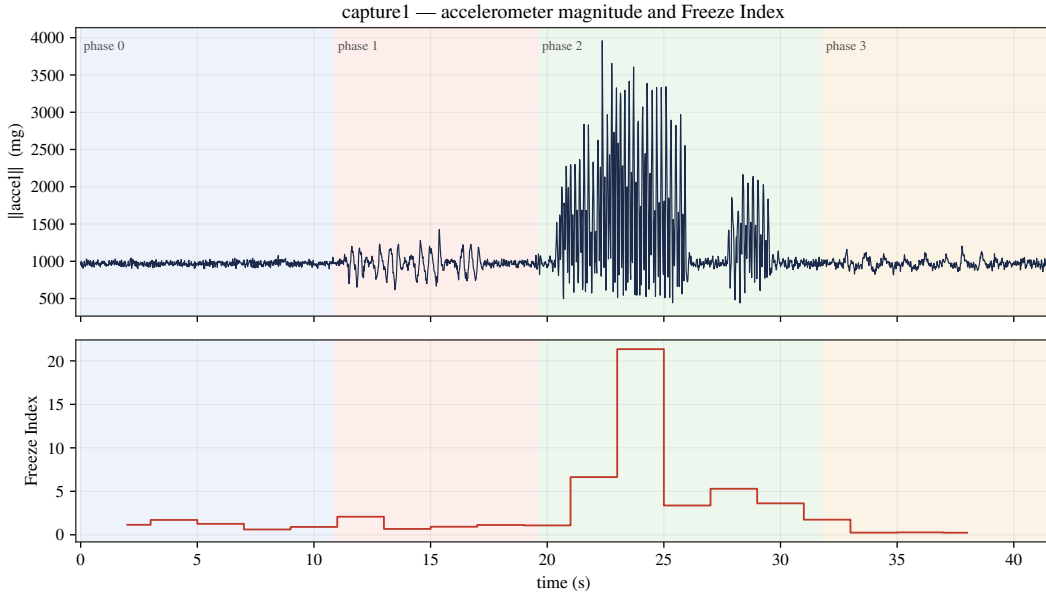


Figure 10. Real ankle magnitude over the worksheet capture (`capture1.csv`). The combined, mean-removed signal makes the freeze tremor stand out as a rapid in-place oscillation, distinct from the slower swing of walking and the near-flat rest segment.

Key idea

Two cheap operations – a square-root sum and a mean subtraction – buy the entire detector its orientation- and posture-independence. They are computed once per sample on the SAMD21 and are the only preprocessing the FI path applies. Notably the detector does *not* band-pass the axes first: filtering before taking the magnitude flattens the FI ratio so walking and freezing look alike, so the magnitude is deliberately formed from the raw, unfiltered axes.

The implementation is a two-line transform; on the host it lives in the torch-free `fog.dsp` module, and on the board it is the same arithmetic in C++:

Listing 1. Forming the orientation-invariant, mean-removed magnitude (`fog.dsp`).

```

1 import numpy as np
2
3 def magnitude(ax, ay, az):
4     """Orientation-invariant scalar from three accelerometer axes."""
5     return np.sqrt(ax**2 + ay**2 + az**2)
6
7 def demean(a):
8     """Remove DC (gravity / posture) before spectral analysis."""
9     return a - a.mean()
```

With one clean motion signal in hand, the next chapter turns it into a spectrum and reads off the band powers that feed the Freeze Index.

5 Fourier Analysis: From Time to Frequency

The accelerometer hands us a wiggling line: the magnitude $a[n]$ sampled 64 Hz, one number every ≈ 15.6 ms. Looking at that line, a calm stroll and a freeze episode are both “a wobble around $1g$ ” — the eye cannot reliably separate them. The whole trick of Cadence is to stop asking *how big* the wobble is and start asking *how fast* it wobbles. Walking paces the leg at roughly 1–2 steps per second; a freeze sets the foot trembling in place at a much faster 3–8 times per second. Those two *rhythms* live in different places on the frequency axis even when their amplitudes overlap. Fourier

analysis is the lens that turns a tangle of time-domain wiggles into a clean picture of which rhythms are present and how strong each one is.

Intuition

A chord on a piano is a single complicated pressure wave hitting your ear, yet you hear three distinct notes. Your ear is doing Fourier analysis: it decomposes one messy waveform into the pure tones that add up to it. Cadence does the same to the ankle signal — it listens for the “note” of a freeze (a 3–8 Hz buzz) hiding underneath the “note” of walking (a 1–2 Hz sway).

5.1 Any signal is a sum of sinusoids

The founding idea, due to Joseph Fourier, is that *any* reasonable signal — no matter how jagged — can be rebuilt by adding together pure sine and cosine waves of different frequencies, each with its own amplitude and phase. A slow sine captures the broad sway; faster sines fill in the finer trembles. Add enough of them and you reproduce the original wiggle exactly. The two questions “what does the signal look like in time?” and “which sinusoids is it made of?” are two descriptions of the *same* object — the second is just the more useful one for telling walking apart from freezing.

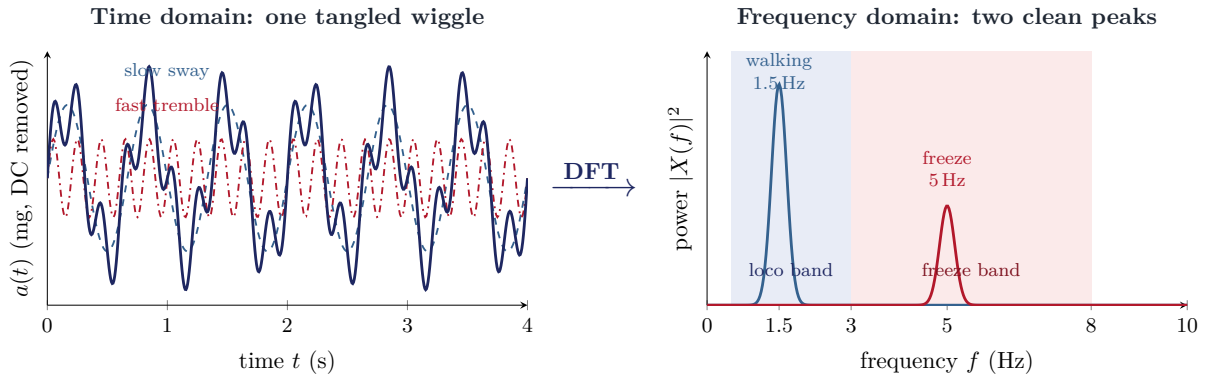


Figure 11. The same signal, two views. **Left:** a slow 1.5 Hz sway (steel, dashed) plus a faster 5 Hz tremble (red, dash-dotted) add to one tangled trace (navy) — hard to read by eye. **Right:** the Discrete Fourier Transform unfolds that trace into a spectrum where each rhythm becomes a sharp peak at its own frequency. Walking energy lands in the locomotor band (0.5–3 Hz, navy shading); freeze energy lands in the freeze band (3–8 Hz, red shading). Separating the two by *rhythm* is what makes them distinguishable.

5.2 The Discrete Fourier Transform and its bins

We never have a continuous signal; we have N evenly spaced samples. The **Discrete Fourier Transform** (DFT) is the finite-data version of Fourier’s idea. It takes a block of N samples and returns N complex numbers $X[k]$, each measuring how much of the k -th sinusoid the block contains.

The maths, unpacked

For a window of N samples $a[0], \dots, a[N-1]$, the DFT coefficients are

$$X[k] = \sum_{n=0}^{N-1} a[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1. \quad (4)$$

The term $e^{-j2\pi kn/N}$ is a pure spinning sinusoid at frequency index k ; the sum measures how strongly the data “rings” at that frequency. Coefficient $X[k]$ maps to a physical frequency

$$f_k = k \cdot \frac{f_s}{N}, \quad k = 0, 1, \dots, \frac{N}{2}. \quad (5)$$

The Fast Fourier Transform (FFT) is simply a clever $\mathcal{O}(N \log N)$ algorithm that computes the *same* $X[k]$ far faster than the naive $\mathcal{O}(N^2)$ sum — the answer is identical.

The spectrum is not continuous: it is sampled at discrete frequencies f_k called **bins**. The spacing between neighbouring bins is the single most important number in this section:

$$\Delta f = \frac{f_s}{N} = \frac{64 \text{ Hz}}{256} = 0.25 \text{ Hz}. \quad (6)$$

With $f_s = 64 \text{ Hz}$ and a window of $N = 256$ samples ($= 4.0 \text{ s}$), Cadence resolves the spectrum in steps of 0.25 Hz from 0 up to the Nyquist limit $f_s/2 = 32 \text{ Hz}$. That Nyquist ceiling comfortably covers every band we care about (the highest is the 8 Hz top of the freeze band), so nothing of interest is aliased.

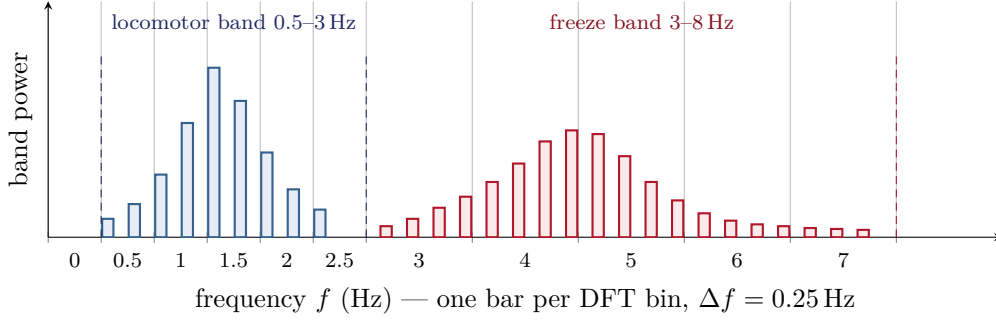


Figure 12. Each bar is one DFT bin, 0.25 Hz wide. **Band power** is nothing more than the sum of the bins falling inside a band: the locomotor band ($0.5\text{--}3 \text{ Hz}$, ≈ 10 bins) and the freeze band ($3\text{--}8 \text{ Hz}$, ≈ 20 bins). The 4 s window is chosen precisely so these bins are fine enough to separate the locomotor and freeze humps cleanly.

Key idea

Resolution costs time. The bin spacing $\Delta f = f_s/N$ shrinks as the window grows. To resolve the $3\text{--}8 \text{ Hz}$ freeze hump we need fine bins, so we use a long $N = 256$ (4 s) window: 0.25 Hz bins put ~ 20 samples across the freeze band. A short window would give coarse, blurry bins that smear walking and freezing together. The price is temporal: a 4 s window cannot localise an event to better than a few seconds — which is fine, because a freeze *lasts* seconds.

5.3 Power, leakage, and the Hann window

We rarely care about the phase of each sinusoid, only how much *energy* sits at each frequency. That is the **power spectrum**, $|X[k]|^2$: square the magnitude of each coefficient. Squaring also makes the quantity additive in a way the next section relies on. Parseval’s theorem guarantees the bookkeeping is honest:

$$\sum_{n=0}^{N-1} a[n]^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2. \quad (7)$$

The maths, unpacked

Parseval says the total energy is the same whether you measure it in time (left, sum of squared samples) or in frequency (right, sum of squared spectrum). No energy is created or destroyed by changing viewpoint. This is why “band power = sum of $|X[k]|^2$ over a band” is a meaningful slice of the signal’s real energy, and why adding the locomotor and freeze band powers gives the leg’s total movement energy (used later by the still-floor gate).

There is a catch. The DFT secretly assumes the N -sample window repeats forever. A real 4 s slice almost never starts and ends at the same value, so the assumed loop has a discontinuity at the seam — a sudden jump that the DFT explains by sprinkling spurious energy across many bins.

This smearing is called **spectral leakage**, and it can wash freeze-band energy into the locomotor band (and vice versa), exactly the contamination we must avoid.

The fix is a **window function** that gently tapers the slice to zero at both ends before transforming, removing the seam. Cadence uses the **Hann window**:

$$w[n] = 0.5 \left(1 - \cos \frac{2\pi n}{N-1} \right), \quad n = 0, 1, \dots, N-1. \quad (8)$$

It rises smoothly from 0, peaks at the centre, and returns to 0 — so however the slice was cut, the windowed version always starts and ends flush.

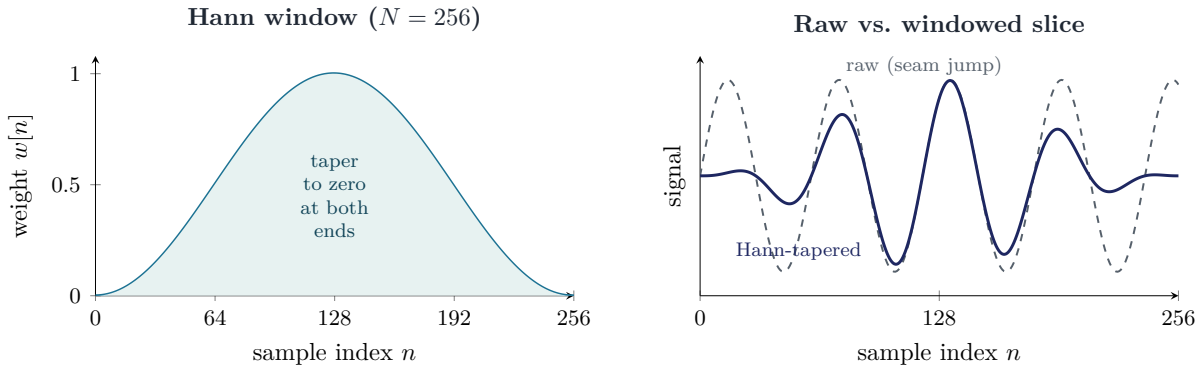


Figure 13. Left: the Hann taper — a raised cosine that weights the centre of the window fully and fades the edges to zero. **Right:** multiplying the raw slice (grey, dashed — note it does *not* return to its starting value, creating a discontinuity when the DFT loops it) by the Hann window gives the tapered slice (navy), which starts and ends flush at zero. The seam vanishes, and with it most of the spectral leakage.

5.4 Welch’s method: a steadier spectrum

A single 4s DFT is noisy — its bin heights jitter from one window to the next. Cadence uses **Welch’s method** to get a steadier estimate of the power spectral density (PSD). The recipe is short:

1. Subtract the window mean from $a[n]$ (kills the gravity/DC term so it does not dominate bin 0).
2. Apply the Hann taper of Eq. (8) (cuts leakage).
3. Take the FFT and form $|X[k]|^2$ (the power at each bin).
4. Average these periodograms across overlapping windows (smooths the jitter).

The result is the PSD: a stable estimate of power-per-frequency, normalised so that summing it over a band yields that band’s power. In code the whole chain is a few lines built on the `fog.dsp` primitives:

Listing 2. The Welch PSD chain, then a band-power slice (from `fog.dsp`).

```
1 def welch_psd(window, fs=64, nfft=256):
2     x = window - window.mean() # 1. remove gravity / DC
3     x = x * np.hanning(len(x)) # 2. Hann taper -> less leakage
4     X = np.fft.rfft(x, n=nfft) # 3. FFT (one-sided, real input)
5     return (np.abs(X) ** 2) / fs # power spectral density, |X|^2
6
7 def band_power(psd, fs, nfft, f_lo, f_hi):
8     df = fs / nfft # 0.25 Hz bin spacing
9     lo = int(np.ceil(f_lo / df)) # first bin in band
10    hi = int(np.floor(f_hi / df)) # last bin in band
11    return psd[lo:hi + 1].sum() # band power = sum of bins
```

Intuition

Welch’s method is the audio equivalent of letting a tuner “settle”: instead of trusting one fleeting snapshot, it averages several overlapping looks at the same few seconds so random

noise cancels out while the genuine walking and freeze tones stand firm. The 0.25 Hz bins then read like a labelled ruler — you can point straight at the 0.5–3 Hz and 3–8 Hz stretches and add up what is there.

Figure 14 shows the payoff on real data. The `capture1.csv` worksheet capture runs through four phases; in the freeze phase the magnitude trace erupts into a fast, dense tremble, and the Freeze Index — a ratio of two band powers we have just learned to compute — jumps from its calm baseline of ≈ 1 to a peak of 21.35.



Figure 14. Real worksheet capture (`capture1.csv`, 64 Hz, 41.4 s). **Top:** the ankle magnitude $\|a\|$ in mg across four phases. The calm phases are a quiet wobble near 1 g; the freeze phase (phase 2) explodes into a fast, high-amplitude tremble. **Bottom:** the Freeze Index computed per window from the band powers defined here — still ≈ 1.1 , walk ≈ 1.2 , then a freeze peak of 21.35. The fast tremble is invisible as “size” alone but obvious once we read it in the frequency domain.

Takeaway

Stop measuring *how big* the ankle wobbles and measure *how fast*: the DFT turns a 4 s window into 0.25 Hz bins, the Hann-tapered Welch PSD makes those bins trustworthy, and walking (0.5–3 Hz) lands elsewhere than freezing (3–8 Hz) — the foundation of the Freeze Index.

6 Band Power & the Freeze Index

The previous stage handed us a Welch power spectral density (PSD): a curve that says *how much* of the leg’s motion lives at each frequency. A spectrum is rich, but a state machine cannot act on a whole curve — it needs a single number it can compare against a threshold. This section reduces the spectrum to exactly that number, the **Freeze Index**, by first collapsing the curve into a handful of *band powers* and then taking a ratio of two of them. The ratio is the small, quietly clever idea that lets a \$30 board tell a freeze from ordinary walking.

Intuition

Walking and freezing both shake the ankle — the difference is *where* in the frequency spectrum the energy sits. A walking stride is a slow, heavy rhythm (roughly one to three steps a second); a freeze is a fast, fluttering tremor of the feet *in place* at 3–8 Hz. So we do not ask “how hard

is the leg moving?” (walking wins that contest) — we ask “what *fraction* of the shaking is fast?”. The Freeze Index is precisely that fraction expressed as a ratio, and it flips decisively the moment a freeze begins.

6.1 From a spectrum to a band power

The PSD $S(f)$ tells us the motion’s power density (power per hertz) at every frequency bin. We rarely care about one bin in isolation; we care about a whole *band* of physiologically meaningful frequencies. The **band power** over a band $[f_1, f_2]$ is simply the PSD summed across the bins that fall inside it:

$$P[f_1, f_2] = \sum_{k: f_1 \leq f_k \leq f_2} S(f_k), \quad f_k = k \Delta f, \quad \Delta f = \frac{f_s}{N} = \frac{64}{256} = 0.25 \text{ Hz}. \quad (9)$$

Because the window is $N = 256$ samples (4.0 s) at $f_s = 64$ Hz, the bins are spaced $\Delta f = 0.25$ Hz apart — fine enough to slice the spectrum cleanly into the three bands that matter clinically.

Table 3. The three frequency bands Cadence reads off the mean-removed magnitude spectrum. Each band is a contiguous run of 0.25 Hz PSD bins summed via Eq. (9).

Band	Range	What lives there
Locomotor	0.5–3 Hz	the walking rhythm: stride and step cadence, the heavy slow swing of normal gait.
Freeze	3–8 Hz	the tell-tale rapid trembling/shuffling of the feet <i>in place</i> during a freeze episode.
Rest-tremor	4–6 Hz	Parkinsonian rest tremor; reported on the dashboard as context, not used in the index.

Key idea

Every band power in Eq. (9) is taken on the *mean-removed magnitude* $a = \|(a_x, a_y, a_z)\|$, with its average subtracted. Combining the three axes makes the measure orientation-invariant (the sleeve can rotate on the ankle); subtracting the mean strips out gravity’s $\sim 1 g$ DC offset so it never contaminates the low band. We are left with pure *motion* power, ready to slice.

6.2 The Freeze Index: a ratio of two bands

Moore *et al.* (2008) defined the Freeze Index as the power in the freeze band divided by the power in the locomotor band:

$$\text{FI} = \frac{P[3\text{--}8 \text{ Hz}]}{P[0.5\text{--}3 \text{ Hz}] + \varepsilon}, \quad \varepsilon = 10^{-9}. \quad (10)$$

The numerator is the energy of the fast freeze tremor; the denominator is the energy of the slow walking rhythm. A tiny guard $\varepsilon = 10^{-9}$ is added to the denominator so a perfectly still leg — whose locomotor power can round to zero — never triggers a divide-by-zero. It is far below any real band power, so it changes no genuine reading; it only tames the 0/0 corner case.

The maths, unpacked

Read Eq. (10) as a competition between two bands.

- **Walking:** the slow stride dominates, so the denominator $P[0.5\text{--}3]$ is large and the numerator is small $\Rightarrow \text{FI} < 1$ to ~ 1 .
- **Freezing:** the fast 3–8 Hz tremor dominates, so the numerator swells and the denominator collapses $\Rightarrow \text{FI} \gg 1$.

The index does not measure *how much* the leg moves — it measures the **shape** of the spectrum, the balance of fast energy against slow energy. That is why a ratio, not a difference, is the right tool: a difference would still be dominated by amplitude.

Why a ratio? Scale-invariance.

Suppose we measured acceleration in milli-*g* on the host but in m s^{-2} on-device — a constant factor c between the two unit systems. Power scales with the *square* of amplitude, so *every* PSD bin, and hence both band powers, pick up the same factor c^2 :

$$\text{FI} = \frac{c^2 P[3-8]}{c^2 P[0.5-3]} = \frac{P[3-8]}{P[0.5-3]}. \quad (11)$$

The factor cancels top and bottom. The Freeze Index is therefore **scale-invariant**: it returns the same value whether the firmware works in milli-*g* or m s^{-2} , and the same value for a heavy stamp or a light shuffle of the *same spectral shape*. Amplitude is normalised away by construction, leaving only the quantity we actually care about — the proportion of fast tremor.

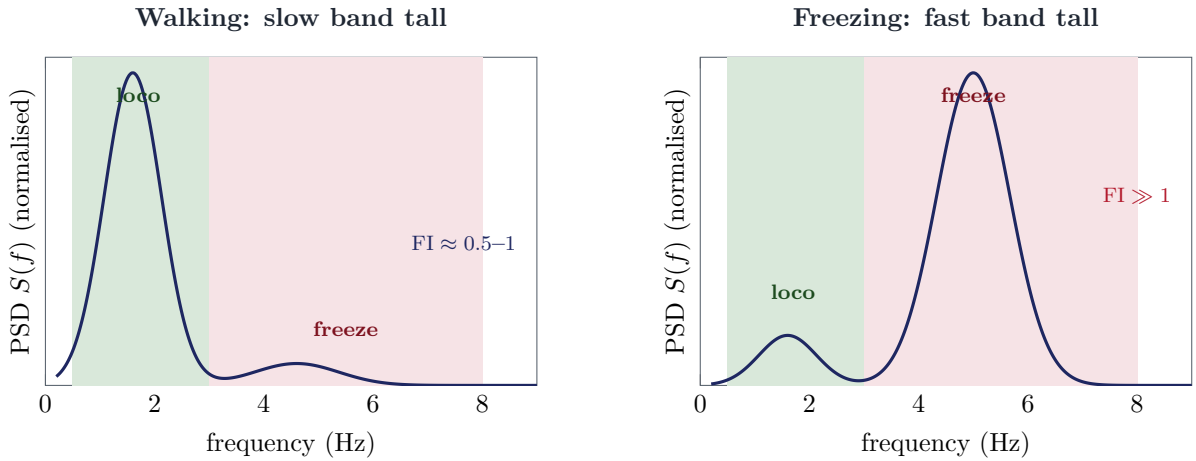


Figure 15. The Freeze Index reads the *shape* of the spectrum. In walking (left) the energy piles up in the green locomotor band (0.5–3 Hz), so the ratio freeze/loco stays near or below one. In a freeze (right) the energy migrates into the red 3–8 Hz band, the denominator collapses and the numerator swells, and the ratio flips far above one. Amplitude (overall height) is irrelevant — only the balance between the two shaded areas drives FI.

6.3 What it does on real data

Figure 15 is the cartoon; Figure 16 is the truth. The capture `capture1.csv` (64 Hz, 41.4 s) walks the ankle through four scripted phases — still, walk, freeze, walk — and we compute FI on every 4 s window. The contrast is exactly what Eq. (10) predicts.

Table 4. Real per-phase Freeze Index on `capture1.csv`. The freeze phase is an order of magnitude above quiet standing and ordinary walking, and its peak windows soar past 20.

Phase	Mean FI	Median FI	Max FI
Still (rest)	~ 1.1	—	1.71
Walk	~ 1.2	—	2.08
Freeze	6.89	4.45	21.35

Two features deserve a careful eye. First, still and walk sit close together (~ 1.1 versus ~ 1.2): **walking is not what trips the index**. Both are quiet in the freeze band, so both keep FI near

one. Second, the gap to the freeze phase is enormous — a mean of 6.89 and a maximum of 21.35, roughly six times higher than either calm phase. That separation is what makes a single scalar threshold viable.

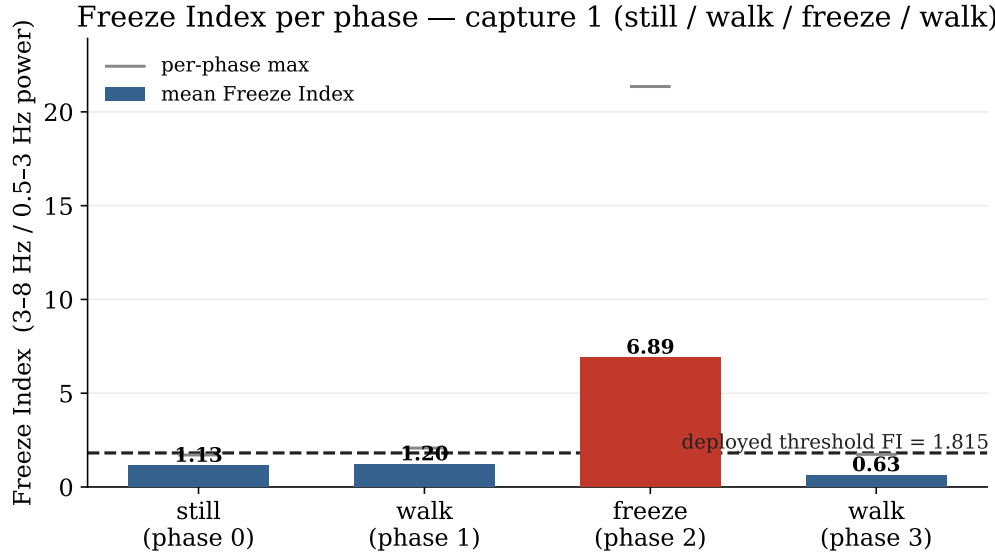


Figure 16. Measured Freeze Index per scripted phase of `capture1.csv` (still / walk / freeze / walk). Bars are the mean FI in each phase; the grey ticks mark the per-phase maximum (the freeze phase peaks at 21.35, off the top of the calm bars). Still and walk hug the deployed threshold line; the freeze phase towers over it. The gap, not the absolute height, is what a detector exploits.

Intuition

Notice how flat the two calm bars are next to the freeze bar. If we had used a *difference* of band powers instead of a ratio, a vigorous walk could rack up large freeze-band energy in absolute terms and look alarming. The ratio refuses to be fooled: a vigorous walk *also* has large locomotor energy, so the fraction stays low. Only when the spectral balance genuinely tips toward 3–8 Hz — a real freeze — does the number climb.

6.4 A scalar the state machine can act on

The Freeze Index turns a four-second spectrum into one comparable number, refreshed every two seconds as the window hops. Downstream, a window is flagged as a candidate freeze when FI crosses a trip line (the deployed value, 1.815, sits just above the calm-phase maxima of 1.71 and 2.08 so that genuine freezes clear it). On the live dashboard the same quantity drives the *Freeze Index* readout (walk ~ 0.5 –1; freeze > 2 , up to 20+) and the *Freeze Margin* meter, which expresses FI as a percentage of that 1.815 trip line. One ratio, computed once per window, feeds the verdict, the margin meter, and the threshold sweep alike.

It is worth being explicit about what the index does *not* settle on its own. A large FI means the spectrum is freeze-shaped, but a near-motionless leg can produce a freeze-shaped spectrum out of pure sensor noise — a few micro-*g* of jitter, mostly high-frequency, divided by an almost-empty locomotor band. The ε guard keeps that quotient finite, but it does not keep it *small*. The Freeze Index alone would therefore raise false alarms whenever the wearer simply stands still. For that reason Cadence never trusts FI in isolation: it is paired with a separate *movement-energy* gate that asks whether the leg is moving at all before the ratio is allowed to vote. That gate, the threshold sweep that sets the 1.815 trip line, and the two-window debounce are the subject of the sections that follow; here it is enough that the spectrum has been distilled into a single, honest, scale-invariant number.

Takeaway

The Freeze Index is one scale-invariant ratio — fast-band power over slow-band power — that ignores how *hard* the leg moves and reports only the *shape* of its motion. On real capture data it stays near 1.1–1.2 for standing and walking but leaps to a mean of 6.89 (max 21.35) during a freeze, giving the detector a clean, single-number signal to act on.

7 Real-Time Windowing

The accelerometer never stops: at 64 Hz it hands Cadence a fresh triple of (a_x, a_y, a_z) samples roughly every 16 ms, indefinitely. A freeze, by contrast, is an *event* with duration — a few seconds of 3 Hz to 8 Hz trembling. To bridge that gap, the firmware does not classify single samples. It slices the unending stream into short, overlapping *windows*, computes one spectral verdict per window, and slides forward. This section is about the geometry of that slicing: how wide each window is, how far it steps, what that buys us, and what it quietly costs.

Intuition

You cannot hear a chord from a single piano key struck for one millisecond — you need to listen for a beat or two to tell which notes are sounding. The freeze signature is a “chord” in the 3 Hz to 8 Hz band. Cadence listens in 4 s chunks, and starts a fresh chunk every 2 s, so a new opinion arrives twice a second while each opinion still rests on enough signal to be trustworthy.

7.1 The streaming geometry

Two integers fix everything. The **window length** is $N = 256$ samples; at $f_s = 64$ Hz that is exactly

$$T_{\text{win}} = \frac{N}{f_s} = \frac{256}{64} = 4.0 \text{ s.}$$

The **hop** (how far the window advances between decisions) is $H = 128$ samples, i.e. 2.0 s. Because $H = N/2$, consecutive windows share exactly half their samples — a **50% overlap** — and a fresh window closes, and therefore a fresh freeze decision is emitted, every

$$T_{\text{hop}} = \frac{H}{f_s} = \frac{128}{64} = 2.0 \text{ s.}$$

Key idea

Window length sets *how much signal each decision sees*; hop sets *how often a decision is made*. They are independent knobs. Cadence deliberately decouples them: a long 4 s window for spectral quality, but a short 2 s hop so the monitor still feels responsive. Overlap is simply the by-product, $1 - H/N = 50\%$.

Figure 17 draws the geometry directly. Each bar is one 4 s analysis window; each window is shifted 2 s to the right of the one before, so its leading half is brand-new signal and its trailing half was already seen by its predecessor. Reading down any vertical line tells you how many windows a given instant of the stream participates in — in steady state, every sample is examined by exactly two windows.

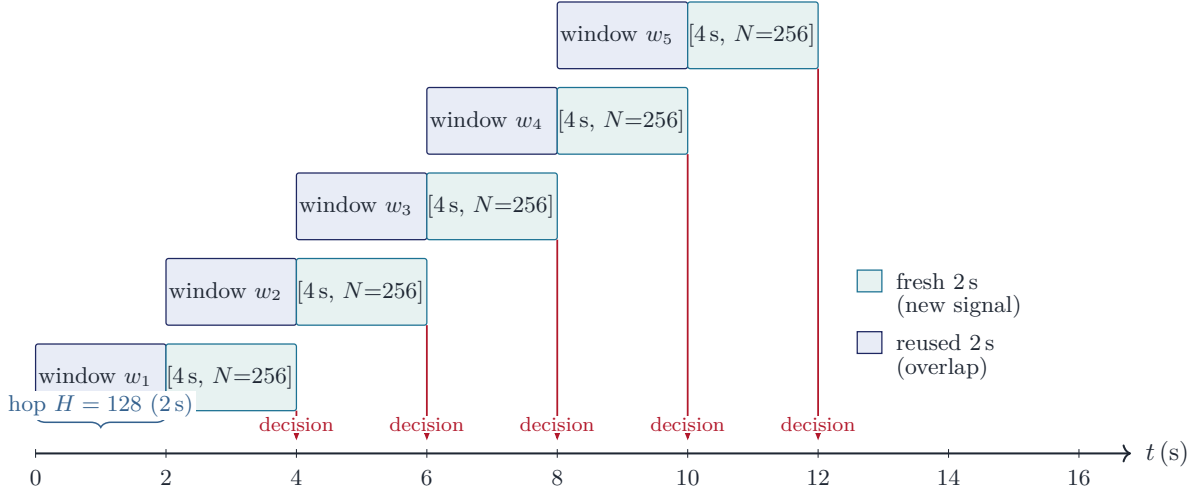


Figure 17. Sliding-window timeline. Each 4s ($N=256$) analysis window steps 2s ($H=128$) to the right of the one before, so half of every window is new signal (teal) and half is carried over from its predecessor (navy). A freeze decision (red arrow) is emitted each time a window closes — a fresh verdict every 2s. In steady state every sample is seen by exactly two windows.

```

1 # Streaming geometry, in software (fog.streaming, schematic)
2 N, H = 256, 128 # window = 4 s, hop = 2 s @ 64 Hz
3 ring = RingBuffer(capacity=N) # holds the most recent 4 s of samples
4 for sample in accel_stream(): # 64 new samples per second
5     ring.push(sample)
6     if ring.advanced_by(H): # a full 2 s of fresh data has arrived
7         window = ring.last(N) # newest 256 samples (50% overlap with previous)
8         verdict = classify(window) # gate -> freeze index -> debounce
9         emit(verdict) # one decision every 2 s

```

7.2 Why 4s? Spectral resolution versus latency

The window length is not a free choice — it is dictated by the physics of the signal we are trying to resolve. The freeze band, 3 Hz to 8 Hz, must be separated cleanly from the locomotor band, 0.5 Hz to 3 Hz, sitting right beside it. A Welch periodogram over N samples places its frequency estimates on a grid whose spacing is

$$\Delta f = \frac{f_s}{N} = \frac{64}{256} = 0.25 \text{ Hz.}$$

At 0.25 Hz per bin the 3 Hz to 8 Hz freeze band is covered by roughly twenty bins and the 0.5 Hz to 3 Hz walking band by ten — ample resolution to add up each band’s power independently and form the freeze index as their ratio.

The maths, unpacked

$\Delta f = f_s/N$ is the single most important line in this section. The bin spacing is *inversely* proportional to window length: double the samples, halve the bin width, sharpen the spectrum. A short 1 s window ($N = 64$) would give $\Delta f = 1 \text{ Hz}$ — so coarse that the 3 Hz to 8 Hz freeze band would collapse into a handful of fat bins and start to smear into the walking band beneath it. The 4 s window is the smallest that resolves the two bands cleanly.

There is a price. A longer window means each decision is an *average* over more time, so it reacts more slowly and blurs together whatever happened across its 4 s span. This is the classic resolution–latency trade-off, summarised in Table 5.

Table 5. The window-length trade-off. Cadence sits at $N = 256$ (bold row): the shortest window that still resolves the freeze band, paired with a half-window hop so decisions still arrive twice a second.

Window N	T_{win}	$\Delta f = f_s/N$	Freeze-band bins	Character
64	1 s	1.0 Hz	~ 5	fast, but bands smear together
128	2 s	0.5 Hz	~ 10	marginal resolution
256	4 s	0.25 Hz	~ 20	Cadence: clean bands, responsive
512	8 s	0.125 Hz	~ 40	sharp, but sluggish and over-smoothed

Intuition

Resolution and latency pull in opposite directions. A wide window is a *telephoto lens* on frequency — it sees the 3 Hz to 8 Hz band in sharp detail but takes a long exposure, so fast changes blur. A narrow window is a *snapshot* — quick, but the frequency picture is grainy. Because a freeze lasts *seconds*, a 4 s exposure is well matched to the event itself: long enough to be sharp, short enough not to miss the episode.

The hop rescues the responsiveness that the long window would otherwise cost us. Even though each window spans 4 s, stepping by only 2 s means the latest decision is never more than 2 s old. Overlap is therefore not wasteful redundancy — it is what lets a slow-exposure window still produce a fast-updating verdict. Figure 18 shows this on a real worksheet capture: the freeze index does not vary smoothly but jumps in a *staircase*, one tread every 2 s, each tread being the verdict of one closed window. The index climbs hard (> 20) only while the foot is trembling in place during the freeze phase, and the 2 s cadence of the steps is exactly the hop drawn in Figure 17.

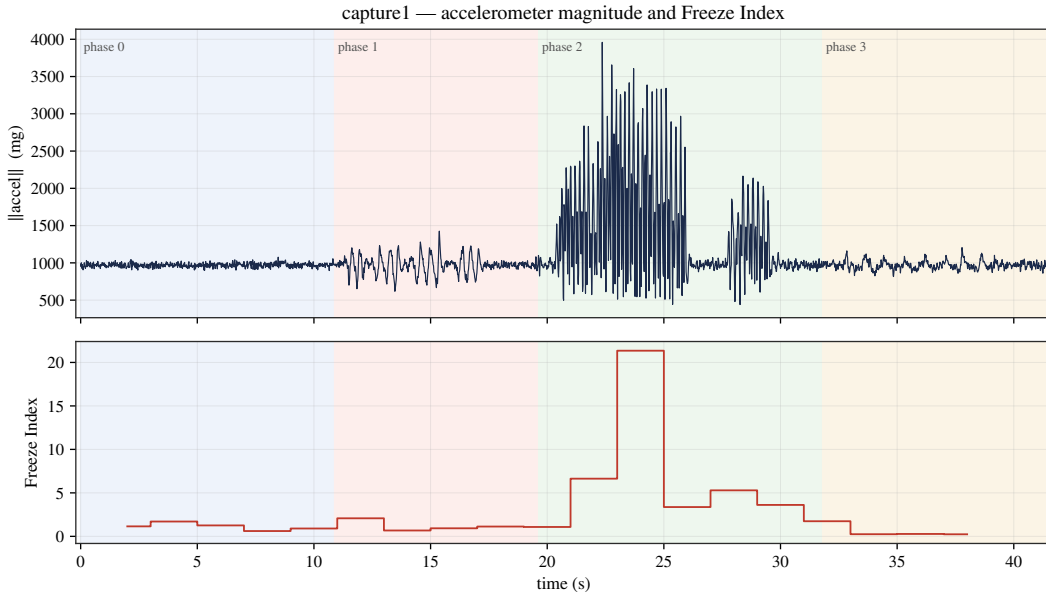


Figure 18. Real worksheet capture (`capture1.csv`, 64 Hz, 41.4 s). *Top*: accelerometer magnitude $\|a\|$, with the freeze phase showing the tell-tale large-amplitude 3 Hz to 8 Hz trembling. *Bottom*: the freeze index. Note that it updates as a *staircase* — one new tread every 2 s hop, each tread the verdict of one 4 s window — and spikes above 20 only during the freeze, where the worksheet records a mean FI of 6.89 and a max of 21.35.

7.3 The ring buffer

Holding “the most recent 4 s” on a SAMD21 with kilobytes of RAM rules out ever copying the stream. Cadence uses a **ring (circular) buffer**: a fixed array of $N = 256$ slots and a write pointer that wraps around to the start when it reaches the end. Each new sample overwrites the oldest one in $O(1)$ time, with no shifting and no allocation. A counter tracks how many fresh samples have

landed since the last decision; the moment it reaches the hop $H = 128$, the firmware reads off the newest 256 slots as the current window and resets the counter.

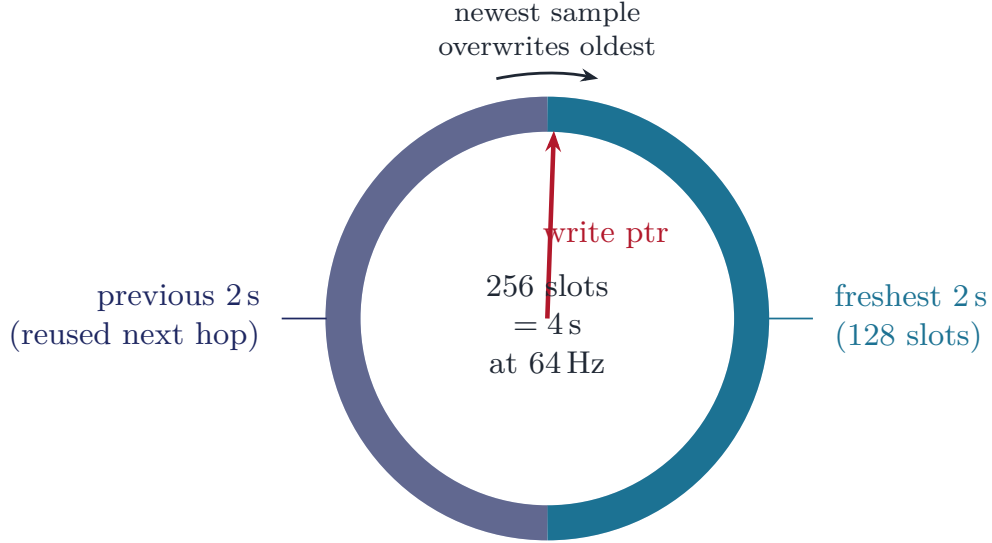


Figure 19. The ring buffer holds exactly $N = 256$ samples (4 s). The write pointer advances one slot per sample and wraps at the end, overwriting the oldest value in constant time. After $H = 128$ new samples arrive (the freshest 2 s, teal), the firmware reads the full ring as the current window and emits a decision; the older half (navy) is reused as the overlap of the next window.

7.4 The cost of windowing: dilution at the edges

Windowing is not free of blind spots, and honesty about them is part of the design. Because every verdict is a 4 s average, a freeze that is *short* relative to the window, or that is *split* across a window boundary, can be diluted below the detection threshold and missed.

- **Too short.** The freeze index is $FI = \text{power}(3-8)/\text{power}(0.5-3)$ computed over the whole 4 s window. If a freeze occupies only, say, 1 s of that window, the trembling energy is averaged together with 3 s of calmer signal, pulling the ratio down — possibly below the deployed trip line of 1.815.
- **Split across a boundary.** A freeze straddling the seam between two windows contributes part of its energy to each. Neither window contains the whole episode, so each sees a watered-down version, and both can fall short.

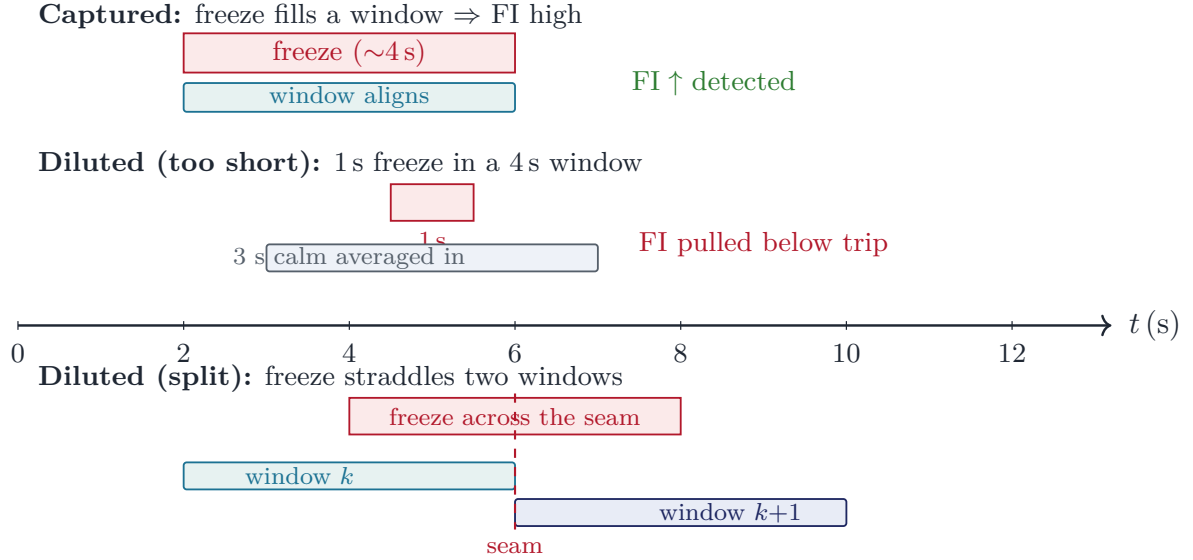


Figure 20. When windowing dilutes a freeze. *Top:* a freeze that fills a window drives the freeze index well above the trip line and is caught. *Middle:* a sub-window (1 s) freeze is averaged together with 3 s of calm signal, pulling the ratio down. *Bottom:* a freeze split across the seam between two windows donates half its energy to each, so neither window sees the full episode.

The 2 s hop is the first line of defence here: a freeze missed by one window’s alignment is examined again, 2 s later, by the next window — whose seam falls in a different place. Overlap thus gives every episode multiple looks. The second line of defence is downstream: the deployed trip line is biased low (1.815 rather than the Youden-optimal 2.10), precisely so that a partially-diluted freeze still clears the bar. The clinical logic is asymmetric — a missed freeze can mean a fall, whereas a false alarm merely lights an LED.

Takeaway

4 s windows give the spectral resolution to separate the 3 Hz to 8 Hz freeze band; a 2 s half-window hop gives a fresh verdict twice a second and a second look at any episode a single window might dilute. Long enough to see the freeze, fast enough to report it.

8 The Shipped Detector: Threshold, Gate, Debounce

Everything in the preceding chapters converges on a single, deliberately small decision rule. Out of all the spectral machinery, Cadence ships exactly one on-board verdict per control window, and that verdict drives the OLED, the NeoPixel colour, and the web dashboard. This is the *glass box*: a rule a clinician can read in one line and trust. The whole detector is

$$\text{FREEZE} \iff \underbrace{(\text{movement energy} > \text{still_floor})}_{\text{the leg is actually moving}} \wedge \underbrace{(\text{FI} > \tau)}_{\text{moving like a freeze}}, \quad \text{then debounced.} \quad (12)$$

Three ingredients, in this order: a **gate** (is the leg moving at all?), a **threshold** on the Freeze Index FI (is the movement the 3–8 Hz tremble of a freeze rather than the swing of a walk?), and a **debounce** (has the new verdict held for long enough to be believed?). The deployed trip line is $\tau = 1.815$. The rest of this section explains why each piece exists, why τ is set below the textbook optimum, and how the three combine into the STILL / WALKING / FREEZE state machine.

Key idea

The detector is not a black box that emits a probability. It is a short Boolean sentence over two physically meaningful quantities — how *hard* the leg is moving (movement energy) and how *freeze-like* that motion is (FI). Both can be read off the live dashboard, so every verdict is auditable after the fact.

8.1 Why a gate comes first: the still-leg problem

Recall that the Freeze Index is a *ratio* of band powers, $FI = P_{3-8}/P_{0.5-3}$. Ratios are scale-invariant, which is exactly what we want when comparing a freeze to a walk — but it is also their weakness when there is almost no signal at all. Stand perfectly still and both band powers collapse towards the sensor’s noise floor. Dividing one small noisy number by another small noisy number can produce *any* value, including a large one. A motionless leg can therefore manufacture a spurious high FI out of pure electronic hash, and a FI-only detector would cry “freeze” at someone sitting in a chair.

The fix is a second quantity that is deliberately *not* scale-invariant:

$$\text{movement energy} = P_{0.5-3} + P_{3-8}, \quad (13)$$

the total band power across the locomotor and freeze bands of the mean-removed magnitude. Unlike the ratio, this number reports *how much the leg is moving* in absolute terms. A still leg has near-zero movement energy; a walking or freezing leg has plenty. The gate is a simple floor on this quantity: if movement energy is below `still_floor`, the device declares `STILL` and never even consults the FI.

The maths, unpacked

FI and movement energy are built from the *same two band powers*, combined two different ways:

$$FI = \frac{P_{3-8}}{P_{0.5-3}} \quad (\text{a ratio — shape}), \quad \text{energy} = P_{0.5-3} + P_{3-8} \quad (\text{a sum — size}).$$

The ratio answers “*what kind of motion?*” and the sum answers “*is there any motion?*” You need both: shape without size is the still-leg trap; size without shape cannot tell a walk from a freeze.

`still_floor` is not a magic constant. It is **auto-calibrated at boot**: when the firmware starts, it samples the resting movement energy of the ankle for a moment and sets the floor just above that quiet baseline. This adapts the gate to the individual sleeve fit, the mounting tightness, and the particular board’s sensor offsets, so “still” means *this* wearer’s still, not a factory guess.

Intuition

Think of the gate as a bouncer at the door. Before anyone asks “is this person dancing or freezing?” the bouncer first checks “is this person moving at all?” If the leg is parked, the question of *how* it is moving is meaningless — and the noisy FI that the maths would hand back is simply ignored. On the dashboard this is the **Movement Energy** panel’s MOVING / STILL badge, and when it reads `STILL` the Freeze Margin meter shows “— / at rest.”

8.2 The threshold: 1.815 versus the textbook optimum 2.10

Once the gate confirms the leg is moving, the detector asks whether the motion is freeze-shaped by comparing FI against the trip line τ . Choosing τ is a classic sensitivity/specificity trade-off, and Cadence makes a deliberate, clinically-motivated choice that differs from the value a naive optimiser would pick.

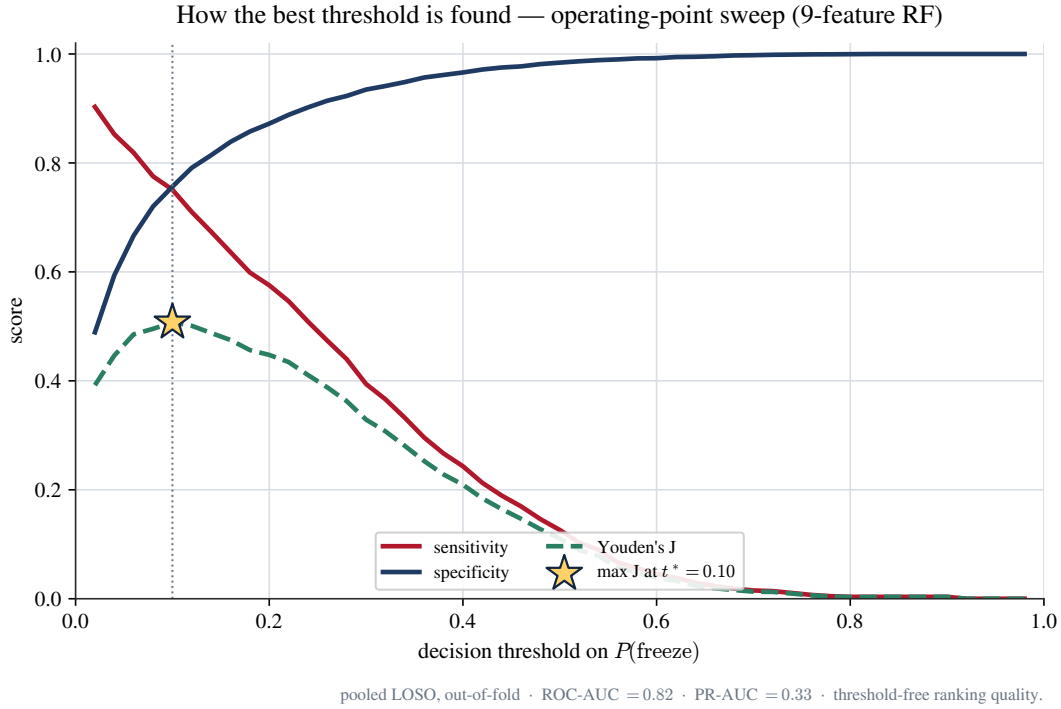


Figure 21. How an operating point is chosen — a threshold sweep. As the trip line moves, **sensitivity** (freezes caught) falls while **specificity** (calm gait correctly left alone) rises; the dashed curve is Youden’s $J = \text{sensitivity} + \text{specificity} - 1$, whose peak (star) is the “clean” balanced optimum. Cadence faces the same trade-off and chooses to sit *left* of the balanced point, buying recall of freezes at the cost of a few false alarms. (Curve shown for the interpretable model of Sec. 8’s companion study; the principle is identical for the FI trip line.)

To pick τ for the shipped rule we ran the real worksheet capture, `capture1.csv` (64 Hz, 41.4 s), through the exact windowing the firmware uses and counted outcomes window-by-window. The capture yields 19 control windows: 6 are genuine freeze windows and 13 are non-freeze (still or walking). Sweeping the trip line gives the two candidate operating points in Table 6.

Table 6. Real threshold sweep on `capture1.csv` (19 windows: 6 freeze, 13 non-freeze). TP/FP/TN/FN counted per window. The shipped board uses $\tau = 1.815$; $\tau = 2.10$ is the balanced (Youden) optimum.

Trip line τ	TP	FP	TN	FN	Sensitivity	Specificity	Note
FI > 1.815	5	1	12	1	0.83	0.92	<i>shipped</i> (sensitivity-biased)
FI > 2.10	5	0	13	1	0.83	1.00	clean Youden optimum

Read the table carefully, because the lesson is subtle. Both thresholds catch the same 5 of 6 freezes (sensitivity 0.83 either way) and both miss the same one short freeze. The *only* difference is a single window: at $\tau = 1.815$ one calm window is wrongly flagged (FP = 1, specificity 0.92), whereas at $\tau = 2.10$ that window is correctly left alone (FP = 0, specificity 1.00). So the higher threshold is strictly cleaner on *this* capture — it is the Youden-optimal point. Why does the board ship the lower, “worse” number?

Takeaway

A missed freeze can mean a fall; a false alarm just lights an LED that nobody acts on. The costs are wildly asymmetric, so Cadence biases toward **sensitivity** and ships $\tau = 1.815$. On this capture both thresholds happen to tie on sensitivity, but 1.815 leaves head-room: it keeps catching freezes that sit just *below* 2.10 on unseen wearers, where the clean optimum would start dropping them.

The point is that 2.10 is optimal *for capture1*, and over-fitting a single short capture is exactly the trap to avoid. The trade-off the firmware actually cares about is the one on the *next* wearer, whose freezes we have not seen. Pushing the trip line down to 1.815 is cheap insurance: it costs one false alarm on this recording (a harmless flicker on the display) and buys a margin of safety against the freezes that a tighter line would let slip. The asymmetry of harm, not the arithmetic of one CSV, sets the threshold.

Key idea

There are *two* different “right” answers. The **Youden** answer (2.10) maximises balanced accuracy on the data in front of you. The **clinical** answer (1.815) minimises the expected *harm*, given that a false negative (missed freeze → possible fall) costs far more than a false positive (a brief, ignorable LED). Cadence is a safety monitor, so it ships the clinical answer.

A note on provenance: a library default constant elsewhere in the code base reads 2.0, but the *deployed* value baked into the monitor firmware is 1.815. The two should not be confused — only 1.815 is the shipped trip line.

8.3 Seeing the rule on real data

Figure 22 makes the whole rule concrete on *capture1*. The top panel is the raw accelerometer magnitude $\|a\|$ in milli-g; the bottom panel is the per-window Freeze Index. The capture is staged in four phases: still, walking, freeze, then recovery.

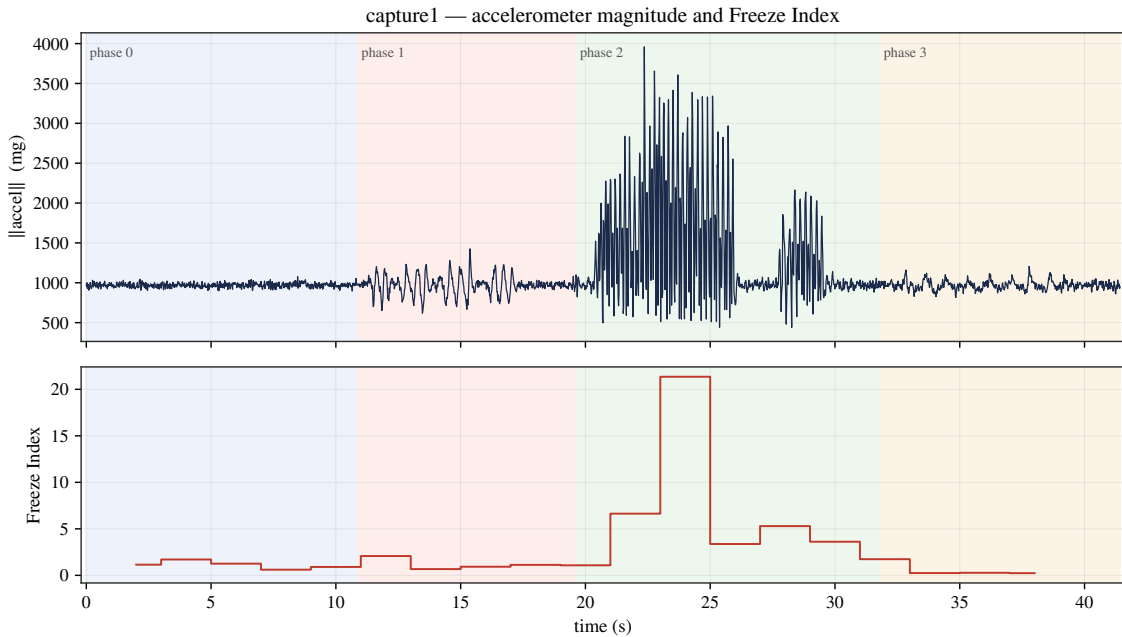


Figure 22. The detector’s two inputs on *capture1*. *Top:* accelerometer magnitude $\|a\|$ (mg) — note the violent, dense oscillation during the freeze phase, with heel-strike-scale peaks that would clip a $\pm 2g$ range. *Bottom:* the per-window Freeze Index, which sits near 1 through still and walking and erupts to a peak of 21.35 during the freeze. The shipped trip line $\tau = 1.815$ cuts cleanly between the two regimes.

The numbers match the picture. Per-phase FI on this capture is around 1.1 at rest (max 1.71) and around 1.2 while walking (max 2.08); during the freeze it jumps to a mean of 6.89, median 4.45, and peak 21.35. The trip line at 1.815 sits comfortably in the empty gap above the walking distribution and well below the freeze surge — which is precisely why a simple horizontal threshold works at all. Note too that the still and walking maxima (1.71 and 2.08) straddle 1.815: that lone window above the line during normal activity is the single false positive in Table 6, and is exactly what the gate and debounce exist to tame.

8.4 Debounce: don't trust a one-window verdict

A 256-sample window with a 128-sample hop produces one fresh verdict every two seconds. Real motion is noisy, and a single unlucky window — one heel-strike landing awkwardly, one transient bump — can momentarily push FI across the line. If the device committed every window instantly, the dashboard would flicker and a one-off jolt could read as a freeze.

Cadence therefore **debounces**: a *new* state must persist for **two consecutive** control windows before it is committed. This is *assert-after-2* for entering a state and *release-after-2* for leaving one. A single anomalous window can never flip the verdict; it takes a sustained, two-window-deep change ($\geq$ about four seconds of agreeing evidence given the overlap) to move the committed state. Because a genuine freeze lasts seconds, real episodes clear this bar easily, while isolated spikes do not.

Intuition

Debounce is the same trick as a mechanical light switch that must be held, not brushed. One stray window is a brush; the switch ignores it. Two windows in a row saying the same thing is a deliberate press, and only then does the verdict change. The cost is a short, bounded latency — but a freeze that lasts seconds has time to spare, and a fall-risk monitor would rather be right than instant.

8.5 Putting it together: the state machine

The gate, the threshold, and the debounce compose into a three-state machine that emits STILL, WALKING, or FREEZE. Figure 23 draws the live decision path for a single control window.

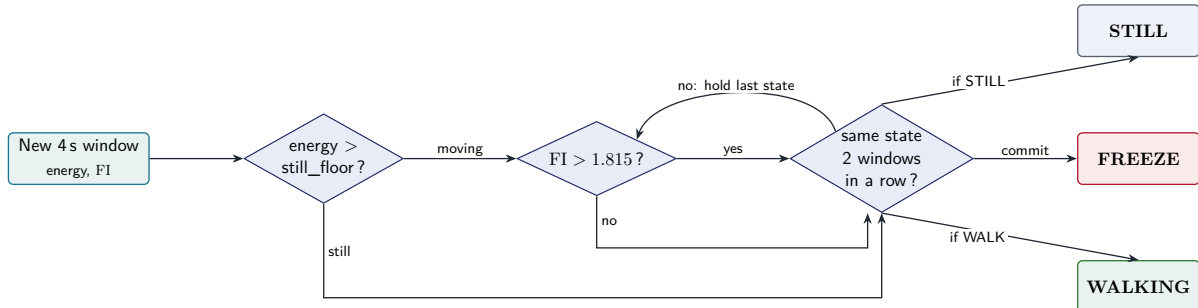


Figure 23. The shipped state machine for one control window. The **gate** (energy vs. `still_floor`) runs first; only if the leg is moving is the FI **threshold** consulted. The resulting candidate state (STILL / WALKING / FREEZE) is committed only after the **debounce** confirms it has held for two consecutive windows; otherwise the previous committed state is retained. Equivalently: *STILL* if not moving; else *FREEZE* if $FI > 1.815$; else *WALKING*.

In plain words, the state for each window is decided as:

1. **Gate.** If movement energy $<$ `still_floor`, the candidate is *STILL* and the FI is not even evaluated (it would be noise).
2. **Threshold.** Otherwise, if $FI > 1.815$ the candidate is *FREEZE*; if not, the leg is moving but not freeze-shaped, so the candidate is *WALKING*.
3. **Debounce.** The candidate becomes the *committed* state only once it has held for two consecutive windows; until then, the last committed state stands.

This committed state is the single output of the device. It paints the NeoPixel ring (state colours), prints to the OLED, and on the live dashboard drives the headline **Gait Status** panel, while the **Freeze Margin** meter shows the live FI as a percentage of the 1.815 trip line so a user can watch a freeze build before the line is crossed.

Takeaway

The whole shipped detector is three guards in series — **gate** (is the leg moving?), **threshold** (is it moving like a freeze?), **debounce** (has it held?). Each one removes a specific failure mode: the gate kills the still-leg FI blow-up, the sensitivity-biased threshold (1.815) refuses to miss a freeze, and the two-window debounce refuses to be fooled by a single jolt. Simple, inspectable, and tuned for the cost that actually matters: a missed freeze.

9 The CNN: FoGNet Architecture

The shipped detector is a *glass box*: one hand-chosen ratio, the freeze index, compared against one hand-chosen trip line. That transparency is a feature, not a bug — a clinician can read off exactly why a window was flagged. But it also fixes the rules in advance. The 3–8 Hz freeze band, the 0.5–3 Hz locomotor band and the simple “ratio of powers” shape were all decided by us, not discovered from data. **FoGNet** asks the opposite question: *given more time, could a network learn the right temporal filters for itself, straight from the raw accelerometer trace?* This section walks the architecture end to end. It is the “given more time” upgrade — a study of what a learned model buys — and is *not* the detector that runs on the ankle.

Intuition

The freeze index is a single filter we designed by hand: “measure energy in 3–8 Hz, divide by energy in 0.5–3 Hz.” A convolutional network is a stack of *many small filters whose shapes are learned*. Instead of telling the model which frequency band matters, we show it thousands of labelled 4-second clips and let gradient descent sculpt the filters that best separate freezing from walking. The freeze index is one filter we wrote; FoGNet is sixteen-plus filters it wrote itself.

9.1 Why a CNN, and why 1-D

The accelerometer hands us a *time series*: three channels (a_x, a_y, a_z) sampled at 64 Hz. A freeze episode is a *local temporal pattern* — a burst of rapid 3–8 Hz trembling — that can appear anywhere inside the 4-second window. Three properties of a 1-D convolutional network match this problem almost exactly:

- **Locality.** A freeze is defined by what neighbouring samples do *relative to each other* (a fast oscillation), not by their absolute time index. Convolution looks at short local stretches, which is the right unit of analysis.
- **Weight sharing.** The same small filter slides across the whole window, so a tremor at 0.5 s and the identical tremor at 3.5 s are scored by the *same* weights. This is far more data-efficient than a dense layer that would have to relearn the pattern at every offset.
- **Translation invariance** (completed by global pooling, below): a freeze *anywhere* in the window should count as a freeze.

We use *1-D* convolution because time is the one axis we slide along; the three accelerometer axes are treated as input *channels* (like the R/G/B channels of an image), all read together by every filter.

Key idea

FoGNet never sees a Welch spectrum, a band, or the freeze index. Its input is the *raw* 3×256 window — three axes $\times$ 4 s at 64 Hz — exactly the bytes the LIS3DH produced. Any notion of “frequency band” it ends up using is something it *learned*, not something we supplied.

9.2 What one convolution actually does

Picture a short filter — say 15 samples wide — as a little template laid over the signal. You multiply the filter against the 15 samples beneath it, add the products into a single number, then shift one sample to the right and repeat. Slide all the way along and you get a new, slightly shorter trace: the filter’s *response*. Where the signal locally resembles the template, the response spikes; elsewhere it stays near zero. A filter tuned to a ~ 5 Hz wobble will light up precisely during a freeze and stay quiet during a smooth stride.

The maths, unpacked

For an input channel $x[n]$ and a learned filter w of length K (here a `Conv1d` cross-correlation), the output at position n is

$$y[n] = b + \sum_{c=1}^{C_{\text{in}}} \sum_{k=0}^{K-1} w_c[k] x_c[n+k], \quad (14)$$

summed over all C_{in} input channels. The weights $w_c[k]$ and the bias b are *the things learned by backprop*; nothing about them is fixed in advance. With C_{out} such filters in a layer, the output has C_{out} channels — one response trace per filter. In FoGNet’s first layer $C_{\text{in}} = 3$ (the axes), $K = 15$, and $C_{\text{out}} = 16$, so 16 different 15-sample templates sweep the window simultaneously.

Each convolution is followed by two cheap, essential operations: **BatchNorm**, which re-centres and re-scales each channel so training stays numerically stable, and a **ReLU** nonlinearity, $\text{ReLU}(z) = \max(0, z)$, which clips negatives to zero. The ReLU is what lets the stack learn patterns more intricate than the freeze index’s single ratio: without it, any stack of convolutions would collapse into one equivalent linear filter.

9.3 Stacking blocks: wider receptive fields

One 15-sample filter sees only $15/64 \approx 0.23$ s of signal at a time — enough for one or two tremor cycles, not a whole episode. Depth fixes this. After the first two convolution blocks FoGNet applies **max-pooling** (stride 2), which keeps the strongest response in each adjacent pair and halves the length of the trace. A filter in the next layer, still only a few samples wide, now spans *twice* as much real time, because each sample it reads already summarises two original samples. Stack the effect and the *receptive field* — the slice of the original 4 s that one deep activation can “see” — grows from a fraction of a second to a large span of the window. Early filters catch fine texture (the wobble itself); deep filters catch coarse structure (a sustained burst of wobble), which is exactly what distinguishes a freeze from a single misstep.

Intuition

Max-pooling is a deliberate, useful blur. By throwing away *where exactly* the peak response sat inside each little pair and keeping only *how strong* it was, the network stops caring about sub-sample timing and starts caring about whether the freeze texture is present at all. Cheaper to compute, and more robust.

9.4 FoGNet, end to end

Figure 24 draws the whole network. The data starts wide and shallow (a long 256-sample window, only 3 channels) and ends short and deep (time mostly pooled away, 64 learned channels), before being collapsed to a single freeze probability. Three trends run left to right: *time shrinks* (pooling), *channels grow* ($3 \rightarrow 16 \rightarrow 32 \rightarrow 64$, more learned features), and *kernels narrow* ($15 \rightarrow 9 \rightarrow 5$, because deeper samples already cover more real time).

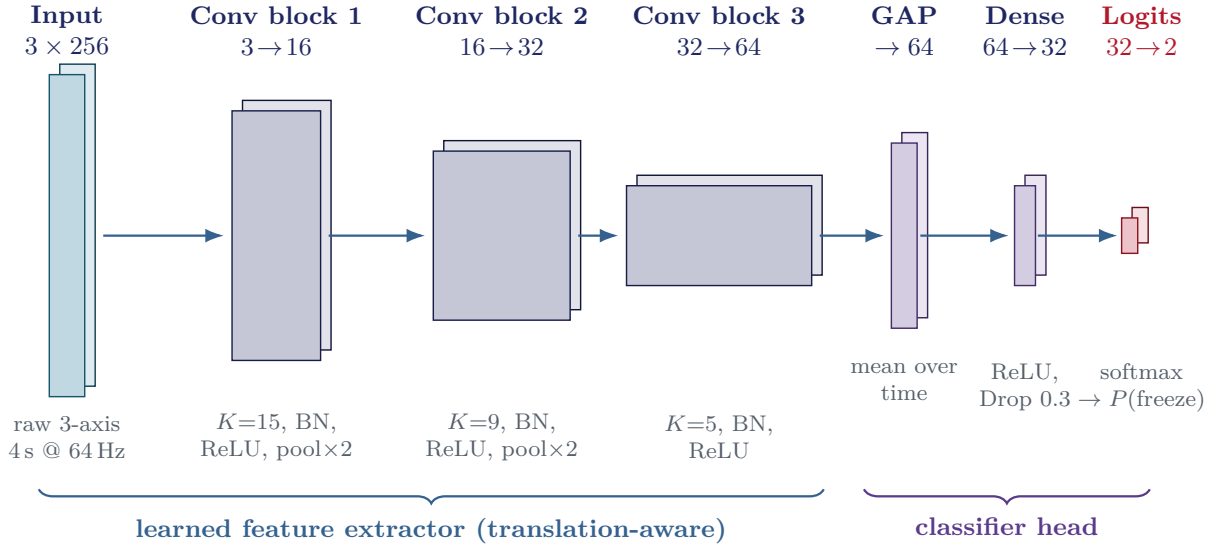


Figure 24. FoGNet, a 1-D CNN (~18k parameters). Tensor volumes shrink in time and grow in channels left to right. Three Conv→BN→ReLU blocks ($3 \rightarrow 16 \rightarrow 32 \rightarrow 64$ channels, kernels 15/9/5, max-pool after the first two) extract learned temporal features; global average pooling collapses each of the 64 channels to one number, which a small dense head maps to two logits and a softmax freeze probability. Runs in under 1 ms on a laptop CPU.

9.5 Global average pooling: a freeze anywhere counts

After the third block the network holds 64 response traces. Rather than flatten them (which would tie every feature to a fixed time slot and explode the parameter count), FoGNet takes the **global average** of each trace — one mean value per channel, a single 64-vector.

Key idea

Global average pooling (GAP) is what makes FoGNet *translation-invariant*. A freeze that erupts at second 1 and the same freeze at second 3 produce strong responses at different *positions*, but their channel *averages* are nearly identical. So a freeze “anywhere in the window still counts” — which is exactly the behaviour the windowed detector wants, and it sidesteps the dilution problem a fixed position-sensitive layer would suffer.

The 64-vector then flows through the classifier head: Linear($64 \rightarrow 32$) → ReLU → Dropout(0.3) → Linear($32 \rightarrow 2$). Dropout randomly mutes 30% of the 32 hidden units during training, forcing the head not to lean on any single feature — a regulariser against overfitting the modest dataset. The final layer emits two raw scores (*logits*), and softmax turns them into a probability:

$$P(\text{freeze}) = \frac{e^{z_1}}{e^{z_0} + e^{z_1}}, \quad (15)$$

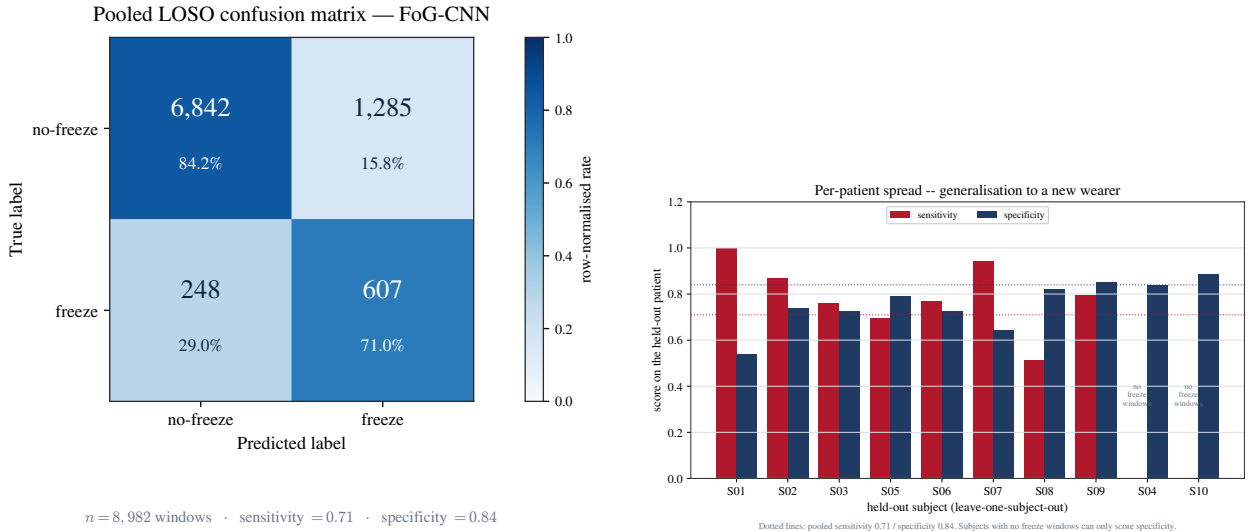
where z_0, z_1 are the “no-freeze” and “freeze” logits. The whole forward pass is tiny — about 18k parameters — and completes in under 1 ms on a laptop CPU.

Table 7. FoGNet layer-by-layer. Time shrinks via pooling; channel count grows; the classifier head turns the 64 pooled features into a freeze probability.

Stage	Operation	Kernel	Channels (out)
Input	raw 3-axis window	—	3
Conv block 1	Conv1d, BN, ReLU, max-pool×2	15	16
Conv block 2	Conv1d, BN, ReLU, max-pool×2	9	32
Conv block 3	Conv1d, BN, ReLU	5	64
Pooling	global average pooling	—	64 (vector)
Head	Linear, ReLU, Dropout 0.3	—	32
Output	Linear → softmax	—	2 logits

9.6 How it is trained and how it scores

FoGNet is trained on the public **Daphnet** FoG dataset (Bachlin et al. 2010). Each labelled 3×256 window is pushed forward to a probability; the *binary cross-entropy* loss penalises confident wrong answers; and gradient descent / backprop nudges every filter weight and bias — the $w_c[k]$ of Eq. (14) — a little in the direction that lowers the loss. Crucially, we evaluate with **Leave-One-Subject-Out (LOSO)** cross-validation: the network is tested on a person it never trained on, the honest test of whether the learned filters *generalise* rather than memorise one gait.



(a) Pooled LOSO confusion ($n = 8982$ windows).

(b) Per-subject scores: generalisation varies by person.

Figure 25. FoGNet under Leave-One-Subject-Out. Pooling the held-out folds gives $TN = 6842$, $FP = 1285$, $FN = 248$, $TP = 607$: sensitivity 0.71, specificity 0.84, accuracy 0.83.

Pooling the held-out folds gives the confusion matrix in Figure 25a: out of 8982 windows, $TP = 607$, $FN = 248$, $TN = 6842$, $FP = 1285$. That is a **sensitivity of 0.71**, a **specificity of 0.84** and an accuracy of 0.83. As argued throughout this report, sensitivity and specificity — not raw accuracy — are the metrics that matter when freezes are rare; the per-subject spread in Figure 25b is the more sobering view, showing how much generalisation depends on the individual.

Takeaway

FoGNet *learns* its temporal filters from raw accelerometer windows instead of relying on one fixed ratio, and global average pooling lets a freeze anywhere in the window count. It is the “given more time” study — a sub-millisecond, $\sim 18k$ -param network (0.71 sensitivity, 0.84 specificity under LOSO) — and it is deliberately *not* what ships on the ankle, where the transparent freeze-index detector runs.

10 Training the CNN: Loss, Gradient Descent & Generalisation

The shipped Cadence monitor decides freeze episodes with a transparent freeze-index rule. The CNN of the previous section — *FoGNet*, a 1-D convolutional network of roughly 18 000 parameters — is the “given more time” upgrade: instead of hand-reading one ratio, it *learns* from data what a freeze window looks like. But a network with eighteen thousand free numbers does not know anything at birth; it has to be *taught*. This section is the story of that teaching. We need three ingredients: a way to turn the network’s raw opinion into a probability (the **sigmoid**), a way to score how wrong that probability is (the **loss**), and a way to nudge the eighteen thousand numbers so the score improves (**gradient descent**). Then comes the part that actually decides whether the model is trustworthy: **how we evaluate it** without fooling ourselves.

Intuition

Training is just *practising with an answer key*. We show FoGNet thousands of 4-second accelerometer windows from real Parkinson’s patients, each stamped “freeze” or “calm” by a clinician. For every window the network guesses, we measure how badly it guessed, and we adjust its internal knobs a hair in the direction that would have made the guess better. Repeat a few hundred thousand times and the knobs settle into a configuration that recognises freezing it has never seen before.

10.1 From a raw score to a probability: the sigmoid

FoGNet’s final layer emits two raw numbers — one *logit* arguing “freeze”, one arguing “calm”. Their difference $z = z_{\text{freeze}} - z_{\text{calm}}$ is a single signed score: positive leans freeze, negative leans calm, zero is a coin-flip. A score is not yet a probability, though — z can be any real number, while a probability must live in $[0, 1]$. The **sigmoid** (equivalently, the 2-class *softmax*) is the squashing function that maps the whole real line onto that interval:

$$\hat{p} = P(\text{freeze} \mid \text{window}) = \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (16)$$

A large positive logit gives $\hat{p} \rightarrow 1$; a large negative one gives $\hat{p} \rightarrow 0$; and $z = 0$ sits exactly at $\hat{p} = 0.5$. The final “freeze / calm” call is made by comparing $\hat{p}$ against a decision threshold t — and, crucially, t need *not* be 0.5. Because a missed freeze is far costlier than a false alarm, we are free to slide t downward to favour sensitivity, exactly as the shipped FI rule biases its trip line. Choosing t honestly is the subject of the calibration discussion at the end of this section.

Figure 26 (left) shows the curve. Two features matter for what follows. First, the steep middle: near $z = 0$ a small change in the score moves the probability a lot, so the network can be *decisive*. Second, the flat tails: once z is far from zero the curve *saturates* and its slope approaches zero. A flat slope means a near-zero gradient, and a near-zero gradient means almost no learning signal — a fact that will return when we discuss how training can stall.

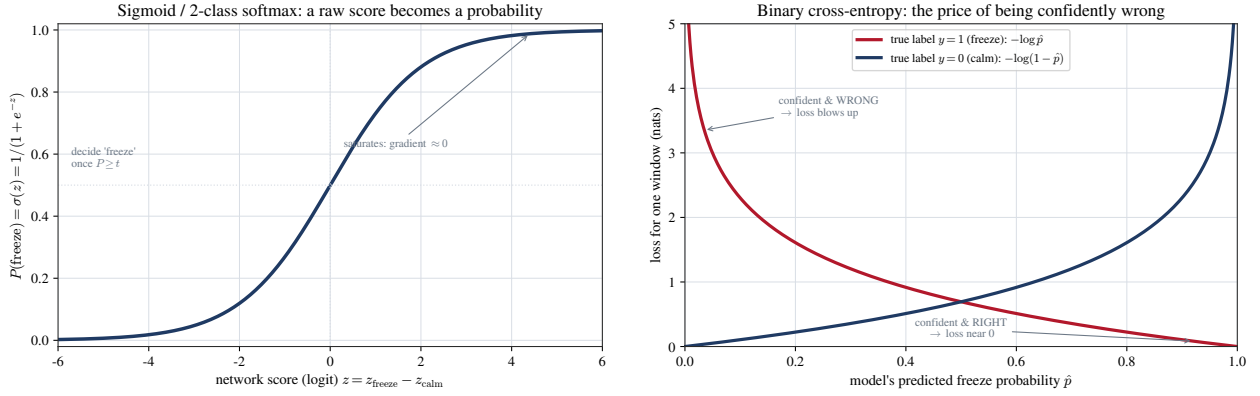
10.2 The loss: punishing confident mistakes

We now need a number that says *how wrong* a probability $\hat{p}$ is, given the true label $y \in \{0, 1\}$ ($y = 1$ for a real freeze window, $y = 0$ for calm gait). FoGNet is trained with **binary cross-entropy** (BCE), also called log-loss:

$$\mathcal{L}(\hat{p}, y) = -[y \log \hat{p} + (1 - y) \log(1 - \hat{p})]. \quad (17)$$

The maths, unpacked

Equation (17) looks like two terms, but for any single window only one of them is ever “on”, because y is either 0 or 1.



(a) The sigmoid maps any score z to a probability in $[0, 1]$; it is steep in the middle (decisive) and flat in the tails (saturated, gradient ≈ 0). (b) Binary cross-entropy. Being confidently *wrong* ($\hat{p}$ near the opposite label) sends the loss towards infinity; being confidently *right* costs almost nothing.

Figure 26. The two functions that turn “guessing” into “learning”. The sigmoid (left) produces a probability; binary cross-entropy (right) prices how bad that probability is given the true label.

- **A true freeze** ($y = 1$). The second term vanishes and the loss collapses to $\mathcal{L} = -\log \hat{p}$. If the network correctly said $\hat{p} = 0.99$, the loss is $-\log(0.99) \approx 0.01$ — almost free. If it confidently insisted “calm” with $\hat{p} = 0.01$, the loss is $-\log(0.01) \approx 4.6$ — a huge penalty.
- **A calm window** ($y = 0$). Now the first term vanishes and $\mathcal{L} = -\log(1 - \hat{p})$, which is the mirror image: cheap when $\hat{p}$ is near 0, explosive when the network falsely cries “freeze”. The log is what makes BCE the right tool. A plain squared-error would mildly scold a confident blunder; the logarithm makes the penalty diverge to $+\infty$ as $\hat{p}$ heads to the *wrong* certainty. The network therefore learns not just to be right, but to be *calibrated* — to reserve its confidence for cases that deserve it.

Figure 26 (right) draws both branches. The training objective is the *average* of Equation (17) over the whole batch of labelled windows; that single scalar is what the optimiser tries to push downhill.

Intuition

BCE is a teacher who marks two things at once: *was the answer right*, and *how sure were you*. A hesitant wrong answer is forgiven; a confident wrong answer is hammered. That is precisely the behaviour we want from a fall-risk monitor — a model that shouts “freeze!” must have earned the right to shout.

10.3 Gradient descent: rolling the weights downhill

The loss is a function of the network’s 18 000 weights: change a weight, and the average loss changes. Picture those weights as coordinates on a vast, hilly landscape whose *height* is the loss. Training is the search for the lowest valley. **Backpropagation** (the chain rule applied layer by layer) computes the *gradient* $\nabla_{\theta} \mathcal{L}$ — the direction of steepest *uphill* at the current spot. To go down, we step the opposite way:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}, \quad (18)$$

where θ collects all the weights and η is the **learning rate** — the size of each step. (Cadence trains with the AdamW optimiser, a refined cousin of Equation (18) that adapts the step per weight, but the “roll downhill” picture is identical.) Repeat the read-the-slope / take-a-step loop over many *epochs* — one epoch being a full pass over the training windows — and the weights descend towards a good valley.

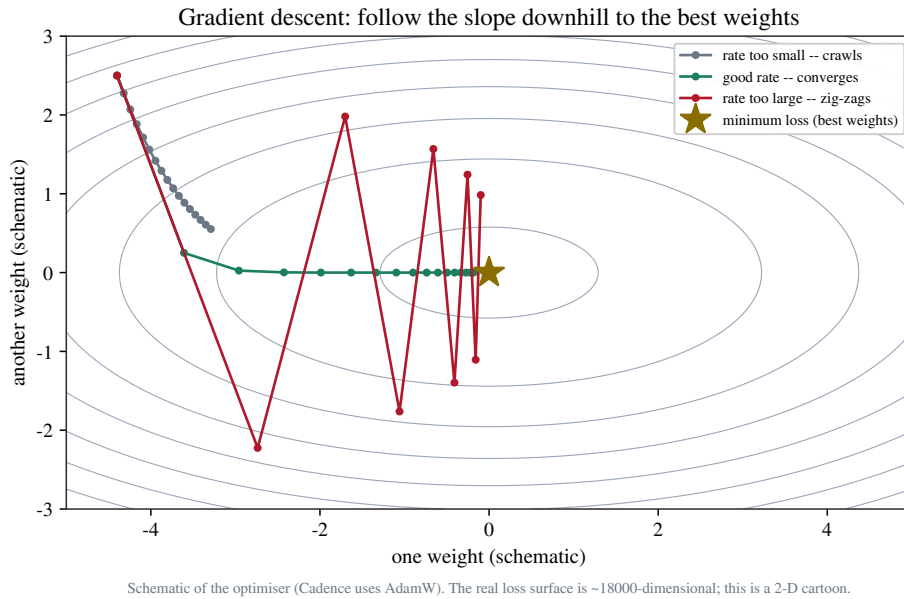


Figure 27. Gradient descent on a 2-D cartoon of the loss surface (the real surface is $\sim 18\,000$ -dimensional). The contours are loss; the star is the best weights. A learning rate too small *crawls*; one too large *zig-zags* and may never settle; a well-chosen rate *converges* straight to the minimum.

Figure 27 makes the role of the learning rate vivid. If η is too small the optimiser inches along and may run out of epochs before reaching the valley. If η is too large it overshoots, ricochets across the basin and can diverge. The “good rate” path walks almost straight to the star. This is also where the saturated sigmoid tails of Figure 26a bite: where the curve is flat the gradient is tiny, the step in Equation (18) barely moves, and learning crawls — one reason normalisation layers (below) are used to keep activations in the lively middle of the curve.

Takeaway

Loss says *how wrong*; the gradient says *which way is wrong*; the learning rate says *how big a correction to make*. Iterate, and eighteen thousand random numbers turn into a freeze detector.

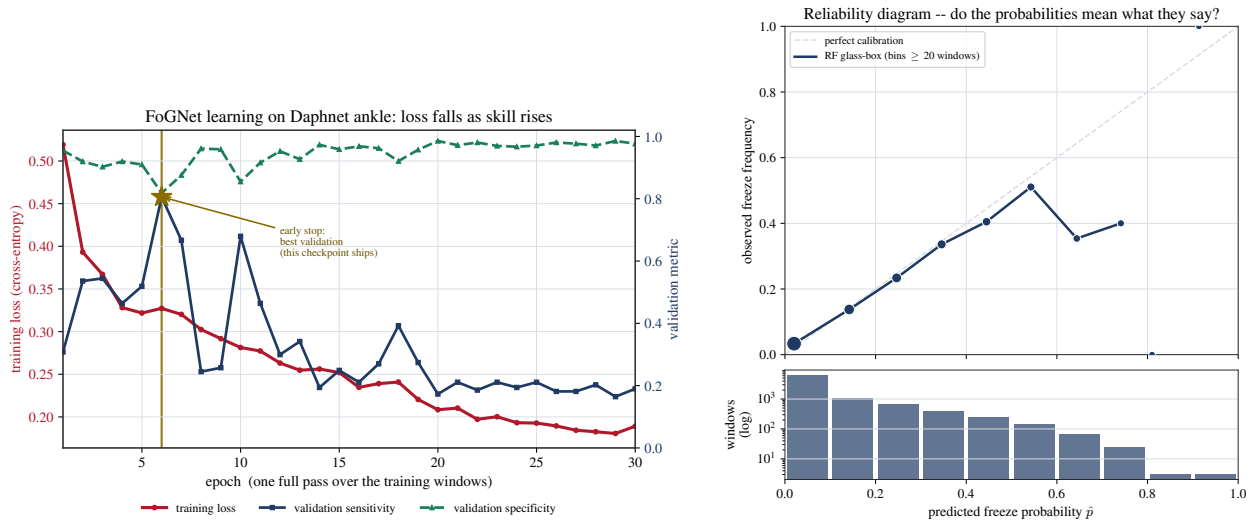
10.4 Over-fitting and the regularisers

A network with 18 000 parameters is powerful enough to *memorise* its training windows — including their noise and the quirks of the particular patients it saw. A memoriser scores beautifully on data it has already seen and fails on anyone new; this is **over-fitting**. FoGNet leans on two built-in defences, visible in its architecture:

- **Dropout (rate 0.3)** in the classifier head. During training, a random 30% of the units are switched off on each pass, so no single neuron can become a load-bearing memoriser; the network is forced to spread its evidence across many redundant features. At inference all units are active again.
- **Batch normalisation** after every convolution. BatchNorm re-centres and re-scales each layer’s activations, which keeps signals in the responsive middle of the nonlinearity (avoiding the dead saturated tails), steadies the gradients, and lets training use a larger, faster learning rate.

The tell-tale sign of over-fitting is a *gap*: the training loss keeps falling while performance on held-out data stops improving — or worsens. We therefore watch a validation metric every epoch and keep the checkpoint that is best on data the network was *not* trained on, a practice called **early stopping**.

Figure 28 (left) is FoGNet’s actual training run on the Daphnet ankle data: loss slides down as the validation sensitivity and specificity rise, and the highlighted epoch marks the checkpoint with



(a) Training curve. The cross-entropy loss (red) falls epoch after epoch while validation sensitivity and specificity climb. The starred epoch is the best validation checkpoint — the weights that actually ship.

(b) Reliability diagram (glass-box model). Points on the dashed line mean a stated probability matches the observed freeze frequency; the histogram shows how many windows land in each confidence bin.

Figure 28. Watching learning happen (left) and checking the probabilities are honest (right). Early stopping reads off the left panel; threshold choice reads off the right.

the best validation balance — the one that ships rather than the lowest-loss one.

10.5 Evaluating without fooling ourselves: Leave-One-Subject-Out

This is the most important methodological decision in the whole pipeline, and the easiest to get catastrophically wrong. The obvious way to test a model is to shuffle all the windows, hold back a random fraction, train on the rest, and report the score on the held-back part. For freeze detection this is a *trap*.

Key idea

A random window split leaks the person, not the freeze. Consecutive 4-second windows from one patient overlap and share that individual’s gait fingerprint — their stride rhythm, their tremor, the exact way their sleeve sits on the ankle. Split them randomly and almost every test window has a near-twin in the training set from the *same body*. The model can then ace the test by recognising *who* is walking rather than *whether* they are freezing — and that skill evaporates the moment a new patient straps the device on. The honest question is never “can it recognise this patient again?” but “can it recognise freezing in someone it has *never met*?”

The fix is **Leave-One-Subject-Out (LOSO)** cross-validation. With N patients we run N rounds; in each, one patient is held out entirely as the test set and the model is trained on the other $N - 1$. Every patient takes a turn as the unseen subject, and we pool the N held-out verdicts into one confusion matrix. No window from a test patient ever touches training, so the score genuinely answers the generalisation question. Figure 29 sketches the protocol.

Run this way, FoGNet’s pooled LOSO confusion over 8982 held-out windows gives $TN = 6842$, $FP = 1285$, $FN = 248$, $TP = 607$ — a sensitivity of 0.71, specificity of 0.84 and accuracy of 0.83. These are honest, new-patient numbers, not the inflated figures a random split would have printed. The interpretable 9-feature glass-box alternative, evaluated under the same protocol, reaches sensitivity 0.75 and specificity 0.76 at its operating point, with a threshold-free ROC-AUC of 0.82.

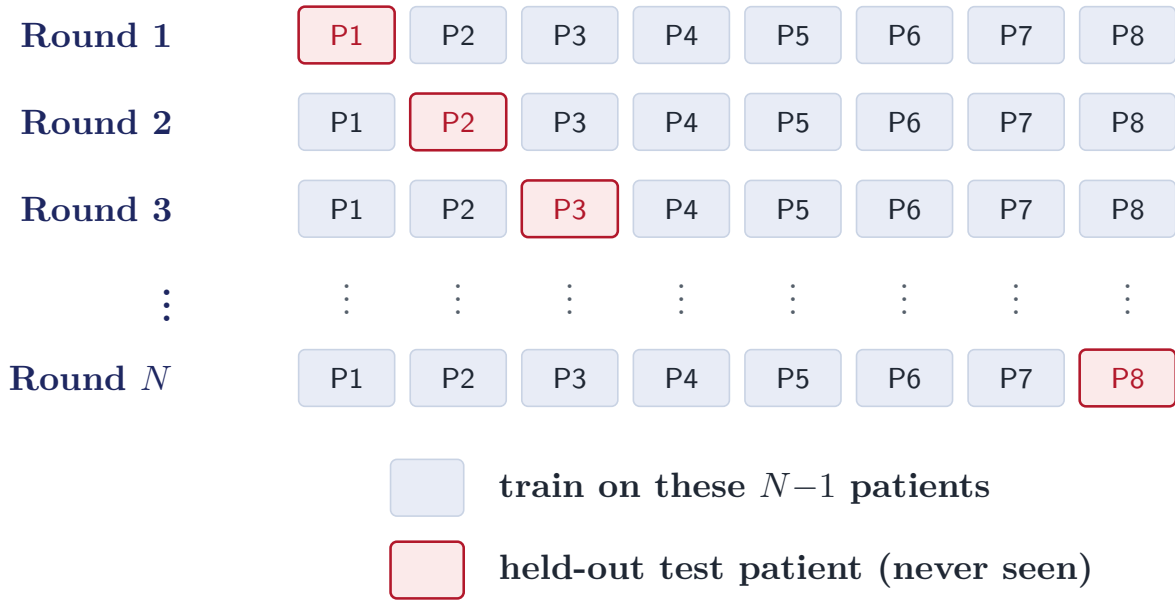


Figure 29. Leave-One-Subject-Out cross-validation. Each round holds out one whole patient (red) and trains on all the others (blue); rotating the held-out subject through everyone measures performance on bodies the network has never met. Pooling the held-out verdicts gives an unbiased confusion matrix.

Nested LOSO for the decision threshold

There is one more leak to seal. The decision threshold t of Section 10 is itself a parameter we tune — and if we tune it by peeking at the very test patients we then report on, we have quietly leaked the answer again. **Nested LOSO** closes this: inside each outer round, the threshold is chosen on a further held-out *inner* fold drawn only from the training patients, never from the outer test subject. The glass-box model’s reported operating point ($t^* \approx 0.10$) is selected this way, so even the threshold is honest. The headline numbers we quote are therefore what a genuinely new wearer should expect, not a flattering self-portrait.

10.6 Calibration: do the probabilities mean what they say?

A final, subtle question: when the model says “ $\hat{p} = 0.7$ ”, do roughly 70% of such windows actually turn out to be freezes? A model can rank windows well yet state *mis-scaled* probabilities — which matters, because the Freeze-Margin meter on the live dashboard and the choice of t both assume the numbers are meaningful. We check this with a **reliability diagram**: bin the predictions by stated probability and plot, per bin, the observed freeze frequency against the predicted one. Figure 28b shows the glass-box model’s diagram. Points hugging the diagonal are well calibrated; the companion histogram reveals that the vast majority of windows sit at low predicted probability — unsurprising, since freezing is rare — so the low-confidence bins are the ones that must be trustworthy, and they are.

Takeaway

Sigmoid turns scores into probabilities; BCE punishes confident errors; gradient descent finds the weights; dropout and BatchNorm stop the model memorising; and — the part that makes the headline numbers believable — **Leave-One-Subject-Out testing measures the freeze, not the person**. Tested honestly, FoGNet catches 71% of freezes while leaving 84% of calm gait undisturbed.

11 Measuring Accuracy: Confusion, Sensitivity & Specificity

Every detector in this report — the shipped on-board freeze-index rule, the glass-box RandomForest, and the FoGNet CNN — ultimately hands down the same kind of verdict: for each 4-second window it answers *freeze* or *no-freeze*. To compare those detectors, and to decide where to set their trip lines, we need a language for “how good is this verdict?” that survives contact with a hard clinical fact: **freezes are rare**. A patient walks, stands and shuffles for minutes between episodes that each last only seconds, so the no-freeze windows vastly outnumber the freeze windows. This section builds that language from the bottom up — the confusion matrix, then the four rates derived from it — and shows why the single number most people reach for, raw accuracy, is the *wrong* headline for Cadence.

Intuition

Think of Cadence as a smoke alarm for the feet. Two ways to be right (stay quiet when the kitchen is calm; sound when there is a fire) and two ways to be wrong (a false alarm when you burn toast; silence during a real fire). A smoke alarm graded only on “percent of seconds it was correct” would score brilliantly by simply never going off — because almost every second of every day is fire-free. That is exactly the trap a freeze monitor must not fall into.

11.1 The confusion matrix: four ways a window can be scored

Fix one detector and one trip line. Run it over a labelled recording and, window by window, cross its *prediction* against the *ground truth*. Every window lands in exactly one of four cells. We treat *freeze* as the “positive” class, because catching a freeze is the event we actually care about.

Table 8. The 2×2 confusion matrix. Rows are what truly happened; columns are what the detector said. The diagonal (TP, TN) is correct; the off-diagonal (FP, FN) is the two distinct ways to be wrong — and for a freeze monitor they cost very different amounts.

		Detector says	
		freeze (positive)	no-freeze (negative)
Truth	freeze	True Positive (TP) a real freeze, caught	False Negative (FN) a real freeze, <i>missed</i>
	no-freeze	False Positive (FP) calm gait, false alarm	True Negative (TN) calm gait, correctly ignored

Key idea

The two errors are *not* symmetric in their consequences. A **false negative** is a freeze the monitor failed to record — and an unrecorded freeze is a fall risk that the clinician never sees. A **false positive** is a moment of calm gait flagged as a freeze — annoying, and it muddies the log, but it hurts nobody. Cadence is therefore tuned to push FN down even at the cost of a few more FP. This is the single design decision behind the deployed freeze-index threshold of 1.815 rather than the cleaner Youden optimum of 2.10.

11.2 Four rates, and why accuracy is the wrong one

From the four counts we build four rates. Each asks a different question of the detector; each is just a ratio of cells in Table 8.

The maths, unpacked

Sensitivity (recall, true-positive rate) — *of the real freezes, what fraction did we catch?*

$$\text{sensitivity} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (19)$$

Specificity (true-negative rate) — *of the calm-gait windows, what fraction did we correctly leave alone?*

$$\text{specificity} = \frac{\text{TN}}{\text{TN} + \text{FP}} \quad (20)$$

Precision (positive predictive value) — *when we shout “freeze”, how often are we right?*

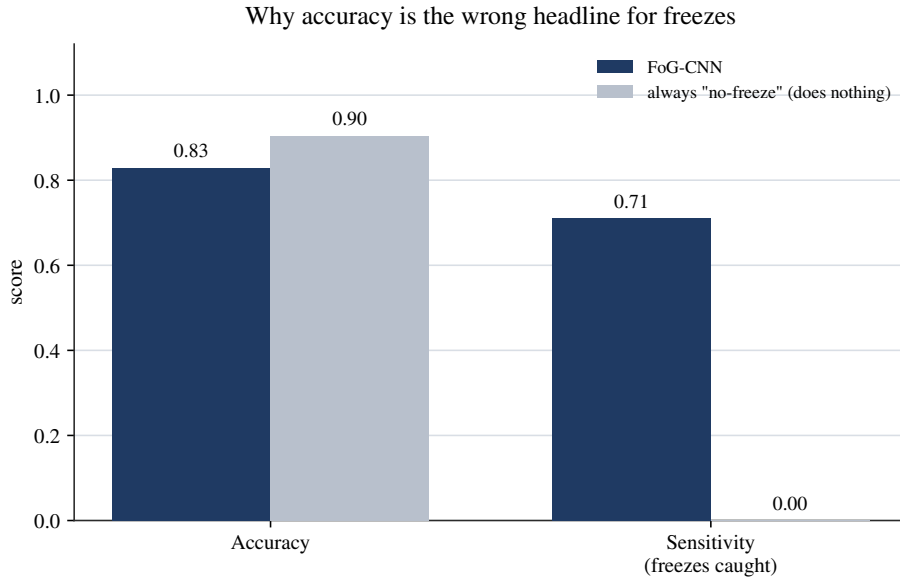
$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (21)$$

Accuracy — *of all windows, what fraction did we get right?*

$$\text{accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (22)$$

The crucial structural difference: sensitivity and specificity each look at *one row* of the matrix, so neither is diluted by how common the other class is. Accuracy pools both rows — and when one class dominates, it is dragged toward whichever rate belongs to the majority.

That last sentence is the whole problem. Freezes occupy under a tenth of the windows in a typical recording, so the no-freeze row dominates the accuracy sum. A detector can score high accuracy while quietly failing at its one job.



The do-nothing model wins on accuracy yet catches 0 freezes — so we report sensitivity & specificity.

Figure 30. The accuracy trap, made concrete. A degenerate “do-nothing” detector that labels *every* window no-freeze scores 0.90 accuracy — it beats our real CNN’s 0.83 — yet its sensitivity is exactly 0.00: it catches not a single freeze. Optimising for accuracy would actively reward the most useless possible monitor. This is why Cadence reports sensitivity and specificity, never accuracy alone.

Takeaway

For a rare event, **accuracy is a vanity metric**. A monitor that never flags a freeze scores ~ 0.90 accuracy and 0.00 sensitivity — useless, but it “looks” right. Cadence is judged on the pair (sensitivity, specificity): freezes caught, and calm gait correctly left alone.

11.3 The real numbers: FoGNet under leave-one-subject-out

The headline evaluation for the CNN upgrade uses *leave-one-subject-out* (LOSO) cross-validation on the public Daphnet FoG dataset: train on every subject but one, test on the held-out subject, rotate through all subjects, then pool every held-out window into one confusion matrix. LOSO is deliberately harsh — the model is always scored on a person it has never seen — which is exactly the situation a new patient strapping on the device represents.

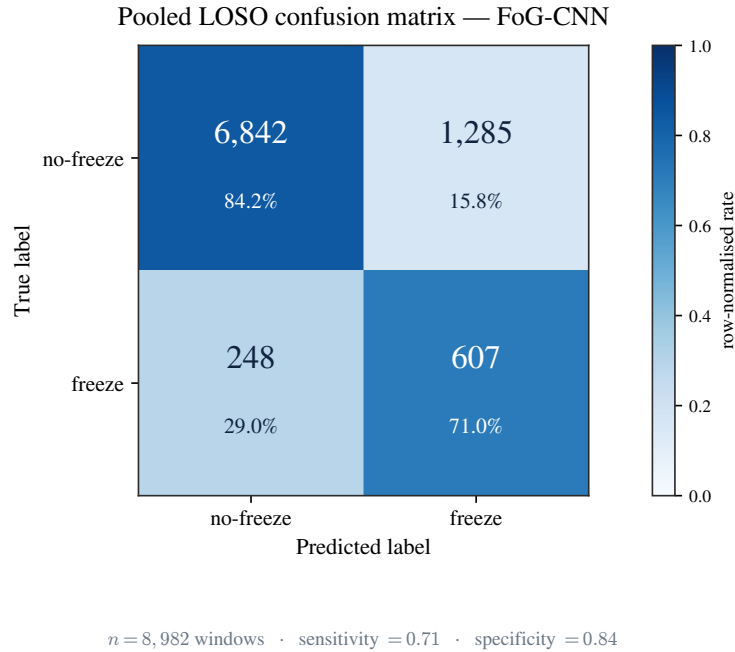


Figure 31. Pooled LOSO confusion matrix for FoGNet over $n = 8982$ held-out windows. Counts are TN = 6842, FP = 1285, FN = 248, TP = 607; the percentages are each row normalised to its own total (so the top row reads as specificity, the bottom as sensitivity).

Plugging those four counts into Equations (19)–(22) gives the operating point of the shipped CNN.

Table 9. FoGNet pooled-LOSO metrics, computed from the counts in Figure 31. Accuracy (0.83) sits *below* the do-nothing baseline of Figure 30 — the clearest possible sign that accuracy is not the metric to chase here.

Metric	Value	From the counts
Sensitivity	0.71	$607 / (607 + 248)$
Specificity	0.84	$6842 / (6842 + 1285)$
Accuracy	0.83	$(607 + 6842) / 8982$

Intuition

Read the matrix as a story about one held-out patient cohort. Of every 100 true freeze windows, FoGNet logs about 71 and lets 29 slip past. Of every 100 calm-gait windows it correctly stays quiet on 84 and cries wolf on 16. Neither number is perfect — this is a tiny model on a hard, noisy, real-world signal — but both are far above the chance line, and crucially neither is hidden by the dominance of the calm-gait class.

11.4 Sweeping the trip line: ROC, PR and Youden’s J

Every metric so far assumed a *fixed* threshold. But a detector that emits a score — a freeze probability $\hat{p}$, or the raw freeze index — has a whole family of operating points: slide the threshold down and you catch more freezes (sensitivity up) at the price of more false alarms (specificity down). Two curves trace that trade-off across all thresholds at once. We illustrate them on the threshold-free RandomForest glass-box, whose continuous score makes the sweep clean.

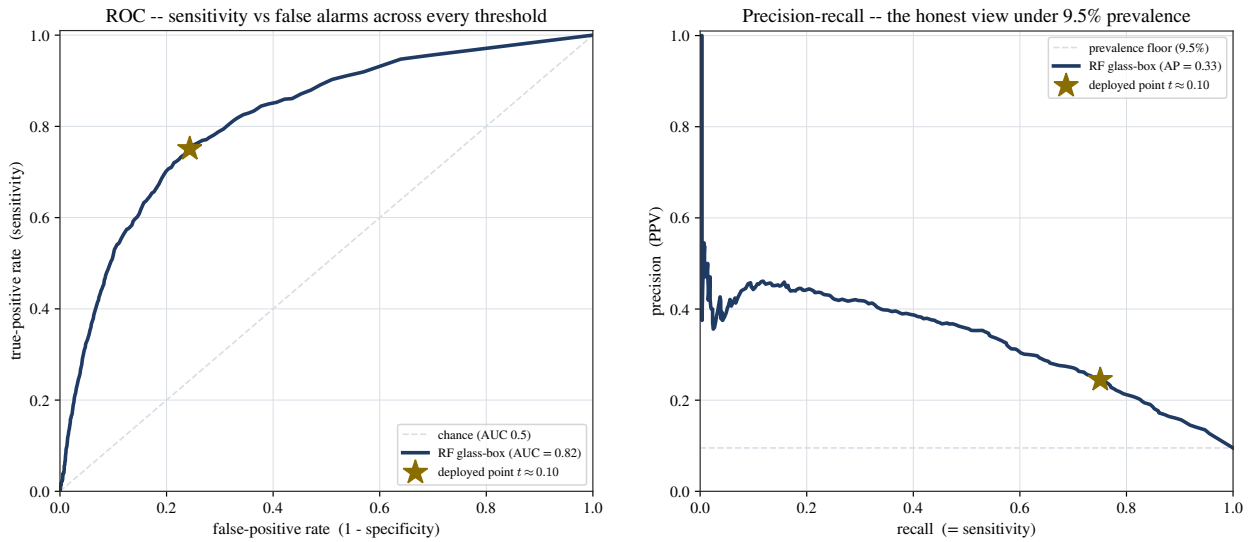


Figure 32. Two threshold sweeps for the RandomForest glass-box. **Left:** the ROC curve plots sensitivity against the false-alarm rate (1 – specificity) as the threshold slides; the dashed diagonal is a coin flip. Area under the curve, $AUC = 0.82$, summarises ranking quality in one number. **Right:** the precision–recall curve, the honest view under 9.5% freeze prevalence; its average precision is only $AP = 0.33$ against a no-skill floor of 0.095. The gold star marks the deployed operating point ($t \approx 0.10$, sensitivity 0.75, specificity 0.76) on both.

Key idea

ROC and PR answer different questions, and the gap between $AUC = 0.82$ and $AP = 0.33$ is not a contradiction — it is the rarity of freezes showing up twice. **ROC** asks “does the score *rank* freezes above calm gait?” and is blind to how many of each there are, so it looks healthy. **Precision** asks “when I shout freeze, am I right?”, and because true freezes are scarce, even a few false positives drown the precision. PR is the metric that refuses to flatter a rare-event detector.

Why does the ranking look so much better than the precision? Because the score itself *does* separate the classes — it just cannot do so perfectly. Figure 33 shows the two score distributions overlaid: calm-gait windows pile up near zero, freeze windows are pushed to the right, but the tails overlap. *No* single threshold cleanly splits them; every choice trades one error for the other.

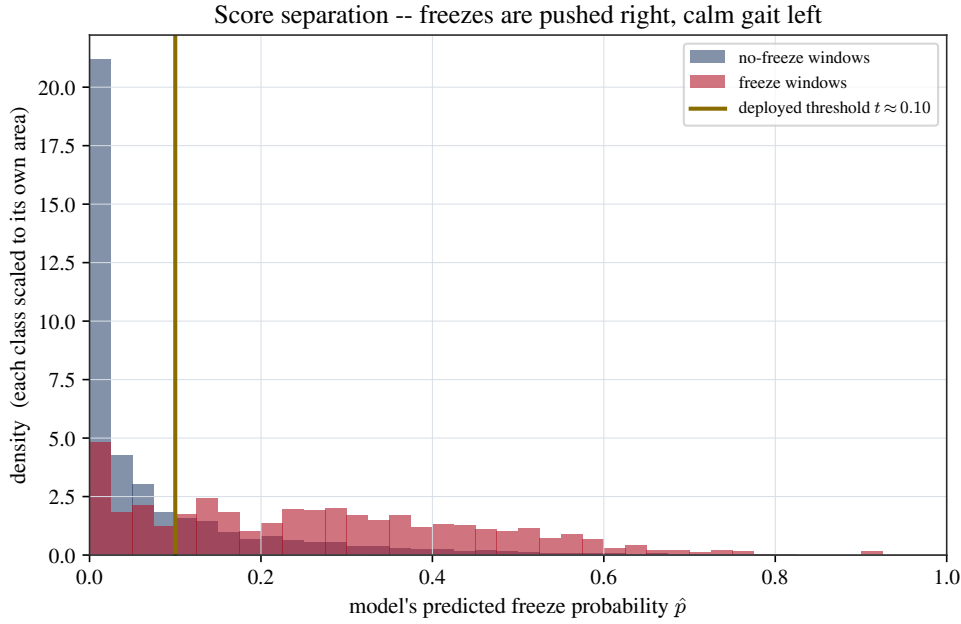


Figure 33. Score separation for the glass-box. Each class is scaled to its own area so the rare freeze class is visible. No-freeze windows (grey) cluster at low predicted probability; freeze windows (red) spread right. The overlap is exactly the region where any threshold must choose between a false positive and a false negative; the gold line marks the deployed $t \approx 0.10$.

Choosing the operating point. Picking a threshold is picking one point on the ROC curve. A principled, prevalence-free default is **Youden’s J statistic**, which rewards the threshold that buys the most sensitivity and specificity together:

$$J = \text{sensitivity} + \text{specificity} - 1, \quad (23)$$

maximised at the ROC point furthest above the chance diagonal. J ranges from 0 (useless) to 1 (perfect), and it treats a missed freeze and a false alarm as equally bad. For the on-board freeze-index rule on the worksheet capture, the J -optimal trip line is $\text{FI} > 2.10$ — which on that capture gives perfect specificity (1.00) at sensitivity 0.83.

Takeaway

Cadence does *not* ship the Youden optimum. Because a missed freeze can mean a fall, the board biases toward sensitivity and trips at $\text{FI} > 1.815$ — buying back a missed-freeze margin (sensitivity 0.83) at the cost of one extra false alarm (specificity 0.92 instead of 1.00). Youden tells you the balanced point; clinical asymmetry tells you to lean past it. The headline is always the pair (*sensitivity, specificity*) — never accuracy.

12 Interpretability: Opening the Box

Cadence ships a detector you can read like a recipe: gate the leg as moving, compute the Freeze Index, compare it against a single printed number. Nothing is hidden. But the project also explored two richer learners — the FoGNet CNN of the previous section and, sitting between the two extremes, a small *glass-box* RandomForest built from nine hand-named features. This section uses that forest as a worked example of *interpretability*: a set of tools that pry open a trained model and ask *which inputs it actually leans on, and in which direction*. We then return to the design question that matters clinically — why a wearable that warns a person about their own falls should prefer a rule it can defend over a network it cannot.

Key idea

A **black box** (the CNN) maps a raw window straight to $P(\text{freeze})$; its 18 k weights are accurate but mute — they cannot tell you *why*. A **glass box** reasons over named quantities a clinician already understands (jerk, dominant frequency, band powers), so every verdict comes with an audit trail. Cadence’s shipped Freeze Index is the most transparent box of all: one ratio, one threshold, fully traceable by hand.

12.1 The interpretable alternative: a 9-feature RandomForest

Instead of feeding the network 256 raw samples per axis, the forest is handed nine summary numbers computed from the same 4 s window: the band powers Cadence already measures (`loco_power`, `freeze_power`, `tremor_power`, `total_power`), the FI itself (`freeze_index`), the magnitude statistics `mag_rms` and `mag_range`, the dominant frequency `dom_freq`, and `jerk_rms` (the RMS of the magnitude’s time-derivative — how abruptly the leg’s acceleration changes). A RandomForest is an ensemble of decision trees, each a chain of `if feature > threshold` splits; every split is a question you can read aloud. Under nested leave-one-subject-out cross-validation, at its operating point $t^* \approx 0.10$ the forest reaches **sensitivity 0.75** and **specificity 0.76**, and as a threshold-free ranker it scores **ROC-AUC 0.82** (PR-AUC 0.33). That is a balanced, honest detector — and, crucially, one whose decisions we can interrogate with the three tools below.

Takeaway

The forest trades a sliver of the CNN’s raw ceiling for something the network cannot offer at any price: every prediction decomposes into contributions from features a neurologist can name.

12.2 Permutation importance: break a feature, watch the score fall

The first question of any model is *which inputs does it actually use?* Permutation importance answers it by sabotage. Take a trained model and a held-out set; shuffle one feature’s column across all windows — so that feature now carries pure noise while every other feature, and the model, stay exactly as they were — and re-score. If balanced accuracy collapses, the model was relying on that feature; if nothing happens, the feature was decoration.

The maths, unpacked

For feature j , importance is the *drop* in balanced accuracy when its column is randomly permuted:

$$I_j = \text{BA}_{\text{intact}} - \mathbb{E}[\text{BA}_{j \text{ shuffled}}]. \quad (24)$$

The expectation is taken over several shuffles (the error bars in Fig. 34 are their spread) and averaged across the held-out subjects, so I_j reads in the same units as accuracy: “shuffling `jerk_rms` costs the forest about 0.05 of balanced accuracy.” Permuting rather than *dropping* the column is deliberate — it preserves that feature’s marginal distribution and only destroys its *link to the label*, isolating the information the model drew from it.

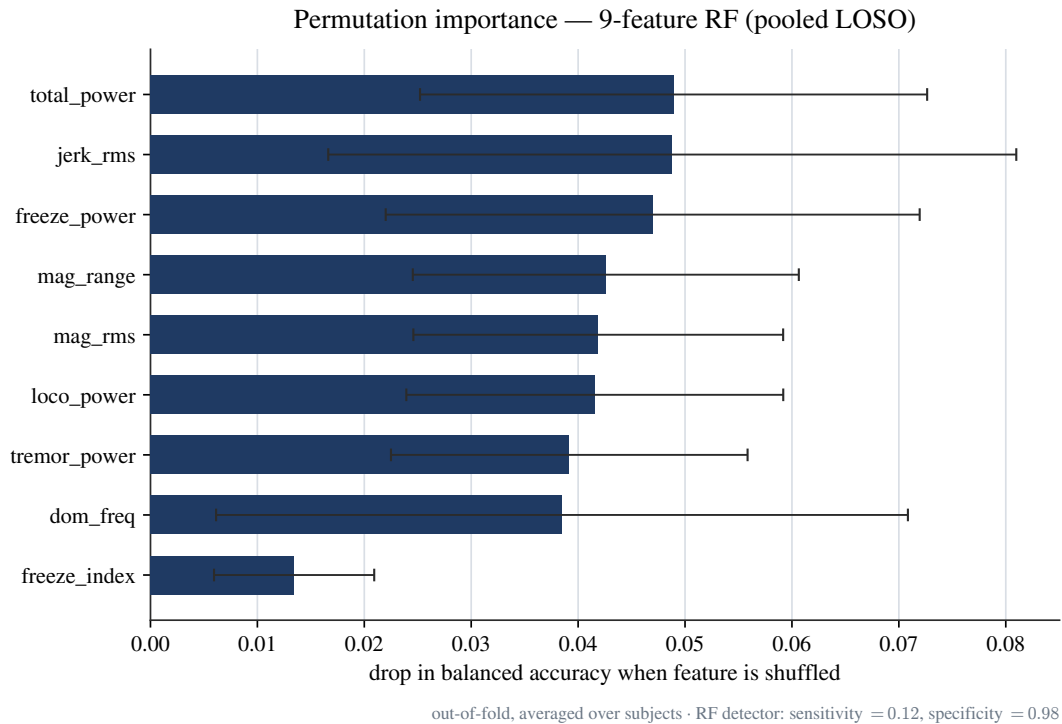


Figure 34. Permutation importance for the 9-feature forest (pooled LOSO, averaged over subjects; bars are the mean drop in balanced accuracy, whiskers the spread over shuffles). Energy-and-motion features — `total_power`, `jerk_rms`, `freeze_power` — carry the most weight. Strikingly, the hand-built `freeze_index` lands *last*: once the forest already has the raw band powers and jerk, the FI ratio adds little *marginal* information. The footnote operating point (sensitivity 0.12, specificity 0.98) is the forest at a naive default threshold — a vivid preview of §12.5 on why the threshold, not the model, sets the trade-off.

Two lessons fall out of Fig. 34. First, the forest’s muscle is in raw motion energy and abruptness, not in the engineered ratio — a reminder that a clever hand-feature can be redundant once a learner sees its raw ingredients. Second, that `freeze_index` ranks low *inside the forest* is not a verdict against the FI: importance is always *marginal* and *model-specific*. In Cadence’s shipped detector the FI is not one voice among nine — it *is* the decision.

12.3 SHAP: why *this* window was called a freeze

Permutation importance is a global headline: it ranks features over the whole cohort. SHAP (SHapley Additive exPlanations) is the close-up. For a single window it splits the model’s output into a sum of per-feature contributions, each signed: a positive SHAP value *pushed this window toward freeze*, a negative one *pulled it back toward calm walking*.

Intuition

Picture the model’s prediction as a bill split fairly among nine diners. SHAP borrows the Shapley value from cooperative game theory: it asks how much each feature *adds* to the prediction, averaged over every order in which features could be revealed. A window with high jerk and a 3–8 Hz dominant frequency gets a large positive “jerk” contribution and a positive “frequency” contribution that *add up* to a high $P(\text{freeze})$ — and you can read the receipt line by line.

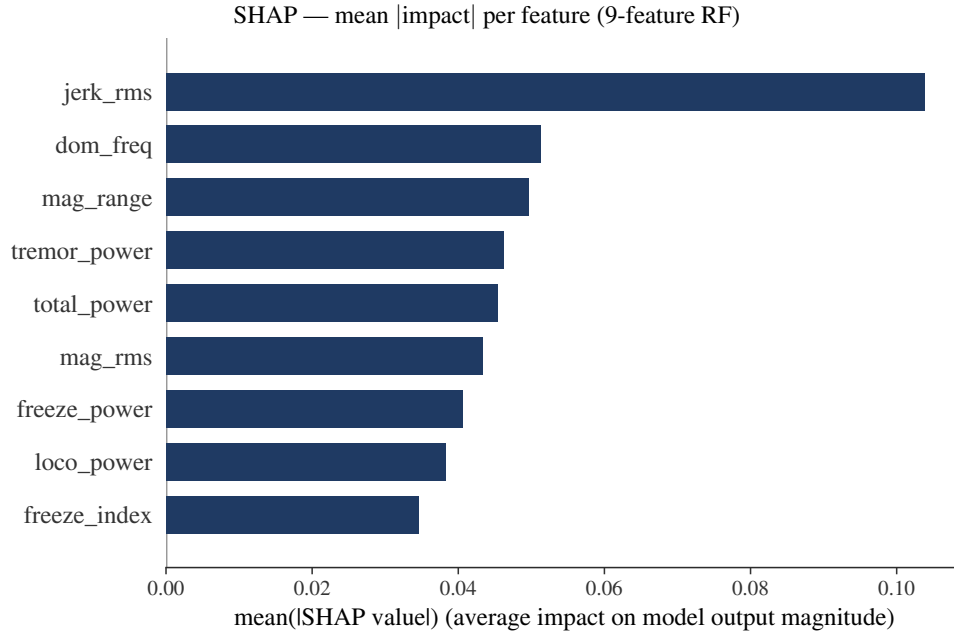


Figure 35. SHAP global summary: mean |SHAP| per feature, i.e. each feature’s average push on the output magnitude across all windows. **jerk_rms** dominates by a wide margin — freezing’s defining 3–8 Hz trembling is, physically, rapid acceleration reversals, which is exactly what jerk measures — followed by **dom_freq** and **mag_range**. The ranking broadly echoes Fig. 34 but is computed by a completely different mechanism (output attribution rather than accuracy sabotage); their agreement is corroboration, not coincidence.

Averaging the magnitude of every window’s SHAP values yields a global bar chart (Fig. 35) that two independent methods — permutation and SHAP — now agree puts motion-and-jerk features at the top. Where SHAP earns its keep is the *per-window* explanation it can also produce: for any flagged second, it states in plain features why the model raised the alarm, which is precisely the artefact a clinician needs to either trust or contest a single alert.

12.4 Partial dependence: which way does the feature push?

Importance tells us *how much* a feature matters; partial dependence tells us *in which direction*. Sweep one feature across its range, and at each value average the model’s prediction over every window’s real settings of the other eight features. The resulting curve is the feature’s marginal effect on $P(\text{freeze})$, with the rest of the data marginalised out.

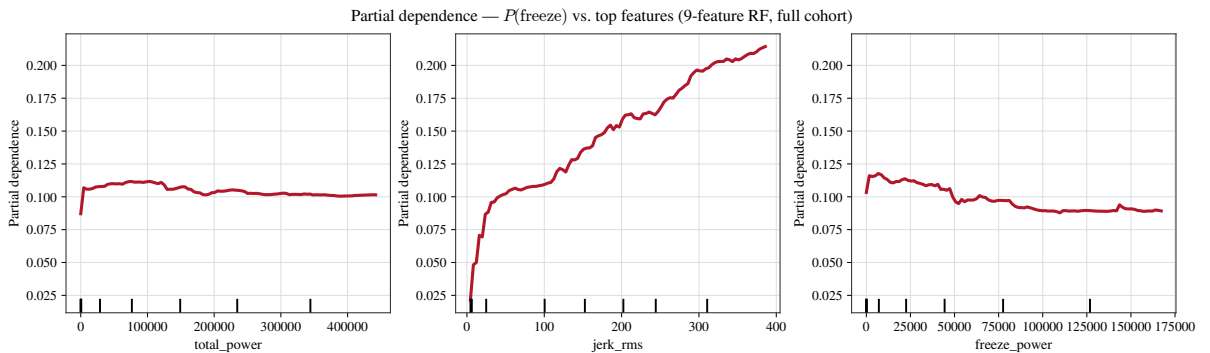


Figure 36. Partial dependence of $P(\text{freeze})$ on the three leading features. **Left:** **total_power** is essentially flat — the forest uses it for *gating* (is the leg moving at all?) more than for direction. **Centre:** **jerk_rms** climbs monotonically; more abrupt acceleration steadily raises the freeze probability, exactly matching the physiology of 3–8 Hz trembling and SHAP’s verdict. **Right:** **freeze_power** drifts gently *down* — on its own, once jerk is accounted for, raw freeze-band energy is not independently incriminating. The rug marks at the foot of each axis show where the data actually live, a reminder to distrust the curve out in the sparse tails.

The monotone `jerk_rms` curve is the satisfying one: it is not merely *important*, it points the physically correct way. This is the litmus test of a trustworthy model — not just that it scores well, but that the reasons it scores well are the reasons a domain expert would give. A model that achieved the same accuracy by riding `jerk_rms` *downward* would be a red flag, however good its numbers.

12.5 The threshold, not the model, sets the trade-off

Interpretability also exposes a trap that pure accuracy hides. Figure 34’s footnote reports the same forest at sensitivity 0.12, specificity 0.98 — it catches almost no freezes — yet the operating point quoted earlier was 0.75 / 0.76. Nothing about the *model* changed between those two numbers; only the decision threshold t did. The freeze class is rare, so the forest’s posterior $P(\text{freeze})$ rarely exceeds the naive $t = 0.5$, and a half-line classifier therefore says “calm” almost always — high specificity, near-blind sensitivity.

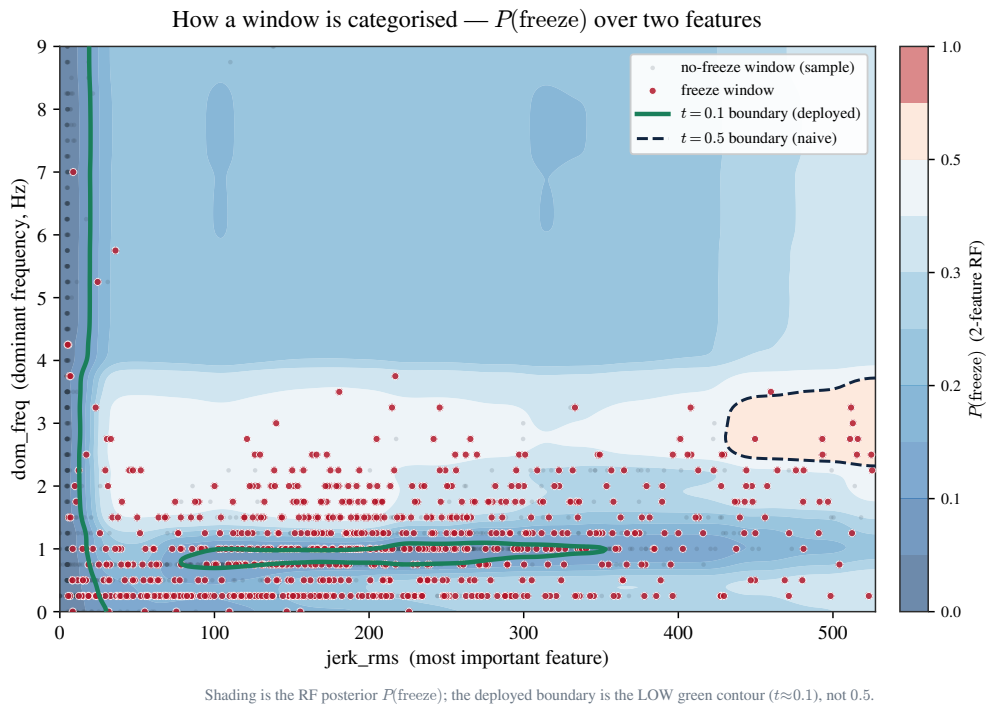


Figure 37. The forest’s posterior $P(\text{freeze})$ shaded over its two most telling features, with real windows overlaid (red = freeze, grey = no-freeze). The **dashed** contour is the naive $t=0.5$ boundary: it walls off only a tiny high-jerk corner, abandoning most red points — this is the sensitivity-0.12 detector. The **solid green** contour is the deployed $t \approx 0.10$ boundary, which sweeps far lower and reclaims the freeze cluster strung along high `jerk_rms`. Same model, same plot — moving one number transforms a timid detector into a balanced one.

Takeaway

For a rare, high-cost event, the operating threshold *is* a design decision, not a default. A missed freeze can mean a fall, so both the forest ($t^* \approx 0.10$) and the shipped FI detector ($t = 1.815$, below the Youden-optimal 2.10) are deliberately biased toward sensitivity — and being able to *see* that choice in Fig. 37 is itself an argument for glass boxes.

12.6 Why Cadence ships the Freeze Index, not a black box

The forest is interpretable; the FI detector is interpretable *and* already on the ankle. Three clinical reasons settle the choice.

Trust.

A person is being told something about their own falls. “You froze because your 3–8 Hz energy was $1.8\times$ your walking energy” is a sentence a patient and a clinician can both check; “the network’s logit crossed zero” is not. Every Cadence verdict on the dashboard — the Freeze Margin meter, the raw FI, the energy gate — traces back to one printed equation.

Debuggability.

When the FI detector misfires we can point at the exact cause — the gate let through a still-leg artefact, or the window straddled a freeze boundary and diluted it — and fix that line. A misfiring CNN offers 18k weights and a shrug. (Indeed, the design lesson that per-axis band-pass filtering *before* magnitude flattens the FI ratio was found precisely because the pipeline was transparent enough to inspect.)

Footprint and honesty.

The FI is two band-power sums and a ratio, well inside the SAMD21’s budget, and it runs untethered with no training set to drift away from. The forest and CNN are research instruments that *justify* and stress-test the shipped rule; the rule itself is what a clinician can audit at the bedside.

Key idea

Interpretability is not a consolation prize for a weaker model — it is the headline feature of a medical wearable. Cadence treats the CNN and RandomForest as *evidence* that its simple, transparent Freeze Index is sound, then ships the box everyone can see straight through.

13 Hardware

Everything in the earlier sections — the magnitude, the Welch spectrum, the freeze index, the gate-and-debounce state machine — has to run on something small enough to live on an ankle and cheap enough to build in a one-week sprint. This section describes that physical build: the single sensing board, the one external display, the optional wireless bridge, how it all sits inside an ankle sleeve, and the two firmware “personalities” the same board can wear.

Key idea

Cadence is a **monitor**, not a therapy. A coin vibration motor is physically fitted to the build, but it is **disabled in firmware**: the device never buzzes, cues, or stimulates. Every freeze it finds is *reported* — on an OLED, on the onboard LEDs, and on a live web dashboard — and nothing more. Read every mention of the motor below as “present on the board, held off in software.”

13.1 The one board that does the work: Circuit Playground Express

The heart of Cadence is a single **Adafruit Circuit Playground Express (CPX)**. It is a 51 mm disc that already carries everything the core detector needs, so the sensing, the maths, and the local display all happen on one part:

- a **SAMD21** Arm Cortex-M0+ microcontroller — the CPU that runs the DSP;
- an onboard **LIS3DH** 3-axis MEMS accelerometer (I^2C), the only sensor Cadence uses, sampled at **64 Hz** on the ± 8 g range;
- a ring of **10 NeoPixels** that we drive as a colour-coded state light;
- two push-buttons and a **slide switch** we repurpose as mode/enable inputs;
- 2 MB of SPI flash for untethered logging.

Intuition

Why one disc instead of a pile of breakout boards? In a one-week wearable sprint the enemy is wiring. The CPX folds the MCU, the accelerometer, and the status LEDs into a part you can sew flat against the limb with *zero* jumper wires for the core job. The only wires we add are the four that drive an external screen — and even those are optional.

The $\pm 8\text{ g}$ range is a deliberate choice, not a default. Heel-strike transients at the ankle routinely exceed $\pm 2\text{ g}$, so a $\pm 2\text{ g}$ setting would clip the very peaks that separate vigorous gait from a freeze. The 64 Hz sample rate gives a Nyquist limit of 32 Hz, which sits comfortably above the highest band of interest (the 3–8 Hz freeze band, and the 4–6 Hz rest-tremor band), so nothing of clinical interest aliases.

13.2 Pins and connections

The wiring is dominated by one addition: an external **SSD1306** OLED for the live freeze read-out. The CPX's onboard I²C bus is already busy with the LIS3DH accelerometer, so the OLED is driven on a *second* I²C bus brought out on the CPX pads — **SDA = A5**, **SCL = A4** — using a separate `Wire` instance. Keeping the screen off the sensor's bus means a slow or chatty display can never stall an accelerometer read. Figure 38 gives the full pad-by-pad map of the build.

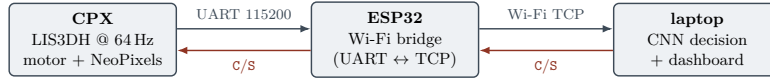
Pins & Connections

Parkinson's Closed-Loop Gait Garment — ENGF0031 Scenario 2

Circuit Playground Express + ESP32 (Wi-Fi bridge) + laptop (wireless)

The wearable is now **untethered**. The Circuit Playground Express (CPX) does all the sensing and drives the cue; a small **ESP32** relays data to and from the laptop over **Wi-Fi**; and the laptop runs the CNN that makes the freeze decision. CPX and ESP32 talk over a short **UART** link — both are 3.3V logic, so they connect directly with *no level shifter*. The motor-driver wiring is unchanged from the earlier builds; only the *link to the laptop* has changed (a Wi-Fi hop via the ESP32 instead of a USB cable).

System overview — where the data goes



Accelerometer stream flows right (**ax,ay,az** at 64 Hz); the freeze-cue command (C/S) returns left.

Board pin map — Circuit Playground Express

Pin / pad	Direction	Connects to	Purpose
A1	digital out	1 k Ω → Q1 base	switches the vibration motor (the cue)
A7	UART TX (out)	ESP32 GPIO16 (RX2)	Serial11 TX — streams ax,ay,az to the ESP32
A6	UART RX (in)	ESP32 GPIO17 (TX2)	Serial11 RX — receives the C/S cue
VOUT	power (3.3 V)	motor (+), diode cathode	board-regulated rail; supplies the motor
GND	power (0 V)	Q1 emitter, motor return	common ground — shared with the ESP32
A0	—	left open (unused)	onboard speaker/DAC pin — avoid
micro-USB	5 V in	USB power bank / charger	power only now; optional debug serial
Button A	onboard input	—	hold still + press to calibrate the energy floor
Slide switch	onboard input	—	cue master-enable (wearer can silence it)
NeoPixels ×10	onboard output	—	state indicator (see legend below)
LIS3DH accel	onboard (I ² C)	—	3-axis accelerometer, sampled at 64 Hz

CPX ↔ ESP32 — the UART link

Three wires carry exactly the bytes the old USB cable used to. **TX** and **RX** **cross over**, and the two boards **must share a ground**. Both run at 3.3 V, so the lines connect directly — no level shifter, no divider.

Figure 38. Pad-by-pad wiring of the Cadence build. The CPX senses, runs the freeze-index detector, drives the onboard NeoPixel ring, and brings out a second I²C bus (A5/A4) for the external SSD1306 OLED; three UART wires (A7/A6 + a shared ground, 3.3 V logic, no level shifter) form the optional link to an ESP32 Wi-Fi bridge. *Note:* this sheet is inherited from an earlier closed-loop iteration and still annotates the coin motor as a “cue” on A1; in shipped Cadence firmware that motor pin is held off — the device is a monitor.

Table 10. Key connections on the Circuit Playground Express. The first block is the always-present core; the second block is the optional wireless bridge.

CPX pin / part	Direction	Role in Cadence
LIS3DH accel	onboard I ² C	3-axis sensor, 64 Hz, ± 8 g — the only sensor used
NeoPixels $\times 10$	onboard out	state-colour ring (still / walking / freeze)
Slide switch	onboard in	selects RECORDER vs. MONITOR role
Button A	onboard in	hold-still calibration of the still-floor gate
A5 (SDA2)	out (I ² C)	external SSD1306 OLED data
A4 (SCL2)	out (I ² C)	external SSD1306 OLED clock
A1	out (held off)	coin vibration motor — disabled in firmware
GND	power (0 V)	common ground (shared with the ESP32)
A7	UART TX	a_x, a_y, a_z stream out to ESP32 (optional bridge)
A6	UART RX	control bytes in from ESP32 (optional bridge)
micro-USB	5 V in	power; also the USB link for log download

13.3 Telling the wearer: NeoPixels and the OLED

Cadence has two local outputs, both read-only indicators of the same state machine:

- the **NeoPixel ring** is a glanceable colour light — it shows the headline STILL / WALKING / FREEZE verdict at arm’s length, with red reserved for a committed freeze;
- the **SSD1306 OLED** carries the detail the LEDs cannot: the live freeze index, the movement-energy gate state, and a text label of the current phase.

Both are downstream of the gate $\rightarrow$ FI $\rightarrow$ debounce pipeline of Section 8; neither feeds back into the decision. Putting the screen on its own bus (A5/A4) is what lets the OLED be as slow as it likes without ever delaying a 64 Hz accelerometer sample.

13.4 Going wireless: the optional 3-board topology

The standalone build is fully untethered: the CPX senses, decides, and displays on the ankle, powered from a small battery over USB. For a richer, screen-rich *live* view — the web dashboard of Section 15 — Cadence offers an optional three-board topology that adds an **ESP32** as a Wi-Fi bridge between the ankle and a laptop.

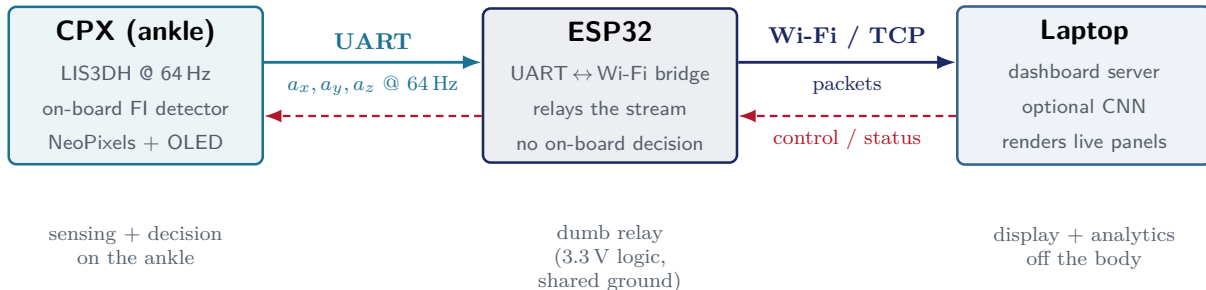


Figure 39. The optional 3-board topology: **CPX** $\xrightarrow{\text{UART}}$ **ESP32** $\xrightarrow{\text{Wi-Fi/TCP}}$ **laptop**. The accelerometer stream flows right (solid); thin control/status bytes return left (dashed). The ESP32 is a pure bridge — it makes no decisions. Because both the CPX and ESP32 run at 3.3 V logic and share a ground, the UART wires connect directly with no level shifter.

Intuition

The split is “decide near the sensor, display far from it.” The freeze decision is small and latency-sensitive, so it stays on the ankle. Pixels, charts, and any heavier model are big and not safety-critical, so they live on a laptop the wearer need not carry. The ESP32 in the middle is deliberately dumb: it shuttles bytes and nothing else, which keeps the part that matters — the detector — on one well-tested board.

13.5 Wearing it: the ankle sleeve

Cadence is built into a snug compression ankle sleeve with flat internal pockets, so nothing dangles and the sensor is firmly coupled to the limb. Firm coupling matters: a loose sensor would swing like a pendulum and pollute the very band the freeze index reads. The CPX and the battery sit in opposite flat pockets so their masses balance near the limb axis, the optional motor sits in a thin pocket against the skin, and the whole assembly is light. Figure 40 shows the layout from the side, in cross-section, and as a flat-pocket layer stack.

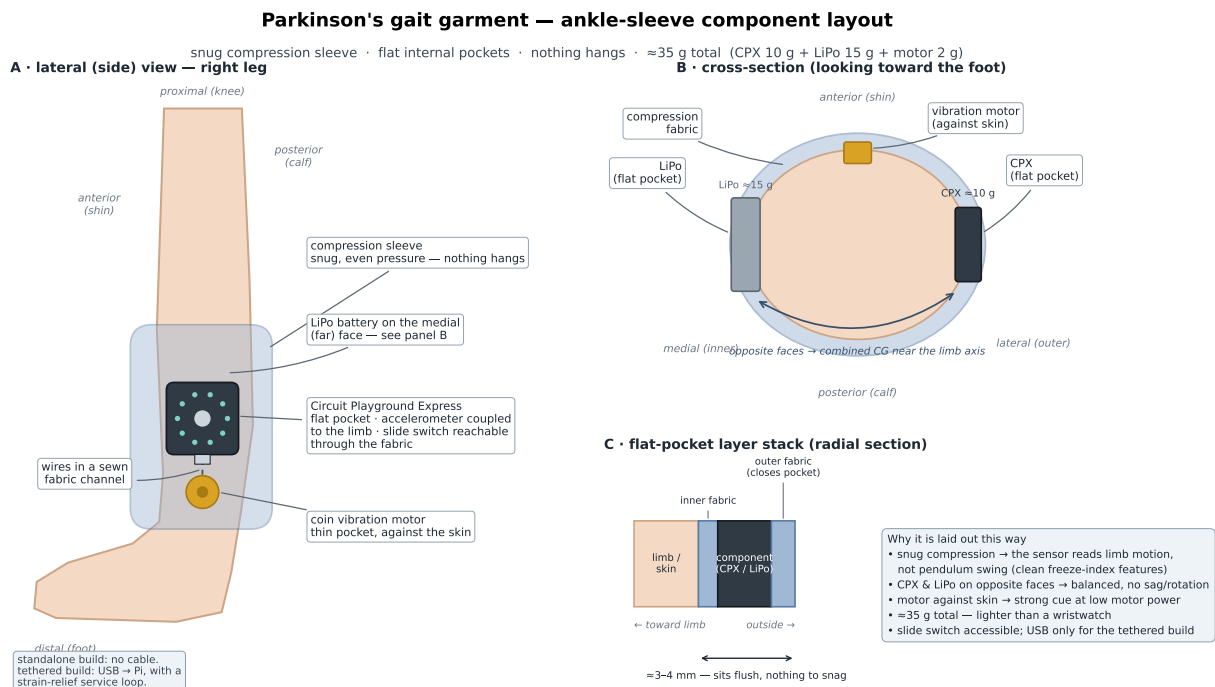


Figure 40. Ankle-sleeve integration: (A) lateral view on the right leg, (B) cross-section looking toward the foot, (C) the flat-pocket layer stack. The sleeve is snug so the accelerometer reads true limb motion rather than pendulum swing; CPX and battery sit on opposite faces to balance; the build sits flush with nothing to snag. The vibration motor shown against the skin is fitted but, in Cadence’s monitor firmware, held off.

Power is a small LiPo cell delivered to the CPX over its USB connector; the same USB connection doubles as the link for pulling logged data off the board. In the wireless build the ESP32 is powered alongside the CPX and shares its ground, which is exactly why the UART lines need no level translation.

13.6 Two firmware roles on one board

The same CPX wears one of two firmware “personalities,” chosen so the project can both *gather* labelled data and *run live* without changing hardware. The slide switch selects between them.

Table 11. The two firmware roles. One board, one accelerometer — two jobs.

	RECORDER	MONITOR
Sketch	<code>cpx_fog_logger.ino</code>	<code>cpx_fog_standalone.ino</code>
Purpose	capture raw accel + auto-labelled phases for analysis	live freeze detection and display
Trigger	flip the slide switch to start/stop logging	runs continuously on power-up
Stores to Output	2 MB onboard SPI flash, untethered a file pulled later over USB with <code>cpx_dump.py</code>	nothing — decides every 2 s in RAM live OLED text + NeoPixel state colours
Motor	n/a	fitted but disabled

The **RECORDER** role is how the project earns its right to claim real numbers: a clinician or wearer flips the slide switch, walks (and freezes), and the board logs accelerometer samples with auto-labelled phases to flash with no laptop attached. Later the file is pulled over USB with `cpx_dump.py` for the offline analysis behind every result in this report. The **MONITOR** role is the shipped device: it runs the gate $\rightarrow$ FI $\rightarrow$ debounce pipeline live and paints the verdict onto the OLED and LEDs.

13.7 Does it fit on the chip?

A fair question for a Cortex-M0+ with no floating-point unit: can the SAMD21 finish a window of DSP inside its budget? The arithmetic is comfortable. A decision is due only once every **2 s** (the 128-sample hop), and the work per decision is one Hann-tapered Welch PSD over a 256-sample window plus a handful of band-power sums and a single ratio. That is a few thousand multiply-adds spread across a two-second window — well within the SAMD21’s headroom, leaving the CPU mostly idle between decisions to service the 64 Hz sampling, the LEDs, and the OLED. Figure 41 traces that whole MONITOR loop end to end.

Key idea

The cadence of the whole device is set by the 2 s hop, not by raw compute. The long 4 s window is what buys the spectral resolution to separate the 3–8 Hz freeze band from the walking rhythm; the 2 s hop is how often that window is re-scored. Both numbers are about *seeing the freeze clearly*, and both leave the little SAMD21 with time to spare.

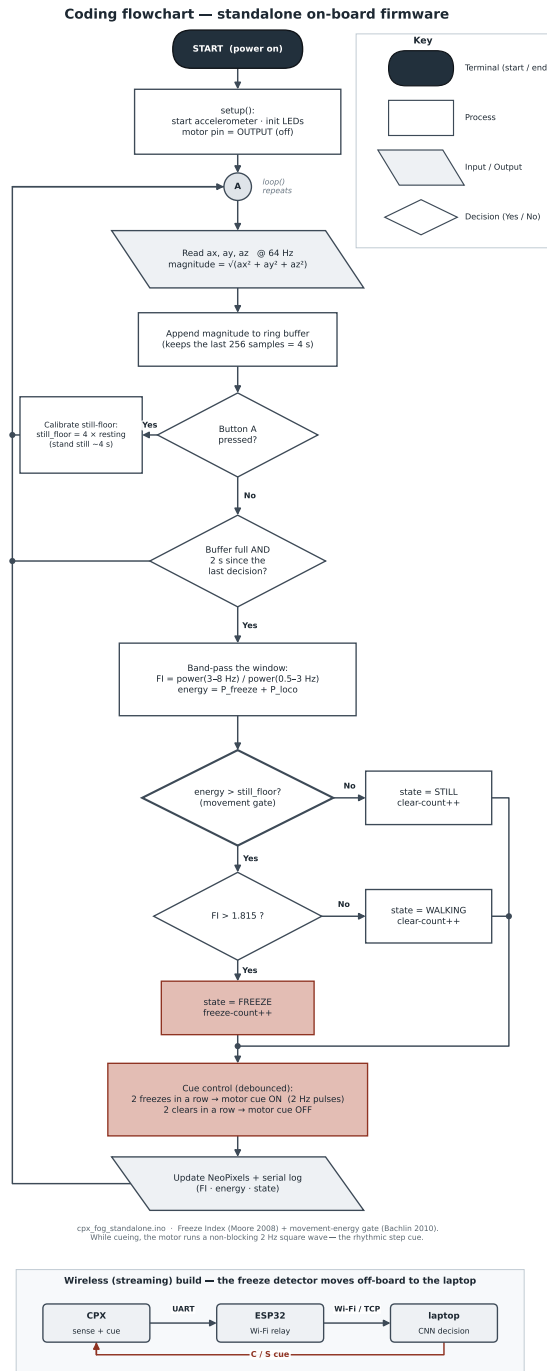


Figure 41. The MONITOR firmware loop (`cpx_fog_standalone.ino`): read the three axes at 64 Hz, form the magnitude, keep the last 256 samples (4 s), and once every 2 s compute the freeze index and movement energy, apply the still-floor gate, the $FI > 1.815$ test and the assert/release-after-two debounce, then update the NeoPixels and log line. *Note:* this diagram is inherited from a closed-loop iteration and still draws a “motor cue” block; in shipped Cadence that output path is held off — only the NeoPixel/OLED/serial update is active.

Takeaway

One 51 mm board does the sensing and the decision; one external OLED and the onboard LEDs report it; an optional ESP32 carries it to a laptop dashboard. The coin motor is on the board but off in code — Cadence watches for freezes and tells you, and that is all.

14 Software

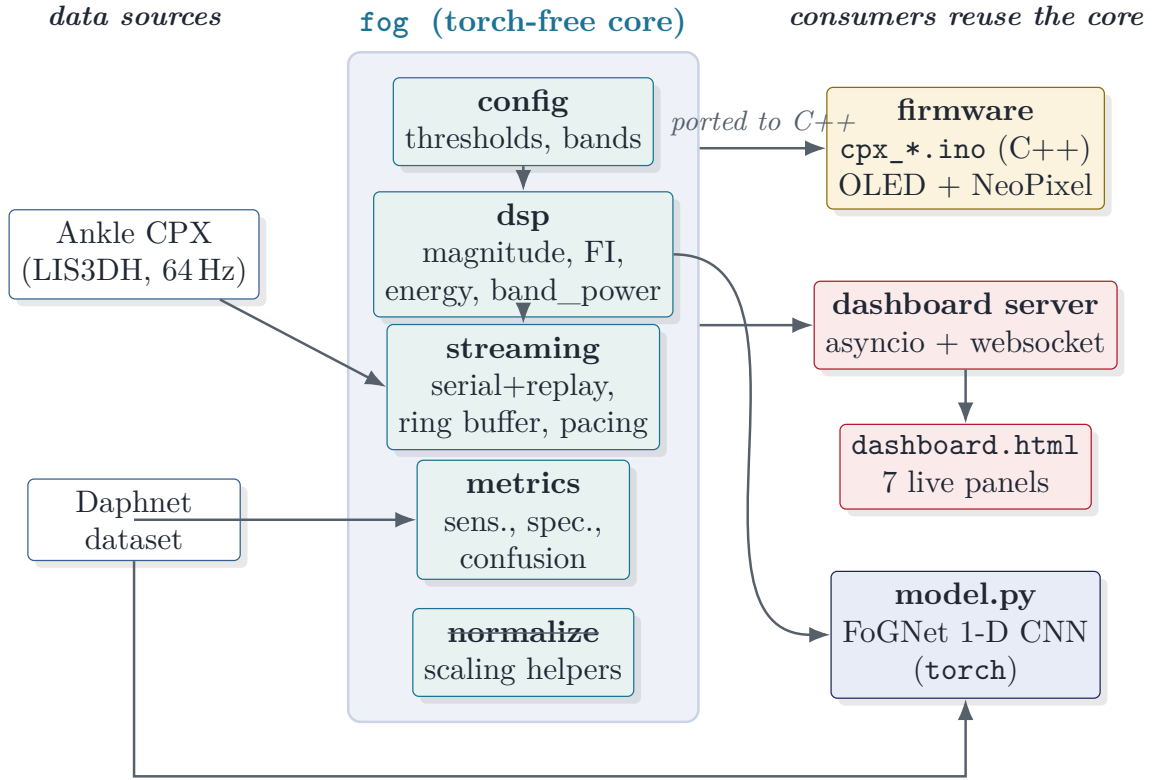
Cadence is small in hardware but rich in software. Everything from the live freeze verdict on the wrist to the leave-one-subject-out study of the CNN is driven by one torch-free Python package, called `fog`, plus a few companions: the optional CNN in `model.py`, the Arduino/C++ firmware, and an `asyncio` dashboard server that streams to a browser. This section walks through how those pieces fit together, what the signal-processing core actually computes, how live samples flow from the ankle to the screen, and a design lesson that quietly shapes the whole pipeline: *the live detector reads raw acceleration on purpose*.

Key idea

One package does the maths twice. The *same* `fog.dsp` functions that score a window in the offline study also score the live stream in the dashboard. There is no second, divergent implementation to keep in sync — the report’s figures and the device’s verdict come from one source of truth.

14.1 Architecture at a glance

The codebase is deliberately layered. At the bottom sits `fog`, a pure-Python package with no machine-learning framework dependency — it needs only NumPy and SciPy. On top of that core, three consumers reuse the same primitives: the firmware (a C++ re-implementation of the detector for the SAMD21), the dashboard server (live display), and the analysis scripts (offline figures). The CNN lives off to one side as a heavier, optional “given more time” upgrade that does pull in `torch`. Figure 42 sketches the modules and the direction data flows between them.



normalize struck through = present in the package but not used in the shipped detection path

Figure 42. Software architecture. The torch-free **fog** core (teal) holds the signal processing and metrics; the firmware, dashboard server, and CNN are thin consumers that reuse it. Live samples enter through **streaming**; the Daphnet benchmark feeds the offline CNN and metrics. Arrows show the direction of data flow.

A short tour of the five core sub-modules:

- **config** — the single place that names the frequency bands (locomotor 0.5–3 Hz, freeze 3–8 Hz, rest-tremor 4–6 Hz), the sampling rate, the window and hop sizes, and the deployed `FI_THRESHOLD`. Nothing else hard-codes a magic number.
- **dsp** — the numerical heart: `magnitude`, `band_power`, `freeze_index`, `movement_energy`, and an `AccelFilter` helper.
- **streaming** — a receiver that reads samples from either a live serial port or a recorded replay, buffers them in a fixed-size ring, and paces replay to wall-clock time so a recording behaves exactly like a live device.
- **metrics** — sensitivity, specificity, precision, accuracy, and confusion-matrix bookkeeping; the vocabulary the evaluation chapters speak in.
- **normalize** — scaling helpers used to standardise feature vectors before they reach the glass-box models.

14.2 The signal-processing core (dsp)

Every quantity the device reports is built from one orientation-invariant scalar. The three accelerometer axes are collapsed into a magnitude $a = \sqrt{a_x^2 + a_y^2 + a_z^2}$, and its mean is subtracted to strip out gravity and any DC offset before spectral analysis. From the mean-removed magnitude, a Hann-tapered Welch power spectral density over a 256-sample (4.0 s) window gives a spectrum with bin spacing $df = f_s/N = 64/256 = 0.25$ Hz. `band_power` then sums the PSD over the bins inside a band, and the two headline quantities follow directly:

$$\text{FI} = \frac{P_{3-8\text{Hz}}}{P_{0.5-3\text{Hz}} + \varepsilon}, \quad \varepsilon = 10^{-9}, \quad (25)$$

$$E_{\text{move}} = P_{0.5-3\text{Hz}} + P_{3-8\text{Hz}}. \quad (26)$$

The maths, unpacked

The Freeze Index in Eq. (25) is a *ratio* of two band powers, so any overall scaling cancels: it reads the same whether the host works in milli-*g* or the device works in m/s². The tiny $\varepsilon = 10^{-9}$ in the denominator is not physics — it is numerical insurance so a perfectly still leg (near-zero locomotor power) cannot trigger a divide-by-zero. Movement energy in Eq. (26) is the *sum* of the same two bands and therefore is *not* scale-invariant; that is deliberate, because it must measure how much the leg is actually moving.

The same `dsp` module is what the C++ firmware mirrors line-for-line. Reproducing `freeze_index` faithfully in Python (Listing 3) is what lets the offline study, the dashboard, and the on-board detector all agree to the bin.

Listing 3. The shipped Freeze Index, faithful to the deployed detector. Magnitude is mean-removed; the ratio is taken over the standard bands with the 10^{-9} guard.

```

1 import numpy as np
2 from .config import FS, FREEZE_BAND, LOCO_BAND, EPS # 64 Hz; (3,8); (0.5,3); 1e-9
3
4 def magnitude(ax, ay, az):
5     """Orientation-invariant scalar, mean removed (drops gravity/DC)."""
6     a = np.sqrt(ax**2 + ay**2 + az**2)
7     return a - a.mean()
8
9 def freeze_index(window):
10    """Moore et al. (2008) Freeze Index on a 256-sample (4 s) window."""
11    p_freeze = band_power(window, *FREEZE_BAND) # power in 3-8 Hz
12    p_loco = band_power(window, *LOCO_BAND) # power in 0.5-3 Hz
13    return p_freeze / (p_loco + EPS) # ratio -> scale-invariant
14
15 def movement_energy(window):
16    """How hard the leg is moving (NOT scale-invariant) -> drives the gate."""
17    return band_power(window, *LOCO_BAND) + band_power(window, *FREEZE_BAND)

```

14.3 Streaming: from ankle to ring buffer

The `streaming` sub-module is what makes the device feel “live”. It exposes one receiver that works two ways. In *serial* mode it reads accelerometer frames off the CPX (or, in the three-board topology, off the ESP32 over TCP) as they arrive at 64 Hz. In *replay* mode it reads the same frames from a recorded capture but *paces* them to wall-clock time, so a saved file plays back at exactly the speed the body produced it. The downstream code cannot tell the difference — which means the dashboard can be demonstrated and debugged off a recording, then run unchanged on the real ankle.

Incoming samples land in a fixed-size *ring buffer* that always holds the most recent 256 samples (4.0s). Every 128 samples (2.0s) the server lifts the current window out of the ring and scores it — this is the 50% overlap that yields one decision every two seconds. A ring buffer is the right structure here: it is constant-memory, never grows, and overwrites the oldest sample as each new one arrives, which is exactly the “sliding window over a never-ending stream” the live detector needs.

Intuition

Think of the ring buffer as a 4-second-wide window you slide along the incoming signal. It is always full and always current. Every two seconds you photograph what is inside it and ask one question — “does this look like a freeze?” — then slide on. The overlap means a freeze that starts mid-window still gets two looks before it ends.

14.4 The dashboard server’s control loop

The dashboard server is an `asyncio` program that ties the stream to the browser. One coroutine fills the ring buffer from `streaming`; another runs the control loop every hop, scores the window with `fog.dsp`, decides a state, and pushes a small JSON payload to `dashboard.html` over a websocket. The client renders seven live panels — gait status, freeze margin, the raw Freeze Index, tremor power (4–6 Hz), movement energy, locomotion power, and the raw 3-axis waveform.

The state decision itself is the “glass box”: three explainable stages in series — a still-floor *gate*, the FI threshold, and a *debounce*. The gate forces STILL whenever movement energy is below an auto-calibrated floor, because a motionless leg has near-zero energy yet sensor noise can spike the ratio. Only if the leg is genuinely moving does the loop compare the Freeze Index to the deployed threshold of 1.815. Listing 4 shows the raw per-window verdict before debounce.

Listing 4. The gated state decision (one window, pre-debounce). The gate runs first so a still leg can never be called a freeze.

```

1 def decide_state(window, still_floor, fi_threshold=1.815):
2     energy = movement_energy(window)
3     if energy < still_floor: # gate: not enough motion
4         return "STILL"
5     if freeze_index(window) > fi_threshold:
6         return "FREEZE" # moving AND high freeze-band ratio
7     return "WALKING" # moving but rhythmic, low FI

```

A raw per-window verdict can still flicker on a single noisy bump, so the committed state is *debounced*: a new state must persist for two consecutive control windows before it is accepted (`assert-after-2`), and likewise must clear for two before it is released (`release-after-2`). One stray window never flips the headline. The *freeze margin* panel surfaces the same logic as a confidence meter: it plots the live FI as a percentage of the 1.815 trip line (100% means at or over the trip), and shows “—/at rest” whenever the gate has forced STILL.

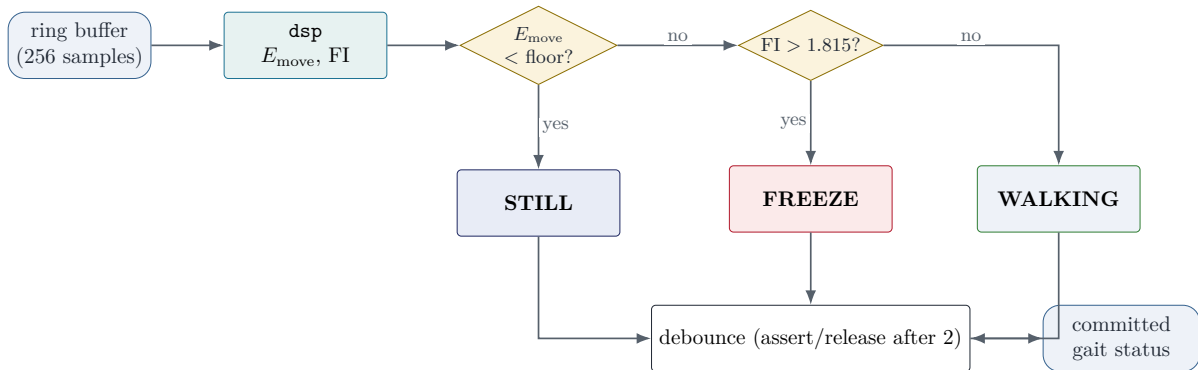


Figure 43. The dashboard control loop, run once per 2-s hop. The still-floor gate acts before the FI test, so a motionless leg is never called a freeze; the raw verdict is then debounced (`assert/release after 2`) into the committed gait status shown on the wrist and the web panel.

14.5 The offline data pipeline

Alongside the live path is a reproducible batch pipeline that turns raw recordings into the figures in this report. It runs in four stages: **capture** (log raw accel and auto-labelled phases to the

CPX flash, then pull them over USB with `cpx_dump.py`); **daphnet** (ingest the public Daphnet FoG benchmark for the model study); **analyze** (window each capture, score it with the *same* `fog.dsp` functions, sweep the FI threshold, and run `metrics`); and **figures** (emit the PDFs the report includes). Because **analyze** calls the identical `dsp` that the dashboard does, the threshold sweep that produced the deployed 1.815 is the same code that runs on the live stream.

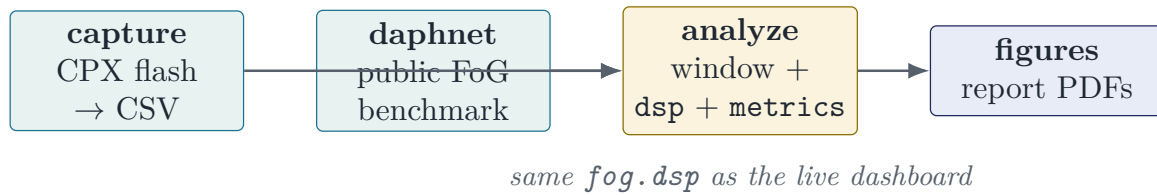


Figure 44. The offline pipeline: **capture** → **daphnet** → **analyze** → **figures**. The **analyze** stage reuses the live signal-processing core, so the offline study and the on-body device never disagree.

14.6 Design lesson: the live detector reads raw acceleration

An obvious-looking “improvement” would be to band-pass filter each axis before taking the magnitude, to clean up the signal. The project tried it, and it *quietly broke the detector*. Filtering each axis before combining them flattens the FI ratio: once the bands are pre-shaped per axis, walking and freezing produce similar ratios, and the very contrast the Freeze Index relies on disappears. The fix was to leave the live FI path *deliberately unfiltered* — magnitude is taken on the raw accelerometer, and the only conditioning is the mean removal that strips gravity. (The `AccelFilter` helper still exists for other uses; it simply is not in the FI path.)

Takeaway

Do not filter before the Freeze Index. Per-axis band-pass filtering ahead of the magnitude flattens the FI ratio so walking and freezing look alike — so the live detector reads *raw* acceleration, with only gravity removed.

This whole core — the `dsp` maths, the gated state machine, the streaming receiver, and the metrics — is held in place by a suite of **73 passing unit tests**. They pin the band-power sums, the scale-invariance of the Freeze Index, the gate-before-threshold ordering, and the debounce timing, so a future refactor cannot silently change what the device decides is a freeze.

15 The Live Dashboard: What Every Metric Means

Everything in this report — the magnitude trick, the Welch spectrum, the Freeze Index, the still-floor gate, the debounce — exists to drive *one screen*. When Cadence is on an ankle and the wearer takes a few steps, a clinician (or the wearer themselves) looks at a single web page and should be able to say, at a glance, “*the leg is walking calmly*,” “*the leg is shuffling in place — that is a freeze*,” or “*the leg is at rest*.” This section is a reading guide for that screen. It walks panel by panel, says in plain words what each number means, gives the *typical* values you see when the leg is still, walking, and freezing, and — most importantly — makes the arithmetic that links the panels explicit, so the dashboard reads as one coherent story rather than seven disconnected dials.

Key idea

The dashboard is a **glass box**, not a black box. Every panel is a quantity you can compute by hand from the raw trace, and the headline verdict is built from them by a rule you can recite: *the gate decides if the leg is moving at all; the Freeze Index decides whether that movement is a freeze; the debounce makes sure the verdict is not a one-off twitch*. Nothing on the screen is a learned weight you cannot inspect.

15.1 The screen at a glance

Figure 45 is an annotated mock of the live page. There are seven panels. One of them — *Gait Status* — is the answer; the other six are the *evidence* that produced it. The trick to reading the dashboard quickly is to know which panels are inputs and which is the output, and how the inputs flow into the output. That flow is always the same three steps, drawn as the spine of the mock:

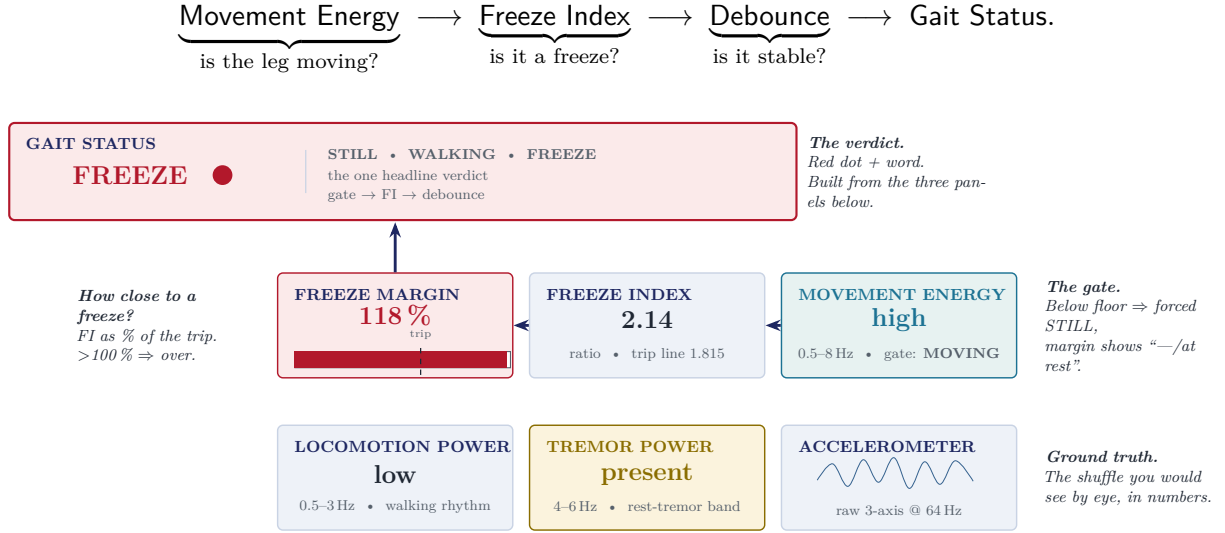


Figure 45. Annotated mock of the live web dashboard. The red GAIT STATUS banner is the single verdict; the six panels beneath it are the evidence. Heavy navy arrows trace the decision spine: *Movement Energy* (the gate) feeds the *Freeze Index*, which feeds the *Freeze Margin* meter and, after debounce, the verdict. Colours follow the report palette: teal = sensing/energy, amber = tremor, red = the freeze decision.

Intuition

Think of the panels as a courtroom. *Movement Energy* is the bailiff at the door: if the leg is barely moving, the case is dismissed before it starts and the verdict is **STILL**. *Freeze Index* is the prosecutor: given that the leg *is* moving, it argues whether the movement is the fast 3–8 Hz shuffle of a freeze or the slower swing of a walk. *Debounce* is the jury foreman, refusing to convict on a single noisy instant. *Gait Status* reads out the sentence.

15.2 The two quantities everything is built from

Before the panels make sense, two numbers must be clear, because every other panel is a slice or a combination of them. Both are computed on the mean-removed acceleration *magnitude* $a = \sqrt{a_x^2 + a_y^2 + a_z^2}$ over the current 4-second window, and both are sums of the Welch power spectrum over a frequency band.

The maths, unpacked

Band power is the area under the power spectrum across a band — literally how much wiggle energy lives at those frequencies. Cadence reads three bands:

$$P_{\text{loco}} = P(0.5\text{--}3\text{ Hz}), \quad P_{\text{freeze}} = P(3\text{--}8\text{ Hz}), \quad P_{\text{tremor}} = P(4\text{--}6\text{ Hz}).$$

From these, the two master quantities are

$$\text{FI} = \frac{P_{\text{freeze}}}{P_{\text{loco}} + 10^{-9}} \quad \text{and} \quad E_{\text{move}} = P_{\text{loco}} + P_{\text{freeze}}.$$

FI is a *ratio* (the freeze band *divided by* the walking band), so it is scale-invariant: it does not care whether you measure in mg or g. E_{move} is a *sum* (the two bands *added*), so it grows with

how hard the leg is actually moving and is *not* scale-invariant. The tiny 10^{-9} stops a perfectly still leg from dividing by zero.

This single pair of formulas is the Rosetta Stone for the whole dashboard:

- **Locomotion Power** is P_{loco} — the denominator of FI and one half of E_{move} .
- **Tremor Power** is P_{tremor} — a narrow sub-band sitting *inside* the freeze band.
- **Freeze Index** is FI — freeze band over loco band.
- **Movement Energy** is E_{move} — freeze band plus loco band.
- **Freeze Margin** re-expresses FI as a percentage of the trip line.
- **Gait Status** combines the gate (on E_{move}), FI, and debounce.

Key idea

Same two ingredients, different recipe. *Divide* the freeze band by the walking band and you get the **shape** of the motion (FI — *is this fast or slow?*). *Add* them and you get the **amount** of motion (E_{move} — *is the leg moving at all?*). The dashboard needs both: amount to know whether to bother, shape to know whether it is a freeze.

15.3 Panel by panel

Gait Status — the verdict

This is the only panel a hurried reader needs. It shows one of three words with a matching NeoPixel colour: **STILL** (the leg is at rest), **WALKING** (ordinary gait), or **FREEZE** (red — a freeze-of-gait episode is happening *now*). The verdict is not read off any single dial; it is the output of the three-step state machine:

1. **Gate.** If E_{move} is below the auto-calibrated still-floor, the leg is not really moving $\Rightarrow$ **STILL**. Stop here.
2. **Classify.** Otherwise, if $\text{FI} > 1.815$, the moving leg is shuffling fast $\Rightarrow$ candidate **FREEZE**; else $\Rightarrow$ candidate **WALKING**.
3. **Debounce.** A new candidate must persist for *two* consecutive 2-second control windows before it is committed (assert-after-2, release-after-2).

Intuition

Why debounce? One heel-strike, one stumble, one bump against a table leg can spike FI for a single window. Without debounce the banner would flicker FREEZE for two seconds every time the wearer kicked a kerb. Requiring two windows in a row means a real verdict is at most ~ 2 s late but is never a phantom.

Freeze Margin — the glass-box confidence

This meter is the newest panel and the most useful for “how worried should I be?”. It expresses the live FI as a percentage of the 1.815 trip line:

$$\text{margin} = \frac{\text{FI}}{1.815} \times 100\%.$$

At **100 %** the leg is exactly on the trip line; **above 100 %** it has crossed into freeze territory; well below 100 % the gait is comfortably calm. Crucially, when the gate has forced the state to **STILL**, the meter does *not* show a misleading small percentage — it shows “— / at rest”, because a freeze ratio computed on a motionless leg is meaningless noise.

Key idea

The Freeze Margin replaces the old, blank “probability” box. In glass-box (FI) mode the CNN is *bypassed* entirely, so there was never a neural-network probability to display — the box sat empty and confused people. The margin meter fills that slot with something the FI detector can actually justify: a transparent “percent of the way to a freeze” that you can recompute by hand from FI and 1.815.

Freeze Index — the raw ratio

The same FI as a bare number, with the trip line drawn at 1.815. A calm walk sits around $FI \approx 0.5$ –1; a genuine freeze pushes well above 2, and on real captures the freeze peaks reach into the tens (the worksheet capture peaked at 21.35). The dashboard caps the displayed scale so a single off-the-chart spike does not flatten the rest of the trace — the same off-scale behaviour visible in Figure 46, where one freeze window’s FI runs clean off the top of the axis.

Tremor Power — the rest-tremor band

This panel reports P_{tremor} , the power in the narrow 4–6 Hz rest-tremor band, with a **present** / **clear** badge. Parkinsonian rest tremor lives here, partly overlapping the lower freeze band. It is reported as a separate readout (not folded into the verdict) so a clinician can see tremor activity independently of the freeze decision — the leg can tremble without freezing, and vice versa.

Movement Energy — the gate

This is $E_{\text{move}} = P_{\text{loco}} + P_{\text{freeze}}$, the 0.5–8 Hz energy, shown with a **MOVING** / **STILL** badge that fires when the energy crosses the auto-calibrated still-floor. It answers one question only: *is the leg doing anything at all?* A still leg has near-zero energy; the gate is what stops sensor noise on a motionless ankle from producing a large, meaningless FI and a phantom freeze. Because E_{move} is a raw sum (not a ratio), it is the one panel whose absolute value genuinely tracks “how hard the leg is moving.”

Locomotion Power — the walking rhythm

This is P_{loco} , the 0.5–3 Hz power: the energy of the slow, rhythmic leg swing of ordinary walking. It is high during a steady walk and low at rest. It is also the *denominator* of FI — which is exactly why a freeze stands out: during a freeze the walking rhythm collapses (P_{loco} falls) while the fast shuffle grows (P_{freeze} rises), so their ratio FI shoots up.

Accelerometer waveform — the ground truth

The live raw 3-axis trace at 64 Hz. This is the unprocessed reality every other panel is distilled from: calm gait looks like clean periodic swings, a freeze looks like a burst of fast, low-amplitude trembling “in place,” and rest looks nearly flat. A skilled eye can read a freeze straight off this trace; the other panels exist to do that reading automatically and continuously.

15.4 Reading a real run off the screen

Figure 46 is a recorded smoketest, and it shows in one picture exactly what the dashboard animates live. The top trace is the accelerometer magnitude through four labelled phases; the bottom trace is the Freeze Index against its trip line. Walk the panels through it:

- **Phase 0 (rest).** The magnitude is nearly flat. *Movement Energy* is below the still-floor, so the gate fires, *Gait Status* reads **STILL**, and the *Freeze Margin* shows “—/at rest” — regardless of whatever noisy FI the bottom trace happens to show, because a gated leg is never called a freeze.

Table 12. Reading the dashboard. Each panel, what it measures, the one-line intuition, and the values you typically see when the leg is still, walking, or freezing. “band” refers to the Welch power spectrum of the mean-removed magnitude.

Panel	What it shows	Intuition	Still / Walk / Freeze
Gait Status	STILL / WALKING / FREEZE	The verdict: gate $\rightarrow$ FI $\rightarrow$ debounce	STILL / WALKING / FREEZE
Freeze Margin	FI as % of the 1.815 trip	How close to a freeze (glass-box confidence)	“—/at rest” / $\sim 30\text{--}55\%$ / $\geq 100\%$
Freeze Index	$FI = P_{\text{freeze}}/P_{\text{loco}}$	Fast shuffle vs. slow swing (a ratio)	gated / $\approx 0.5\text{--}1$ / >2 , up to 20+
Tremor Power	4–6 Hz band power	Rest-tremor activity, reported separately	clear / clear / present (often)
Movement Energy	$P_{\text{loco}} + P_{\text{freeze}}$ (0.5–8 Hz)	How hard the leg moves — the gate	STILL (low) / MOVING (high) / MOVING
Locomotion Power	0.5–3 Hz band power	The walking-rhythm energy	low / high / collapses
Accelerometer	raw 3-axis trace @ 64 Hz	The shuffle you would see by eye	flat / clean swings / fast tremble

- **Phase 1 (walking).** The magnitude breaks into large, regular swings. *Locomotion Power* (and hence *Movement Energy*) rises, so the gate opens and the leg is **MOVING**. But the energy is dominated by the slow walking rhythm, so FI mostly sits at or below the dashed trip line: *Gait Status* reads **WALKING**.
- **Phases 2–3 (freeze).** The magnitude turns into bursts of fast, lower-amplitude trembling. Now the freeze band swells while the walking band collapses, so FI spikes hard — one window even runs off the top of the axis (a freeze peak the figure marks as off-scale). The *Freeze Margin* crosses 100 %, and after two consecutive windows confirm it the debounce commits the verdict: *Gait Status* turns red, **FREEZE**.

Takeaway

To read the leg off the screen in three seconds: glance at **Movement Energy** (is it moving?); if yes, glance at the **Freeze Margin** (over 100 % = freeze); the **Gait Status** banner has already done both, plus debounce, and shows the answer in one word and one colour. Everything else on the page is there to let you check that answer by hand.

16 Results, Limitations, Future Work & Conclusion

The aim of this project was modest and specific: build a low-cost, ankle-worn device that *monitors* freeze of gait (FoG) and reports it honestly on a screen, rather than a black box that issues opaque verdicts. Everything in the preceding sections — the single accelerometer, the orientation-invariant magnitude, the Welch spectrum, the Freeze Index, and the glass-box detector — exists to serve that aim. This closing section asks the only question that matters: *does it work, and where does it fail?* We summarise the measured performance with real numbers, are candid about the limitations a single ankle sensor inevitably carries, sketch the upgrades we would pursue with more time, and tie the result back to the original goal.



Figure 46. A recorded smoketest, exactly the signals the dashboard animates live. *Top:* accelerometer magnitude $\|a\|$ in mg across four phases — flat at rest, big regular swings while walking, fast low-amplitude bursts during a freeze. *Bottom:* the Freeze Index against its trip line (dashed). At rest the gate forces STILL; walking keeps FI near or below the line; the freeze drives FI far above it, with one window’s peak running clean off the top of the axis. Reading the two traces together is exactly what the seven panels do, automatically, every two seconds.

Key idea

We report **sensitivity** (the fraction of true freezes caught) and **specificity** (the fraction of calm gait correctly left alone), *not* raw accuracy. Freezes are rare events, so accuracy flatters a lazy model: a detector that *never* flags a freeze can score ~ 0.90 accuracy yet 0.00 sensitivity — it is useless to a patient at risk of a fall. The two numbers that matter are the two we quote everywhere below.

16.1 What works — the headline numbers

Three detectors were evaluated, each answering a different “what if” question. The *shipped* detector is the glass-box Freeze Index rule running on the board. The *CNN* (FoGNet) and the *RandomForest* are the “given more time” upgrades, evaluated offline. Table 13 collects their operating points side by side.

Table 13. Measured operating points. The shipped FI rule is scored on the worksheet capture (`capture1.csv`, 19 windows: 6 freeze, 13 non-freeze); the CNN and the RandomForest are scored under leave-one-subject-out (LOSO) cross-validation on the public Daphnet FoG dataset. Sensitivity and specificity are the headline pair.

Detector	Evaluation	Sens.	Spec.	Confusion / ranking
FI rule, FI > 1.815 (shipped)	capture1 sweep	0.83	0.92	TP 5, FP 1, TN 12, FN 1
FI rule, FI > 2.10 (Youden)	capture1 sweep	0.83	1.00	TP 5, FP 0, TN 13, FN 1
FoGNet (1-D CNN, $\sim 18k$ params)	pooled LOSO	0.71	0.84	acc. 0.83; $n = 8982$
RandomForest (9 features)	nested LOSO	0.75	0.76	ROC-AUC 0.82, PR-AUC 0.33

Intuition

Read the FI rule's two rows as a dial, not a verdict. Raising the trip line from 1.815 to 2.10 removes the single false alarm (specificity climbs $0.92 \rightarrow 1.00$) *without losing any extra freezes* (sensitivity stays 0.83) on this capture. The Youden-optimal point is therefore 2.10 — yet the board deliberately ships the *lower* 1.815. The reason is asymmetric cost: on a fresh, unseen wearer a missed freeze can mean a fall, while a stray false alarm only lights an LED. We trade a little specificity for a margin of safety on people the device has never seen.

The shipped Freeze Index behaves exactly as the physics predicts. On `capture1.csv` the per-phase FI cleanly separates the three regimes: standing *still* sits at ~ 1.1 (max 1.71), *walking* at ~ 1.2 (max 2.08), and genuine *freezing* jumps to a mean of 6.89 (median 4.45, max 21.35). The freeze distribution lives almost entirely above both calm regimes, which is precisely why a single ratio threshold can separate them. The two false-classified windows in the sweep (one FP, one FN at $FI > 1.815$) are exactly the boundary cases one expects: a short freeze diluted by the 4s window, and a vigorous walking transient that momentarily borrowed freeze-band energy.

The CNN tells a complementary story. FoGNet was trained on Daphnet with leave-one-subject-out cross-validation — every test subject is held out entirely, so the score reflects generalisation to a *new person*, not to new windows from a known person. Its pooled confusion matrix (TN 6842, FP 1285, FN 248, TP 607) gives sensitivity 0.71, specificity 0.84, and accuracy 0.83 (Figure 47). That its accuracy and the FI rule's are not directly comparable is the whole point: they are measured on different data, under different protocols, and the CNN's honest LOSO sensitivity of 0.71 is the realistic figure for a model meeting a brand-new wearer.

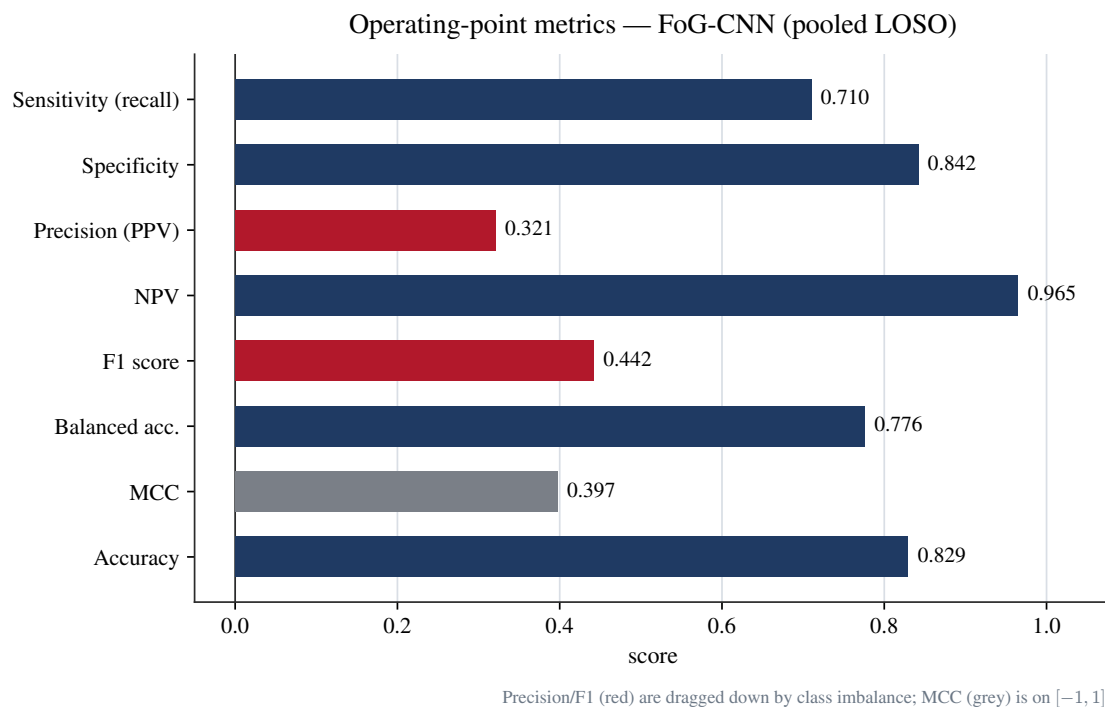


Figure 47. Operating-point metric panel for the FoGNet CNN under pooled leave-one-subject-out evaluation. Sensitivity (0.71) and specificity (0.84) are the headline pair; precision and F1 (red) are dragged down by class imbalance, and accuracy (0.83) is reported but *not* trusted, because on a rare-event problem accuracy hides exactly the errors that hurt a patient most.

Figure 48 breaks the CNN down per held-out subject and exposes the variance that the single pooled number averages away.

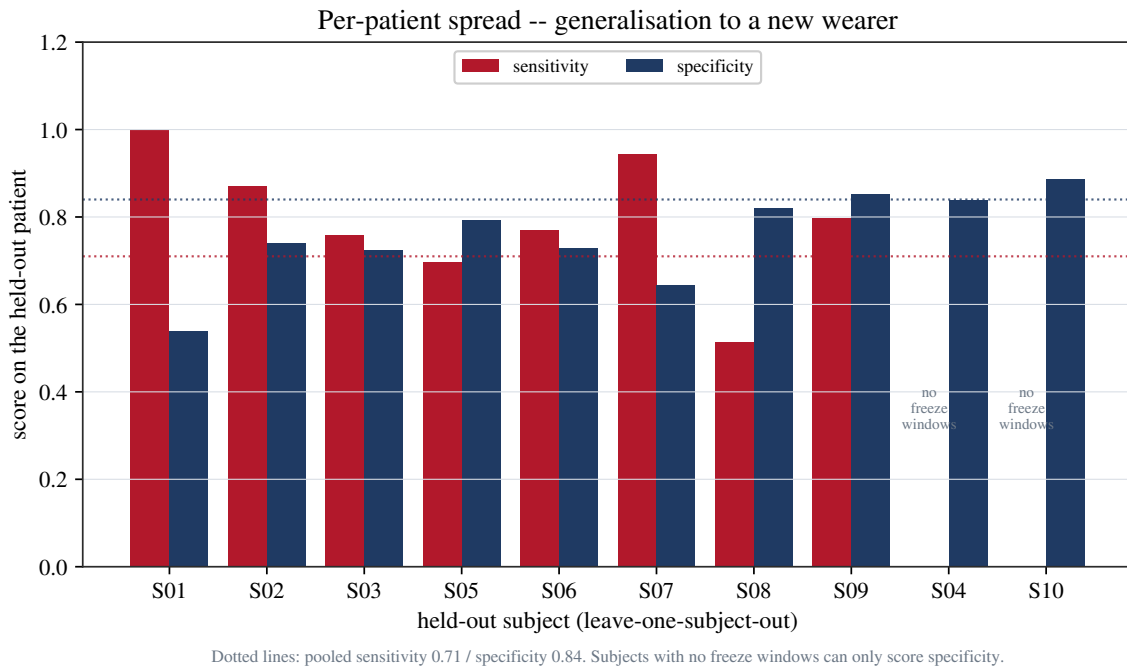


Figure 48. Per-subject LOSO performance. Each bar is one held-out wearer the model never saw in training. The spread between subjects — not the pooled average — is the honest measure of how this device behaves on someone new, and it is the single strongest argument for the per-patient calibration proposed in Section 16.

Takeaway

On its worksheet capture the shipped glass-box rule catches 5 of 6 freezes and raises just one false alarm (0.83 sensitivity, 0.92 specificity). The CNN, judged on the much harder “never met this person” LOSO protocol, reaches 0.71 / 0.84. Both detect FoG well; neither is perfect, and the gap between them is mostly the gap between “seen this signal” and “seen this person”.

16.2 Limitations — where Cadence falls short

A monitor that overstates its reach is worse than one that is candid about its blind spots. Cadence has several, and they follow directly from its design choices.

A single ankle sensor sees one thing.

One accelerometer on one ankle captures the trembling foot beautifully, but it is blind to the rest of the body. It cannot tell a freeze apart from a confounder that produces the same 3–8 Hz vibration somewhere else in the signal chain (see below), and it cannot disambiguate the kind of freeze that involves the trunk or the contralateral leg.

Standing still is not a freeze.

The tell-tale FoG signature is rapid 3–8 Hz trembling of the feet *in place*. A patient who is genuinely motionless — deliberately standing — produces almost no movement energy, so the still-floor gate forces the verdict to STILL and the FI is never consulted. This is correct behaviour, but it means a hypothetical “akinetik” freeze with no tremor would not register. Cadence detects *trembling* freezes, which are the common and fall-relevant kind, not every conceivable arrest of gait.

A small, personal dataset.

The shipped threshold was tuned on a single worksheet capture (19 windows). That is enough to demonstrate the principle and to set a sane default, but it is far too small to claim a population-level operating point.

The LOSO generalisation gap.

The CNN’s pooled LOSO numbers (0.71 / 0.84) are already the cautious figures, yet Figure 48

shows real subject-to-subject variance underneath them. A model tuned to the average wearer will under-serve those at the tails of that distribution.

The Freeze Index has confounders.

FI is a ratio of freeze-band to locomotor-band power. *Anything* that injects 3–8 Hz energy can inflate it: voluntarily shaking the leg, the vibration of a moving vehicle, or simply picking the device up and handling it. The still-floor gate and the assert-after-2 / release-after-2 debounce suppress the brief and the low-energy versions of these, but a sustained, leg-borne 3–8 Hz vibration is, to a single accelerometer, indistinguishable from a tremor.

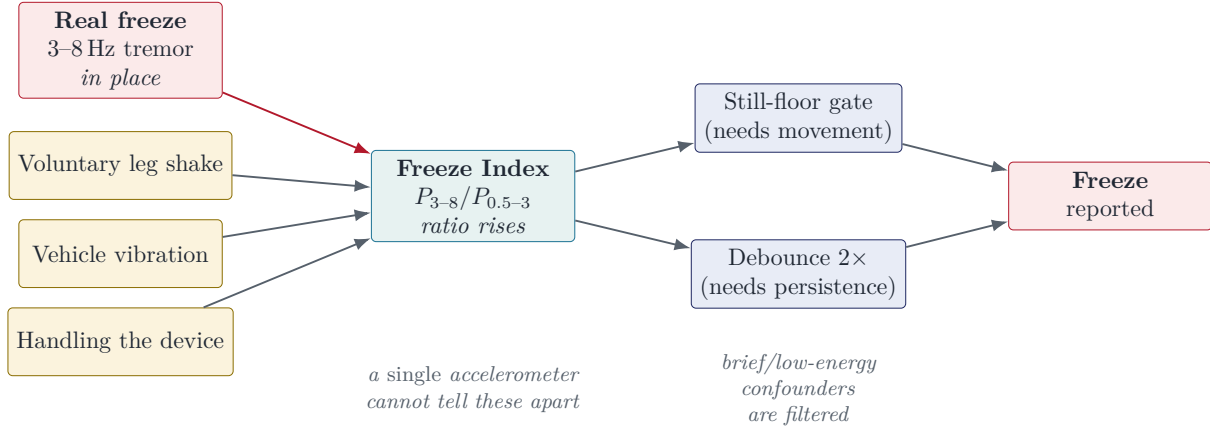


Figure 49. The confounder problem and the two guards that mitigate it. A real freeze and several confounders all raise the Freeze Index because each injects 3–8 Hz energy. The still-floor gate and the two-window debounce reject the brief and the motionless ones, but a *sustained, leg-borne* 3–8 Hz vibration remains, to one ankle sensor, indistinguishable from a tremor.

16.3 Future work

The limitations above are not dead ends; each maps onto a concrete next step, ordered roughly from “cheapest and most impactful” to “a significant programme in its own right”.

1. **Per-patient threshold calibration.** The single biggest win. Because FI is scale-invariant but its *distribution* shifts between people (Figure 48), a brief enrolment — a minute of calm walking to learn a wearer’s baseline FI and movement energy — would let the trip line move away from a one-size-fits-all 1.815 toward that wearer’s own Youden-optimal point. This is data the device already computes at boot for the still-floor gate; extending it to the FI threshold is mostly bookkeeping.
2. **Sensor fusion with a gyroscope.** Adding angular velocity gives the system a second, partly independent view of the same motion. A real in-place tremor has a characteristic rotational signature that a whole-body vehicle vibration does not, which directly attacks the confounder problem of Figure 49.
3. **The CNN on-device.** FoGNet runs in under a millisecond on a laptop CPU and has only ~18k parameters. Quantised and converted with TensorFlow Lite for Microcontrollers, it is a realistic fit for a more capable host such as an ESP32-S3, turning the offline “given more time” model into a second on-device opinion alongside the glass-box rule.
4. **More body sites.** A second sensor — the other ankle, the waist, or the thigh — would let the device distinguish bilateral from unilateral events and would help separate true gait freezes from local leg artefacts.
5. **A regulated closed-loop cueing version.** Cadence already carries a coin vibration motor, fitted but *disabled* in firmware by design. A future, properly regulated device could close the loop and deliver a rhythmic cue when a freeze is detected — the well-evidenced intervention for FoG. We flag this explicitly as a *later* step: turning a monitor into a therapy is a regulatory and clinical undertaking, not a firmware flag, and it is out of scope for a one-week wearable sprint.

Intuition

Notice the throughline: every future item buys back a specific limitation. Calibration answers the LOSO gap; the gyro answers the confounders; on-device CNN and extra sites answer the single-sensor blind spot; and the cueing version answers the unspoken wish behind any monitor — to eventually *help*, not just watch. We have ordered them so that the cheapest fixes attack the most common failure first.

16.4 Conclusion

We set out to build a low-cost, ankle-worn *monitor* for freeze of gait — a device that detects the tell-tale 3–8 Hz trembling, reports it transparently on an OLED, an LED ring, and a live dashboard, and explains *why* it decided what it decided. Cadence does this. On its worksheet capture the shipped glass-box Freeze Index rule catches 5 of 6 freezes against a single false alarm (sensitivity 0.83, specificity 0.92), and the cleaner sensitivity-plus-specificity optimum sits at $FI > 2.10$ — we ship the more cautious 1.815 on purpose, because a missed freeze can mean a fall. The offline CNN, held to the demanding “never met this person” LOSO standard, reaches a realistic 0.71 / 0.84, and a 9-feature RandomForest confirms the signal is learnable and interpretable (0.75 / 0.76, ROC-AUC 0.82).

None of these numbers is perfect, and we have been deliberate about saying so: one ankle sensor, a small personal dataset, a real generalisation gap, and a Freeze Index that any 3–8 Hz vibration can fool. But the device is honest about all of it. It reports the headline pair that actually matters for a rare, dangerous event; it shows the wearer how close they are to the trip line rather than a bare verdict; and it keeps its therapy motor switched off until a properly regulated successor could earn the right to use it. Within the scope of a one-week ENGF0031 “Smart Clothing” sprint, Cadence meets its aim: an affordable, transparent, untethered freeze-of-gait monitor whose every decision can be traced back to a physical quantity you can point at on the screen.

Takeaway

Cadence is a *monitor*, and a good one: it catches most freezes, leaves calm gait alone, and never hides its reasoning. Its honesty about what it cannot yet do — the single sensor, the small dataset, the confounders — is not a weakness of the report but the foundation of the roadmap that would make it a clinical tool.

Appendix A Signal-processing parameters

Every quantitative choice in the live pipeline is collected in Table 14 so the device can be reproduced exactly. The values are fixed in `fog.config` and mirrored in the firmware; the only field that adapts at runtime is the still-floor, auto-calibrated from resting energy at boot.

Table 14. The complete signal-processing parameter set for the shipped detector.

Parameter	Value	Role
Sampling rate f_s	64 Hz	Nyquist 32 Hz covers every band of interest
Accelerometer range	± 8 g	heel-strike exceeds ± 2 g, so ± 2 g would clip
Window length N	256 samples (4.0 s)	buys spectral resolution for the 3–8 Hz band
Hop length	128 samples (2.0 s)	50% overlap $\Rightarrow$ one decision every 2 s
Bin spacing $df = f_s/N$	0.25 Hz	spectral resolution of the Welch PSD
Spectral estimator	Welch, Hann taper	on the mean-removed magnitude
Locomotor band	0.5–3 Hz	walking-rhythm energy (FI denominator)
Freeze band	3–8 Hz	FoG tremble energy (FI numerator)
Rest-tremor band	4–6 Hz	reported separately as tremor power
Freeze Index threshold	1.815	deployed trip line (Youden optimum is 2.10)
Still-floor gate	auto (boot)	forces STILL when energy below floor
Debounce	2 windows	assert-after-2 / release-after-2

Appendix B Glossary

FoG — Freeze of Gait

A sudden, brief inability to step (“feet glued to the floor”), common in mid/late Parkinson’s and a major fall risk; its signature is rapid 3–8 Hz trembling of the feet in place.

FI — Freeze Index

The ratio $P_{3-8\text{ Hz}}/P_{0.5-3\text{ Hz}}$ of the mean-removed magnitude (Moore et al. 2008). Being a ratio, it is scale-invariant: mg versus g cancels.

LOSO — Leave-One-Subject-Out

Cross-validation in which an entire subject is held out for testing, so the score measures generalisation to a *new person*, not merely to new windows from a known person.

Sensitivity

$TP/(TP + FN)$ — the fraction of true freezes the detector catches.

Specificity

$TN/(TN + FP)$ — the fraction of calm (non-freeze) windows correctly left alone.

PSD — Power Spectral Density

How signal power is distributed across frequency; here estimated with a Hann-tapered Welch method. Band power is the PSD summed over a band’s bins.

GAP — Global Average Pooling

In FoGNet, the operation that collapses each of the 64 feature channels to a single number, yielding a fixed-size descriptor regardless of input length.